TEST AUTOMATION FRAMEWORK

Playwright API Framework

Complete design, file and change-management documentation

VERSION	GENERATED
1.0	2026-09-04

FILES DOCUMENTED	EXPORTS EXTRACTED
90	150

Contents

Overview

What this framework is, and how to read this document

This is a **production-grade API automation framework** built on Playwright Test and TypeScript. It covers REST, GraphQL, Server-Sent Events, NDJSON streaming, WebSockets and SOAP/XML, with contract validation, seven authentication schemes, latency measurement, security auditing, a programmable stub server, and a reporting pipeline designed for pull requests rather than for a person watching a terminal.

It ships with **49 working tests across four real, public APIs** — a genuine create-read-update-delete lifecycle, `Link` header pagination, client verification against a neutral echo server, OpenAPI conformance, latency budgets and security auditing. `npm ci && npm test` produces a meaningful result on a fresh clone, with no credentials and no configuration.

What this document is

A complete reference. **Every file in the repository has an entry** — what it is for, when you would need to change it, exactly how, and why the change belongs in that file rather than somewhere else. The build fails if a file has no entry, so this cannot quietly fall behind the code.

It is generated by `npm run docs`, which reads the TypeScript source with the compiler API, merges the extracted API surface with the prose in `scripts/docs/content/`, and emits this HTML site plus a PDF of the same material.

How to read it

IF YOU WANT TO…	GO TO
See the framework actually working	**Worked example** — the CRUD suite against a live API, explained line by line
Understand how the pieces fit together	**Architecture** — layers, the path of a request, and why each boundary exists
Find the file that owns a concern	**Project structure** — every file, grouped by role, linked to its entry
Change something	**Playbooks** — thirty common tasks, each with the file, the steps and the reasoning
Know what a symbol does	**API index** — every exported symbol, filterable, linked to its definition
Understand a failure	**Troubleshooting** — failure modes, what causes them and how to diagnose them
Write a test somebody else can maintain	**Conventions** — naming, tags, anti-patterns and the review checklist

The ideas the framework is built on

Six decisions shape everything else. Each is argued where it applies, but they are worth stating up front because they explain why the code looks the way it does.

1. **One place per concern.** The environment is read in exactly one file. Requests are sent in exactly one file. An endpoint's path is known to exactly one service object. Change is then a single edit, and the compiler finds everything affected.

2. **A response is a snapshot, not a stream.** The body is captured into a buffer before the response object exists, so it can be read as text, JSON, NDJSON, XML or bytes, in any order, as many times as a test likes — and every assertion can be synchronous and chainable.
3. **Failures must explain themselves.** Every assertion reports the request, the status, the timing and a body excerpt. An error message is usually all anyone sees in CI, and a message that costs nothing to write saves somebody twenty minutes.
4. **Retry transport faults, never verdicts.** A connection reset is worth retrying; a 422 is an answer. The two are separated structurally in the client, not by convention.
5. **Generate data, deterministically.** Faker is seeded from each test's own title, so data is unique between tests and identical between runs — random enough to catch collisions, reproducible enough to debug.
6. **Clean up at the point of creation.** A service method that creates a resource registers its own deletion in the same statement. Cleanup that depends on somebody remembering is cleanup that does not happen.

What is in the box

AREA	CAPABILITY
Request	Fluent builder: path params, query with three array encodings, six body encodings, per-request timeout, retries, accepted statuses, idempotency keys, anonymous mode, redirect control
Transport	One engine: credential injection, exponential backoff with jitter and `Retry-After`, read-only guard, timing capture, reporter steps, exchange listeners
Response	JSON / NDJSON / XML / text / binary readers, JSON-path navigation, Zod parsing, 15 assertion methods, soft mode, report attachment
Auth	None, Basic, Bearer, API key (header or query), OAuth2 client-credentials / password / refresh, HMAC request signing, cookie session, JWT decoding
Contracts	Zod schema library, JSON Schema via Ajv, OpenAPI conformance with `$ref` resolution, a path-pattern registry, and automatic enforcement under `STRICT_CONTRACTS`
Protocols	GraphQL (queries, mutations, batching, introspection, error assertions), SSE, NDJSON streaming, WebSocket with pre-buffering
Testing support	Stub server with delays, failures and controlled flakiness; exchange recorder for offline replay; latency percentiles; structural diffing; security audits; pagination walkers; polling
Reporting	HTML, JUnit, JSON, CTRF, optional Allure, and a custom machine-readable run summary

Getting started

First run — no configuration required SHELL

```
npm ci
npm test
```

That is the whole setup. Copy `.env.example` to `.env` only when you point the framework at your own API.

Useful selections SHELL

```shell
npx playwright test --grep-invert @live   # everything that spends no API quota
npx playwright test --project=contract    # offline: no network at all
npx playwright test --grep @smoke         # the pre-deploy subset
```

Then read the **Worked example**, then **Architecture**, then write your first spec following **Conventions**. `make` lists every other command.

What it is tested against

Four real services, each chosen for what it alone can prove.

API	PROVES	LIMITS
`api.restful-api.dev`	A real CRUD lifecycle — the only one of the four that genuinely persists writes	**50 anonymous requests / 24 h**; the live suite skips rather than fails
`jsonplaceholder.typicode.com`	Real `Link` header pagination over a stable 100-record dataset	None. Writes are simulated
`httpbin.org`	That the client puts on the wire what it claims to	None
The built-in stub server	The same lifecycle offline, plus failure modes no real API produces on demand	None

> **NOTE** There is no free, unauthenticated API offering both genuine persistence and an unlimited quota — several were tried and rejected, and the **Worked example** records which and why. So the CRUD suite is split: a small live suite proves the API behaves as we believe, and an unlimited stubbed suite runs the same service object against an executable model of the same contract on every run.

Architecture

Layers, the path of a request, and the reasoning behind each decision

The framework is a stack of layers, each of which knows only about the layer beneath it. A test at the top can reach down as far as it needs, but nothing reaches upward — which is what makes any single layer replaceable without touching the others.

The layers

```
tests/                 what should be true
  ↓
src/fixtures/          the test and expect a spec imports; wiring and lifecycle
  ↓
src/services/          one object per resource: paths, payloads, domain methods
  ↓
src/core/              the request builder, the HTTP engine, the response wrapper
  ↓  ↘
  │    src/auth/         credentials, injected per request
  │    src/contracts/    schemas, JSON Schema, OpenAPI conformance
  │    src/protocols/    GraphQL, SSE, NDJSON, WebSocket
  │    src/mocks/        stub server and exchange recorder
  ↓
src/config/            the only reader of process.env, validated at start-up
  ↓
Playwright APIRequestContext → the network
```

The path of a request

Following one call all the way down is the fastest way to understand the framework. This is what happens when a test writes `await api.users.create()`.

1. **The fixture builds the client.** `src/fixtures/api.fixture.ts` constructs an `HttpClient` bound to Playwright's `request` context and to whichever auth provider the environment implies, then attaches the latency collector and the exchange recorder to it.

2. **The service composes the request.** `UserService.create` builds a payload from the seeded factory and describes the call — path, body, idempotency key, acceptable status — through the fluent builder. Nothing has been sent yet.

```ts
const response = await this.post()
  .json(payload)
  .idempotencyKey()
  .expectStatus(201)
  .as('create user')
  .send();
```

3. **The builder resolves the request.** `RequestBuilder.build()` fills path placeholders, encodes the query, decides the content type from the body kind, and returns an immutable `RequestSpec`. A placeholder left unfilled fails here, with a message naming it.

4. **The client applies every cross-cutting policy.** `HttpClient.dispatch` refuses mutating verbs on a read-only environment, opens a reporter step, merges credentials from the auth provider under the request's own headers, and sends.

5. **The transport runs, and only the transport is retried.** A connection failure or a retryable status backs off — honouring `Retry-After` when the server sent one — and tries again. Everything after a response has arrived sits outside that catch, so a schema violation is never retried.

6. **The response is captured whole.** Status, headers and the complete body buffer become a `ResponseSnapshot`, the underlying response is disposed, and an `ApiResponse` wraps it.

7. **Observers see it.** The latency collector records the timing, the recorder stores a redacted copy, and — when `STRICT_CONTRACTS` is on — the contract guard validates the payload against its registered schema and throws if it disagrees.

8. **The service returns a domain value.** `response.parse(UserSchema)` validates and narrows the type, the deletion is registered with the cleanup registry, and the test receives a fully typed `User`.

```
const user = response.parse(UserSchema, 'user');
return this.track(user, `user ${user.id}`, () => this.remove(user.id));
```

Why a builder rather than an options object

A request has more than a dozen independent dimensions. Expressed as one options object, every call site has to be read in full to find the two fields that matter, and adding a thirteenth dimension is a breaking change for everybody. A builder lets each call name only what it cares about, in any order, and lets a new capability be added without touching a single existing test.

The builder is also **thenable**, which is what keeps the simple case simple: `await http.get('/users')` sends the request, while `await http.get('/users').query({ page: 2 }).timeout(5000)` sends a different one. Nothing is sent until the builder is awaited.

Why the response is a snapshot

Most HTTP clients hand back a stream that can be consumed once. That forces every test to decide up front whether it wants text or JSON, makes a second look at the body impossible, and turns every assertion into an `await`.

Capturing the body into a buffer before the wrapper exists removes all three problems at once. It costs one buffer per request — irrelevant for an API suite — and buys synchronous, chainable, re-readable assertions.

What the snapshot makes possible TYPESCRIPT

```typescript
const response = await http.get('/report').send();

response
  .expectOk()
  .expectContentType('application/json')
  .expectPath('summary.total', 42)
  .expectFasterThan(500);

const rows = response.ndjson<Row>();        // same body, different reader
const raw = response.text();                // and again
const checksum = createHash('sha256').update(response.buffer()).digest('hex');
```

Why service objects

A service object is to an API suite what a page object is to a UI suite. Its value is not abstraction for its own sake — it is that when `/v1/users` becomes `/v1/accounts`, exactly one file changes rather than every test that happened to mention users.

RULE A service method returns a domain value and never asserts. Assertions belong in tests, so a negative-path test can reuse the same service instead of working around it. Where a test genuinely needs the response — a `Location` header, a status, a timing — the service exposes a `raw*` method rather than forcing the test to bypass it.

Fixtures and scope

Playwright constructs a fixture only when a test names it, so the fixture file can grow indefinitely without slowing anything down: a test that never mentions `socket` never opens a WebSocket.

SCOPE	BUILT	USE FOR	EXAMPLES HERE
worker	once per worker process	expensive, shareable, effectively immutable things	`tokens`, `workerRequest`, `mockServer`, `contract`
test	once per test	anything a test can mutate, and anything that must not leak between tests	`http`, `api`, `cleanup`, `data`, `latency`, `recorder`

CAREFUL Putting mutable state in worker scope is how a suite becomes order-dependent: it passes locally with one worker and fails in CI with eight, and the failure moves every run. If a test can change it, it belongs in test scope.

Authentication

Every provider implements one interface — `headers(): Promise<HeaderMap>` — called per request, so a token that refreshes mid-run is picked up without rebuilding anything. `defaultAuthProvider` chooses between them from what the environment supplies, in order: OAuth2 if a token endpoint is configured, then an API key, then Basic credentials, then nothing.

That ordering is what makes moving an environment from API keys to OAuth a `.env` change rather than a code change.

Tokens are cached in a `TokenStore` keyed by grant and subject, and are treated as expired thirty seconds early. Both details matter: a token endpoint is usually rate-limited harder than the API, and a token that expires mid-flight produces a 401 nobody can reproduce.

Contracts

Three mechanisms, because they answer three different questions.

MECHANISM	ANSWERS	WHERE
Zod schemas	"Is this the shape the test expects?" — and gives the test a type as a by-product	`src/contracts/schemas.ts`, and each service
JSON Schema (Ajv)	"Does this satisfy a schema somebody else published?"	`src/contracts/json-schema.ts`
OpenAPI conformance	"Does the API still match its own published document?"	`src/contracts/openapi.ts`

The registry ties them to traffic. A schema registered against `GET /users/{id}` is found for a real request to `/v1/users/42`, and with `STRICT_CONTRACTS=true` every matching response is validated whether or not the test asked. That is on in CI and off locally, so a work-in-progress schema does not block a developer but drift cannot reach main.

Projects

`playwright.config.ts` defines five projects. They exist because these groups of tests have genuinely different requirements, not to tidy the file listing.

PROJECT	WHY IT IS SEPARATE
`setup`	Authenticates once and writes the session to `storage/`. Everything else declares `dependencies: ['setup']`, so nothing runs unauthenticated.
`api`	The functional suite. Fully parallel.
`contract`	Answers a different question — "does this still match what we published" — and usually runs on a different trigger. It can run against the stub server with no environment at all.
`performance`	One worker, no retries. A percentile measured while eight workers hammer the same endpoint measures the load, not the endpoint.
`security`	Deliberately hostile payloads. Separated so a pipeline can point it at a dedicated environment and never at production.

What a run leaves behind

ARTEFACT	PATH	AUDIENCE
HTML report	`reports/html/`	A person investigating a failure
Run summary	`reports/summary.json`	A pipeline gate, or a chat message
JUnit XML	`reports/junit/results.xml`	CI test-result panels
CTRF JSON	`reports/ctrf/`	Cross-tool result aggregation
Recordings	`reports/recordings/`	Failed tests only — the full exchange, redacted
Traces	`test-results/`	Every request and response, replayable

Testing against an API you cannot call freely

The CRUD target allows 50 anonymous requests a day. That constraint produced a pattern worth generalising, because most teams meet some version of it — a rate limit, a shared sandbox, a dependency that is down half the time.

SUITE	RUNS AGAINST	PROVES	HOW OFTEN
`tests/api/objects.crud.spec.ts`	The live API	The API really behaves as we believe	A few times a day, within quota
`tests/contract/objects.stubbed.spec.ts`	`src/mocks/objects.stub.ts`	Our client handles that behaviour correctly	Every run, offline, unlimited

Both drive **the same `ObjectService`**. That is what makes the arrangement work: the stub is not a parallel implementation to keep in step, it is a server the real client talks to.

> **RULE** A model nobody checks against reality is a fixture that encodes an assumption and then defends it forever. The live suite is what keeps the model honest; the model is what makes the coverage affordable. Neither is sufficient alone.

The stub also buys something the live API cannot give at any quota: failure modes on demand. A real service will not produce a 503, a two-second delay or a malformed body when you ask it to, so those paths would otherwise never be tested.

Deliberate omissions

Things that were considered and left out, so nobody has to re-derive the reasoning.

- **No load testing.** Latency percentiles from a functional suite are useful signal; pretending they are a load test is not. Use a tool built for it.
- **No JWT signature verification.** Verifying needs the issuer's key and is the API's job. The suite decodes claims — it never treats a decoded token as trusted.

- **No external `$ref` resolution in OpenAPI.** Bundle the specification with your own tooling first; a second resolver that behaves slightly differently from the team's is worse than none.
- **No authorisation-code OAuth flow.** It needs a browser. The UI suite can perform it and hand the resulting token to `.bearer(token)` on the request builder.
- **No assertion retrying.** `retry()` exists for genuinely transient work. Retrying until an assertion passes is how a real defect gets shipped.

Worked example

The CRUD suite against a real API, explained line by line

A line-by-line reading of the CRUD suite in `tests/api/objects.crud.spec.ts`, which runs against **https://api.restful-api.dev** — a live public API that genuinely persists writes. Every call is explained: what it does, why it is there, and what would go wrong without it.

If you read one page of this document, read this one. Everything else is reference; this is the framework doing its job.

The APIs, and why each was chosen

Finding a suitable target was itself a piece of work worth recording. **There is no free, unauthenticated public API that offers both genuine persistence and an unlimited quota.** Several were tried and rejected:

CANDIDATE	OUTCOME
`api.restful-api.dev`	**Chosen.** Real persistence, no auth — but 50 anonymous requests per 24 hours.
`jsonplaceholder.typicode.com`	**Chosen for reads.** Unlimited and stable, with real `Link` header pagination — but writes are simulated and store nothing.
`httpbin.org`	**Chosen for client verification.** Echoes back exactly what it received.
`petstore3.swagger.io`	Rejected — its demo instance answers 500 to every write.
`api.escuelajs.co`	Rejected — 500 on create.
`dummyjson.com`	Rejected — writes are simulated; a created id 404s on read.
`gorest.co.in`	Rejected — writes need a bearer token, so it cannot run on a fresh clone.

So the suite is split. The live file proves the API behaves as we believe, a few times a day. `tests/contract/objects.stubbed.spec.ts` runs the **same service object** against an executable model of the same contract, unlimited and offline, on every run. If the two disagree, either the API changed or we misread it.

Step 1 — the imports

tests/api/objects.crud.spec.ts TYPESCRIPT

```typescript
import { test, expect } from '../../src/fixtures';
import { uniqueId } from '../../src/utils/data.utils';
import {
  ApiObjectSchema,
  CreatedObjectSchema,
  DeleteAcknowledgementSchema,
} from '../../src/services/object.service';
```

RULE `test` and `expect` come from `../../src/fixtures`, **never** from `@playwright/test`. The Playwright versions have no cleanup registry, no contract guard and none of the custom matchers — and nothing will tell you. This is the single most expensive mistake available in the repository.

The schemas are imported from the service rather than redefined, so the test and the service can never disagree about what a valid object looks like.

Step 2 — the quota guard

Skipping is not failing TYPESCRIPT

```typescript
let quotaProbe: Promise<boolean> | undefined;

test.beforeEach(async ({ api }) => {
  quotaProbe ??= api.objects.quotaAvailable();

  test.skip(
    !(await quotaProbe),
    'The restful-api.dev anonymous quota (50 requests / 24h) is exhausted. ' +
      'The same lifecycle is covered offline by tests/contract/objects.stubbed.spec.t
  );
});
```

- **Why a module-level variable?** The probe costs a request. Caching it means one probe per worker process rather than one per test.
- **Why cache the promise, not the result?** Concurrent tests in the same worker then share a single in-flight request instead of racing to make several.
- **Why skip rather than fail?** A red build caused by somebody else having spent the day's quota teaches nobody anything, and a suite people learn to ignore is worse than no suite.
- **Why `??=`?** It assigns only when the variable is still undefined — the concise form of "memoise this".

Step 3 — generating the payload

TYPESCRIPT

```typescript
function deviceFor(label: string): { name: string; data: Record<string, unknown> } {
  return {
    name: `${label} ${uniqueId('pw')}`,
    data: {
      year: 2026,
      price: 1849.99,
      'CPU model': 'Apple M4 Pro',
      'Hard disk size': '1 TB',
    },
  };
}
```

- `uniqueId('pw')` appends a timestamp and random suffix, so two parallel workers cannot produce the same name. A hard-coded name would collide the moment the suite ran twice.
- The `data` keys include one containing a **space**. That is a shape this API really accepts, and a useful check that nothing in the framework mangles keys on the way out.

Step 4 — CREATE

The test TYPESCRIPT

```typescript
const created = await test.step('create the object', async () => {
  const object = await api.objects.create(device);

  expect(object.id, 'the API must assign an id').toBeTruthy();
  expect(object.name).toBe(device.name);
  expect(object.data).toMatchObject(device.data);
  expect(object.createdAt).toBeGreaterThan(Date.now() - 120_000);

  return object;
});
```

And what the service does underneath TYPESCRIPT

```typescript
const response = await this.post()
  .json({ name: input.name, data: input.data ?? null })
  .idempotencyKey()
  .expectStatus(200)
  .as('create object')
  .send();

const created = response.parse(CreatedObjectSchema, 'object-created');
return this.track(created, `object ${created.id}`, () => this.remove(created.id));
```

CALL	WHAT IT DOES	WHAT BREAKS WITHOUT IT
`this.post()`	A builder for `POST /objects`, pre-labelled for the report	
`.json(...)`	JSON body, `Content-Type: application/json` set automatically	
`.idempotencyKey()`	Generates a key so the POST is safe to retry	The client refuses to retry a POST at all — a transient 503 on a create becomes a hard failure
`.expectStatus(200)`	Enforces the status the API **really** returns	Expecting 201 would raise on every single create
`.as(...)`	Names the call in the report, logs and recording	Failures identifiable only by URL
`.send()`	Issues the request; nothing happens until now	
`.parse(schema)`	Validates **and narrows** the type	`json<T>()` would assert a shape rather than check one
`this.track(...)`	Registers the deletion at the moment of creation	A leaked object per run, found by somebody else months later

NOTE Asserting the echo — that `name` and `data` came back as sent — matters more than it looks. An API that silently drops a field on write is a real defect, and a create test that only checked the status would never see it.

The `createdAt` check proves the server generated a timestamp *now* rather than echoing something stale. It is epoch milliseconds, and it appears on the create response and nowhere else — a quirk the schemas encode by splitting `CreatedObject`, `Object` and `UpdatedObject` into three.

Step 5 — READ, and why it is not optional

```typescript
const fetched = await api.objects.require(created.id);

expect(fetched.id).toBe(created.id);
expect(fetched.name).toBe(created.name);
expect(fetched.data).toMatchObject(device.data);
expect(fetched, 'reads carry no createdAt').not.toHaveProperty('createdAt');
```

RULE The read is what proves the write actually persisted. Without it, a create test passes just as happily against an API that accepts the request and throws it away.

`require` is the throwing form; `find` returns `undefined` for a 404 so a caller can branch on absence. Having both means neither a `try`/`catch` nor a null check is ever forced on the test.

The final assertion records a real quirk rather than glossing over it: reads carry **no timestamps at all**, even though the create carried `createdAt`. A single schema requiring it would fail on every read — which is exactly the mistake a schema copied from documentation would make.

Step 6 — REPLACE, and the assertion most people forget

```typescript
const replaced = await api.objects.replace(created.id, {
  name: `MacBook Pro M5 ${uniqueId('pw')}`,
  data: { year: 2027, price: 2199.99, 'CPU model': 'Apple M5 Max' },
});

expect(replaced.updatedAt).toBeGreaterThan(0);

const afterReplace = await api.objects.require(created.id);
expect(afterReplace.data).not.toHaveProperty('Hard disk size');
```

The last line is the whole point of testing PUT. A full replacement must **discard** what it was not sent — `Hard disk size` was present before and absent from the replacement, so it must be gone rather than merged. A test that only asserted the status, or only that the new fields arrived, would pass against a server that quietly merged instead.

`updatedAt` appears here and not on creates — the mirror image of the `createdAt` quirk, and the reason `UpdatedObjectSchema` exists.

Step 7 — PATCH, the other half of that distinction

TYPESCRIPT

```typescript
const patched = await api.objects.update(created.id, { name: patchedName });

expect(patched.name).toBe(patchedName);
expect(patched.data, 'PATCH must not clear untouched fields').toMatchObject({
  'CPU model': 'Apple M5 Max',
});
```

Only `name` was sent, so `data` must survive. That is precisely what distinguishes PATCH from PUT, and precisely what a test asserting only the status would fail to check. The two assertions together — PUT discards, PATCH preserves — are what make the pair meaningful.

Step 8 — DELETE, and the follow-up read

TYPESCRIPT

```typescript
await api.objects.remove(created.id);

const afterDelete = await api.objects.find(created.id);
expect(afterDelete, 'the object must be gone after DELETE').toBeUndefined();
```

A delete test without the follow-up read proves only that the API accepted the request, which is not the same thing as the resource being gone.

And the service call that makes cleanup safe — TYPESCRIPT

```typescript
await this.del('/{id}').param('id', id).expectStatus(200, 404).send();
this.cleanup.forget(`object ${id}`);
```

404 is accepted because `remove` also runs from the cleanup registry, after tests that already deleted their own object. If a repeated delete failed, every such test would go red in teardown for no reason. `forget` then removes the registration so teardown does not try a third time.

What the framework did that the test never mentions

HAPPENED AUTOMATICALLY	WHERE IT COMES FROM
A client was built and authenticated for the environment	the `http` fixture
Every request became a step in the HTML report	`HttpClient.dispatch`

HAPPENED AUTOMATICALLY	WHERE IT COMES FROM
Every request and response was timed and bucketed by route	the `latency` fixture, attached as `latency.json`
Every exchange was recorded, redacted, and saved **only because a failure would need it**	the `recorder` fixture
Every response was validated against its registered schema when `STRICT_CONTRACTS=true`	the `contractGuard` auto fixture
The created object was deleted, even if the test failed	the `cleanup` fixture draining the registry
A transient 503 would have been retried with backoff; a 404 would not	`HttpClient.execute`
A mutating verb against a read-only environment would have been refused before sending	`guardReadOnly`

That list is the argument for a framework rather than a folder of scripts. None of it appears in the test, none of it can be forgotten, and all of it applies to the next test somebody writes.

Two real bugs this work found

1. Text bodies were being re-encoded

A test sending a deliberately malformed body — to prove the API rejects it — was passing for the wrong reason. Playwright re-serialises a **string** `data` when the content type is JSON, so `.text('{ not json', 'application/json')` arrived at the server as the valid JSON string `"{ not json"`.

The stub could not see it, because the stub received exactly what the transport chose to send. Only the neutral echo server could. The fix, in `http.client.ts`, is to send text as a Buffer, which is written verbatim:

TYPESCRIPT

```typescript
case 'text':
  options.data = Buffer.from(spec.text ?? '', 'utf8');
  break;
```

2. A flaky assertion of my own making

A security test searched a whole response body for `49` — the result of `{{7*7}}` had the template been evaluated. It passed alone and failed in the full run, because httpbin echoes numeric headers such as `Content-Length` and the substring appeared for entirely unrelated reasons. Asserting on the specific field instead is both stricter and stable.

> **RULE** A flaky test deserves a diagnosis, not a retry. Both of these looked like environment noise and were not.

Adapting this to your own API

1. Add your environment to `src/config/environments.ts` — URLs, prefix, latency budget, read-only flag. No secrets.

2. Copy `src/services/object.service.ts`. Declare the schema first and infer the type from it, so the runtime check and the compile-time type cannot disagree.

```
export const OrderSchema = z.object({ id: identifier, status: z.enum(['draft', 'pai
export type Order = z.infer<typeof OrderSchema>;
```

3. Register the schemas so the contract guard can find them from a real request.

```
registerSchemas([
  { name: 'order', method: 'GET', pathPattern: '/orders/{id}', status: '2xx', schem
]);
```

4. Add the service to the `api` fixture in `src/fixtures/api.fixture.ts` — one line, and every test can reach it.

5. Copy the shape of `objects.crud.spec.ts`: one composing lifecycle test, several focused ones. The lifecycle tells you *that* something broke; the focused tests tell you *what*.

6. If your API has a quota or is otherwise unreliable, model it in `src/mocks/` and run the same service against both.

Project structure

Every folder and file, with its purpose

Every file in the repository, grouped by role. Each entry links to its full reference — purpose, when and how to change it, and why the change belongs there.

```
playwright-api-framework/
├── playwright.config.ts        Projects, reporters, timeouts, artefacts
├── tsconfig.json               Strict TypeScript + path aliases
├── eslint.config.mjs           Type-aware linting
├── package.json                Dependencies and every command
├── .env.example                Documented environment template
├── Dockerfile / docker-compose.yml   CI-identical local runs
├── Makefile                    Discoverable task shortcuts
├── .github/workflows/          Sharded CI with merged reports
├── .vscode/                    Shared editor behaviour
├── docs/                       This documentation (site + PDF)
├── scripts/docs/               The documentation generator
├── src/
│   ├── config/                 Validated environment resolution
│   ├── core/                   HTTP client, request builder, response, errors
│   ├── auth/                   Basic, bearer, API key, OAuth2, HMAC, session, JWT
│   ├── contracts/              Zod schemas, JSON Schema, OpenAPI, the registry
│   ├── protocols/              GraphQL, SSE, NDJSON, WebSocket
│   ├── services/               Service objects — the API's page objects
│   ├── fixtures/               The test and expect that specs import
│   ├── hooks/                  Global setup/teardown, session capture
│   ├── mocks/                  Stub server, exchange recorder, executable contract m
│   ├── utils/                  Logging, polling, paging, diffing, security, data
│   ├── reporters/              Custom run summary
│   ├── data/                   Static test data
│   └── types/                  Shared type vocabulary
└── tests/                      api/ · contract/ · performance/ · security/
```

Root configuration

FILE	PURPOSE
`.editorconfig`	Editor settings honoured by editors that never load the project's tooling — indentation, line endings, trailing whitespace, final newline.
`.env.example`	The committed template for `.env`.
`.gitignore`	Keeps generated output and — more importantly — live credentials out of the repository.
`.nvmrc`	Pins the Node version for `nvm` and for CI's `node-version-file`.

FILE	PURPOSE
`.prettierignore`	Files Prettier must not touch: generated output, and anything whose exact bytes matter.
`.prettierrc.json`	Formatting settings.
`eslint.config.mjs`	Type-aware linting.
`package.json`	Declares every dependency and every command.
`playwright.config.ts`	Decides how the suite runs: which groups of tests exist, what they share, how long they get, and what evidence a failure leaves.
`tsconfig.json`	Compiler settings and path aliases.

CI, containers and editor tooling

FILE	PURPOSE
`.github/actions/setup/action.yml`	Node, dependencies and browsers, in the one place every job shares.
`.github/workflows/api-tests.yml`	The pipeline: cheap checks first, then the suite split across four shards, then a job that merges the shard reports into one readable result — plus an offline contract job and a documentation build.
`.vscode/extensions.json`	Recommended extensions, offered to anybody who opens the repository.
`.vscode/launch.json`	Two debug configurations: the current spec with breakpoints, and the whole suite serially.
`.vscode/settings.json`	Shared editor behaviour: format on save, ESLint autofix, the workspace TypeScript version, and hiding generated folders from search.
`docker-compose.yml`	Two ready-made containerised runs: the full suite against a real environment, and the contract suite against the built-in stub server with no network at all.
`Dockerfile`	Builds the image the suite runs in, so a local run and a CI run execute against the same runtime rather than two similar ones.
`Makefile`	Discoverable shortcuts.

Documentation tooling

FILE	PURPOSE
`scripts/docs/artifact.css`	The stylesheet for the hosted single-page manual.
`scripts/docs/build-docs.mjs`	Assembles the site.

FILE	PURPOSE
`scripts/docs/build-pdf.mjs`	Renders `print.html` to PDF using the Chromium that ships with Playwright, so the PDF and the HTML cannot diverge — they are the same document, printed.
`scripts/docs/content/`	The authored prose, split across one guides module and six file modules.
`scripts/docs/docs.css`	The stylesheet for the local site and the print sheet.
`scripts/docs/extract-api.mjs`	Reads the TypeScript source with the compiler API and writes `docs/generated/api.json`: every exported class, function, interface, type and constant, with its signature and its JSDoc summary.
`scripts/docs/highlight.mjs`	Build-time syntax highlighting for eight languages.
`scripts/docs/renderer.mjs`	Turns content blocks and extracted API entries into HTML.

Configuration (src/config)

FILE	PURPOSE
`src/config/env.config.ts`	The only file in the framework that reads `process.env`.
`src/config/environments.ts`	The table of named targets — `demo`, `local`, `mock`, `dev`, `staging`, `production` — holding URLs, path prefixes, TLS behaviour, latency budgets and the read-only flag, plus the public APIs the demonstration suite reaches alongside them.
`src/config/timeouts.ts`	Named time budgets.

Core (src/core)

FILE	PURPOSE
`src/core/api.response.ts`	The response wrapper — the highest-leverage class in the framework, since every assertion a test makes goes through it.
`src/core/base.service.ts`	The base class for service objects: path joining, request helpers scoped to `basePath`, reporter steps, and access to the cleanup registry.
`src/core/errors.ts`	The error hierarchy.
`src/core/http.client.ts`	The HTTP engine.
`src/core/request.builder.ts`	The fluent builder.

Types (src/types)

FILE	PURPOSE
`src/types/index.ts`	The shared vocabulary.

Authentication (src/auth)

FILE	PURPOSE
`src/auth/jwt.ts`	JWT inspection — decoding only.
`src/auth/oauth2.auth.ts`	OAuth 2.0 token acquisition: client credentials, resource-owner password, and refresh.
`src/auth/static.auth.ts`	The three credential schemes that need no network call: `NoAuth`, `BasicAuth` and `ApiKeyAuth`.
`src/auth/token.store.ts`	Caches access tokens, with an expiry skew, and optionally persists them to a file so separate worker processes can share one.

Contracts (src/contracts)

FILE	PURPOSE
`src/contracts/json-schema.ts`	JSON Schema validation via Ajv, for schemas the team was *given* rather than wrote — an OpenAPI document, a partner's published contract, a schema from another repository.
`src/contracts/openapi.ts`	Validates responses against an OpenAPI document: finds the operation for a method and path, resolves local `$refs`, picks the schema for the status, and hands it to Ajv.
`src/contracts/schema.registry.ts`	Maps method + path pattern + status to a schema.
`src/contracts/schemas.ts`	Reusable schema building blocks: identifiers, timestamps, money, pagination envelopes, and the standard error shapes.

Protocol clients (src/protocols)

FILE	PURPOSE
`src/protocols/graphql.client.ts`	GraphQL client: queries, mutations, batching, introspection, and a response wrapper whose assertions understand GraphQL's failure model.
`src/protocols/sse.client.ts`	Server-Sent Events and NDJSON streaming.
`src/protocols/websocket.client.ts`	WebSocket client built on Node's global `WebSocket`, with every received frame buffered from the moment it connects.

Service objects (src/services)

FILE	PURPOSE
`src/services/object.service.ts`	The service object for https://api.restful-api.dev/objects — the live CRUD resource the lifecycle suite runs against.
`src/services/post.service.ts`	The service object for https://jsonplaceholder.typicode.com/posts — reads and real `Link` header pagination over a stable 100-record dataset.
`src/services/template.service.ts`	A worked example of a service object — copy it to start a new one.

Fixtures (src/fixtures)

FILE	PURPOSE
`src/fixtures/api.fixture.ts`	The fixtures every test imports.
`src/fixtures/custom-matchers.ts`	Nine custom `expect` matchers for API responses, registered through `expect.extend` — which returns a *typed* `expect`, so no separate type declaration is needed.
`src/fixtures/index.ts`	The single import for every test.

Hooks (src/hooks)

FILE	PURPOSE
`src/hooks/auth.setup.ts`	Captures an authenticated session before the suite runs.
`src/hooks/global.setup.ts`	Runs once before any test.
`src/hooks/global.teardown.ts`	Runs once after every test.

Utilities (src/utils)

FILE	PURPOSE
`src/utils/cleanup.registry.ts`	Tracks resources a test created so they can be removed afterwards.
`src/utils/data.utils.ts`	Test data: seeded Faker, factories, unique identifiers, and curated collections of adversarial strings, numbers and dates.
`src/utils/diff.utils.ts`	Structural comparison: a value diff with ignorable fields, and a *shape* comparison for detecting breaking changes.
`src/utils/file.utils.ts`	Resolving a path against the repository root, so the suite behaves the same regardless of the working directory it was launched from.

FILE	PURPOSE
src/utils/header.utils.ts	Header parsing: case-insensitive lookup, Set-Cookie, Content-Type, Link, Retry-After, rate-limit counters, and redaction.
src/utils/jsonpath.utils.ts	A small, dependency-free JSON path reader supporting property access, array indices (including negative), wildcards and recursive descent.
src/utils/logger.ts	A dependency-free structured logger.
src/utils/pagination.utils.ts	Walkers for the four pagination conventions — Link header, cursor, offset — plus a normaliser and a defect detector.
src/utils/performance.utils.ts	Latency measurement: percentiles, a collector that samples every request, and a sampler for repeated calls.
src/utils/security.utils.ts	Security checks that belong in a functional API suite: information disclosure, security headers, CORS, and authorisation matrices.
src/utils/xml.utils.ts	XML and SOAP support: parsing, hand-rolled serialisation, path reading, envelope construction and fault extraction.

Stubbing and recording (src/mocks)

FILE	PURPOSE
src/mocks/mock.server.ts	A programmable stub server: match a request, return a response — with delays, failures and controlled flakiness on demand.
src/mocks/objects.stub.ts	An executable model of the api.restful-api.dev/objects contract: the whole CRUD resource, in memory, reproducing every observed quirk.
src/mocks/recorder.ts	Records exchanges and replays them as stubs.

Reporters (src/reporters)

FILE	PURPOSE
src/reporters/summary.reporter.ts	Writes reports/summary.json: one machine-readable file saying whether the run passed, which tests failed and why, which were slowest, and how long it all took.

Test data (src/data)

FILE	PURPOSE
src/data/openapi.json	An OpenAPI 3.0.3 document describing the /objects resource.
src/data/README.md	Documents what lives in src/data/, what belongs there and what does not.

Tests folder

FILE	PURPOSE
`tests/api/client.behaviour.spec.ts`	Tests the framework itself against https://httpbin.org, a request-inspection service that echoes back exactly what it received.
`tests/api/objects.crud.spec.ts`	The headline suite: a full create-read-update-delete lifecycle against https://api.restful-api.dev, a live public API that genuinely persists writes.
`tests/api/posts.pagination.spec.ts`	Reads and pagination against https://jsonplaceholder.typicode.com — the API chosen because it has genuine RFC 8288 `Link` header pagination, a completely stable 100-record dataset, and no quota.
`tests/contract/objects.openapi.spec.ts`	Validates every response in a lifecycle against the OpenAPI document in `src/data/openapi.json`, proves the check has teeth, and reports the contract-coverage gap.
`tests/contract/objects.stubbed.spec.ts`	The same `ObjectService`, the same lifecycle, run against an in-memory model of the same contract.
`tests/performance/posts.latency.spec.ts`	Latency budgets and percentiles, measured against a real endpoint under the `performance` project's single worker.
`tests/README.md`	Where each kind of test belongs, and the conventions the tooling already enforces.
`tests/security/response.hygiene.spec.ts`	Response hygiene, injection transport, CORS and authorisation, against real services.

Repository documentation

FILE	PURPOSE
`docs/README.md`	Explains what is in `docs/`: which files are generated, how to read them, how to rebuild them, and where to edit each part.
`README.md`	The repository's front door: what the framework is, how to start it, what is in it, and where the full documentation lives.

Reference: configuration & tooling

Root config, CI, containers, editor and docs tooling

Root configuration

.editorconfig

Editor settings honoured by editors that never load the project's tooling — indentation, line endings, trailing whitespace, final newline.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A file type needs different handling. Markdown already does: trailing whitespace is significant there.

> **HOW TO CHANGE IT**
>
> 1. Add a section for the glob.
>
> ```
> [*.md]
> trim_trailing_whitespace = false
> ```

> **WHY THE CHANGE BELONGS HERE**
>
> It is the lowest common denominator: it works in an editor with no plugins, which is the situation somebody making a one-line fix on a strange machine is in.

Related: `.vscode/settings.json` · `.prettierrc.json`

.env.example

The committed template for `.env`. In practice it is the primary documentation for configuring the suite, which is why every variable carries a comment rather than only a name.

GROUP	VARIABLES	NOTES
Target	`TEST_ENV`, `API_BASE_URL`, `GRAPHQL_URL`, `WS_URL`	`TEST_ENV` selects an entry from `environments.ts`; the URLs override it
Credentials	`API_KEY`, `*_USERNAME`, `*_PASSWORD`	Reached only through `getUser(role)`
OAuth2	`OAUTH_TOKEN_URL`, `OAUTH_CLIENT_ID`, `OAUTH_CLIENT_SECRET`, `OAUTH_SCOPE`	Presence of a token URL is what selects OAuth as the default scheme
Signing	`HMAC_KEY_ID`, `HMAC_SECRET`	For APIs that require a per-request signature
Execution	`API_TIMEOUT`, `RETRY_COUNT`, `WORKERS`, `TRACE`	Tuning without editing code

GROUP	VARIABLES	NOTES
Diagnostics	`LOG_LEVEL`, `LOG_BODIES`	`LOG_BODIES` is verbose and off by default
Policy	`STRICT_CONTRACTS`, `STRICT_CONTENT_TYPE`, `LATENCY_BUDGET_MS`	`STRICT_CONTRACTS` is on in CI, off locally
Tooling	`MOCK_SERVER_PORT`, `ALLURE`	

Layering

`.env` is read first, then `.env.<TEST_ENV>` on top of it. Neither overrides a variable already exported in the shell, which is how CI injects secrets without touching a file.

Precedence, weakest to strongest — TYPESCRIPT

```
.env  →  .env.staging  →  shell environment  →  inline on the command line
```

WHEN YOU NEED TO CHANGE IT

- You add a variable to the schema in `env.config.ts` — always add it here too.
- A default proves wrong for most people.
- A variable's purpose is not obvious from its name.

HOW TO CHANGE IT

1. Add the variable with a comment saying what it does and what values it accepts.

   ```
   # Reject responses whose Content-Type does not match what was requested.
   STRICT_CONTENT_TYPE=true
   ```

2. Add it to the Zod schema in `src/config/env.config.ts`, or it will be ignored.
3. Add it to the CI `env:` block if the pipeline needs it.

WHY THE CHANGE BELONGS HERE

This file is committed and `.env` is not, so it is the only place a new joiner can discover what is configurable. A variable that exists in the schema but not here is a variable nobody knows about.

WATCH OUT FOR

- Never put a real value here. Placeholders and empty strings only — this file is in the repository.
- A variable added here but not to the schema is silently ignored, which looks exactly like a bug in the framework.

Related: `src/config/env.config.ts` · `.gitignore` · `.github/workflows/api-tests.yml`

.gitignore

Keeps generated output and — more importantly — live credentials out of the repository. Two entries here are security controls rather than housekeeping.

PATTERN	WHY
`.env*` with `!.env.example`	**Security.** Real configuration never enters the repository; the template does.
`storage/*.json`	**Security.** Captured sessions and access tokens. Committing one is equivalent to committing a password.
`reports/`, `test-results/`, `blob-report/`	Regenerated by every run; committing them creates conflicts on every branch.
`docs/generated/`, `docs/site/`	Rebuilt by `npm run docs`. The PDF is committed; the site is not.
`node_modules/`	Reproduced from `package-lock.json`.

WHEN YOU NEED TO CHANGE IT

- A tool starts writing output into the working tree.
- A new directory will hold credentials.

HOW TO CHANGE IT

1. Add the pattern with a comment explaining the category.
2. If the file was already tracked, ignoring it is not enough — untrack it, then rotate anything secret it contained.

```
git rm --cached storage/session-staging.json
```

WHY THE CHANGE BELONGS HERE

A `.gitignore` is the last line of defence against a credential leak, and it only works before the first commit of the file. Adding a directory here at the same time as the code that writes into it is the habit that keeps it working.

WATCH OUT FOR

- Ignoring a file does not remove it from history. If a secret was committed, rotate it — deletion does not help.
- The `!.env.example` negation must come after the `.env*` pattern, or the template is ignored too.

Related: `.env.example` · `src/auth/token.store.ts`

`.nvmrc`

Pins the Node version for `nvm` and for CI's `node-version-file`. Node 22 LTS — chosen because it is the first release with a stable global `WebSocket`, which is what lets the socket client have no dependency.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The team moves to a new LTS.

> **HOW TO CHANGE IT**
>
> 1. Update this file, the `engines` floor in `package.json`, and the `Dockerfile` base image together.
> 2. Verify on the new version before merging.
>
>    ```
>    nvm use && npm ci && npm run validate
>    ```

> **WHY THE CHANGE BELONGS HERE**
>
> One file that CI and every developer both read is what stops "works on my machine" from being about the runtime.

> **WATCH OUT FOR**
>
> - Newer Node is usually fine, but two things here depend on version: global `WebSocket` (22+) and streaming `fetch`. Older Node will fail at run time, not at install.

Related: `package.json` · `Dockerfile` · `.github/workflows/api-tests.yml`

`.prettierignore`

Files Prettier must not touch: generated output, and anything whose exact bytes matter.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A tool starts generating files that the formatter would rewrite.

> **HOW TO CHANGE IT**
>
> 1. Add the path. Keep it aligned with the ESLint ignore list, so the two tools agree on what is generated.

> **WHY THE CHANGE BELONGS HERE**
>
> Formatting generated output produces a diff on every build and tells you nothing.

> **WATCH OUT FOR**
>
> - `docs/site/` and `docs/generated/` must be here as well as in ESLint's ignores; the two lists are maintained separately.

Related: `.prettierrc.json` · `eslint.config.mjs`

.prettierrc.json

10 lines

Formatting settings. Their value is not that they are the right settings — it is that nobody has to have an opinion about them again.

SETTING	VALUE	NOTE
printWidth	100	Wide enough for a fluent builder chain, narrow enough for a side-by-side diff
singleQuote	true	Matches the TypeScript ecosystem
trailingComma	all	A one-line diff when adding an argument, rather than two
endOfLine	lf	Stops Windows checkouts from reformatting every file

> **WHEN YOU NEED TO CHANGE IT**
>
> - Essentially never, unless the team agrees to a different house style.

> **HOW TO CHANGE IT**
>
> 1. Change the setting, reformat everything in one commit, and keep that commit free of anything else.
>
>    ```
>    npm run format
>    ```

> **WHY THE CHANGE BELONGS HERE**
>
> A formatting change touches every file. Isolating it in its own commit is what keeps `git blame` useful afterwards.

> **WATCH OUT FOR**
>
> - Reformatting mixed into a functional change makes the review impossible and the history worse.

Related: `.prettierignore` · `eslint.config.mjs`

eslint.config.mjs

118 lines

Type-aware linting. It uses the type checker rather than only the syntax tree, which is what allows rules like "this promise is never awaited" and "this template literal stringifies an object" to exist at all.

Layers, in order

1. Ignores — build output, reports, generated documentation.
2. `js.configs.recommended`, then `strictTypeChecked` and `stylisticTypeChecked` from typescript-eslint.
3. Project rules — explicit return types, `import type`, no `console` except `warn`/`error`, `eqeqeq`.
4. A fixtures block that turns off `no-empty-pattern`.

5. A tests block adding the Playwright plugin.
6. A block disabling type-aware rules for `.mjs`, which has no `tsconfig` to check against.
7. `eslint-config-prettier` **last**, so formatting rules never fight the formatter.

Two documented exceptions

Fixtures need an empty destructuring pattern — TYPESCRIPT

```typescript
// Playwright resolves a fixture's dependencies by reading its parameter
// destructuring, so a fixture with no dependencies must be `async ({}, use)`.
files: ['src/fixtures/**/*.ts'],
rules: { 'no-empty-pattern': 'off' },
```

Payload readers exist to be parameterised by the caller — TYPESCRIPT

```typescript
// The rule flags a type parameter used only in the return position — which
// is exactly the shape of `response.json<User>()`. Following it would replace
// the pattern with a cast at every call site.
'@typescript-eslint/no-unnecessary-type-parameters': 'off',
```

Both are written this way on purpose: an exception with a reason beside it is an exception the next person can evaluate. An exception without one just gets copied.

WHEN YOU NEED TO CHANGE IT

- A rule is producing more noise than signal across the codebase.
- You add a folder that needs different rules — scripts, generated code.
- A new class of mistake is worth catching automatically.

HOW TO CHANGE IT

1. Add a scoped block rather than weakening the global config, so the exception stays visible.

```
{
  files: ['src/experimental/**/*.ts'],
  rules: { '@typescript-eslint/no-unsafe-assignment': 'off' },
},
```

2. Always leave a comment saying why. Every existing exception has one.
3. Check the whole repository, not just the file you were looking at.

```
npm run lint
```

WHY THE CHANGE BELONGS HERE

Lint rules are the cheapest possible code review: they run before a human looks, they never get tired, and they are consistent. What they cost is judgement, which is why every exception carries its reasoning.

> **WATCH OUT FOR**
>
> - `eslint-config-prettier` must stay last, or formatting rules will conflict with the formatter and produce an unfixable loop.
> - Type-aware rules need `projectService: true`; without it they silently do nothing rather than erroring.
> - `--max-warnings=0` means a warning fails CI. That is deliberate — warnings nobody must fix are warnings nobody reads.

Related: `tsconfig.json` · `.prettierrc.json` · `package.json`

`package.json` 72 lines

Declares every dependency and every command. In practice this is the framework's table of contents: if a task is worth doing twice, it has a script here.

Script groups

GROUP	SCRIPTS	FOR
Run	`test`, `test:api`, `test:contract`, `test:performance`, `test:security`	One project at a time, matching the projects in `playwright.config.ts`
Select	`test:smoke`, `test:regression`, `test:failed`, `test:serial`	A subset by tag, by last result, or without parallelism
Debug	`test:debug`, `test:ui`	The inspector and the interactive UI mode
Report	`report`, `report:allure`	Open what a run produced
Quality	`lint`, `lint:fix`, `format`, `format:check`, `typecheck`, `validate`	`validate` is what CI runs first — the three checks in one
Docs	`docs`, `docs:api`, `docs:html`, `docs:pdf`, `docs:open`	Regenerate this documentation

Notable dependency choices

PACKAGE	WHY THIS ONE
`zod@^3`	**Pinned to v3 deliberately.** The v4 CommonJS build fails to initialise under Playwright's transform, and Playwright loads config through that transform. Upgrading requires verifying that first.
`ajv` + `ajv-formats`	JSON Schema validation for specifications the team was *given* rather than wrote. `ajv-formats` supplies `date-time`, `email`, `uri` — without it those keywords are silently ignored.
`fast-xml-parser`	XML and SOAP parsing with no native bindings. Only the parser is used; its builder is deprecated, so `xml.utils.ts` serialises by hand.
`@faker-js/faker`	Data generation, seeded per test so runs are reproducible.

PACKAGE	WHY THIS ONE
`csv-parse`	Reads the data-driven case tables in `src/data/`.
`playwright-ctrf-json-reporter`	A common test-result format, for aggregating across tools.
`allure-playwright`	Optional and off by default — it is heavy, and `ALLURE=true` turns it on.

Two things that are absent on purpose

- **No `"type": "module"`.** Playwright transforms TypeScript itself, and declaring the package ESM breaks interoperability with CommonJS dependencies loaded through that transform.
- **No WebSocket or HTTP client library.** Node has both built in — a global `WebSocket` since 22, and streaming `fetch`. A dependency here would be a second implementation to keep current.

Git hooks

The `prepare` script installs Husky, and `lint-staged` (configured in this file) runs ESLint and Prettier on staged files. `.husky/pre-commit` runs `lint-staged`; `.husky/pre-push` runs `typecheck`, because a type error is cheap to catch locally and expensive to catch in CI.

> **WHEN YOU NEED TO CHANGE IT**
>
> - You add a dependency.
> - A task is worth running twice — give it a script rather than a wiki page.
> - You add a project to `playwright.config.ts` and want a shortcut for it.
> - The Node floor moves (also update `.nvmrc` and the `Dockerfile` tag).

> **HOW TO CHANGE IT**
>
> 1. Add a dependency. Everything here is a devDependency: nothing ships to production.
>
>    ```
>    npm install --save-dev <package>
>    ```
>
> 2. Add a script named for the outcome, not the mechanics.
>
>    ```
>    "test:orders": "playwright test --grep @orders"
>    ```
>
> 3. Mirror it in the `Makefile` if it is a task somebody would look for.
> 4. Re-run validation, then regenerate the docs so this page reflects it.
>
>    ```
>    npm run validate && npm run docs
>    ```

> **WHY THE CHANGE BELONGS HERE**
>
> Scripts are the discoverable interface to the repository. A command that lives only in somebody's shell history is a command the next person re-invents, slightly differently.

> **WATCH OUT FOR**
>
> - Do not upgrade zod to v4 without checking it loads under Playwright's transform; the failure is an obscure `core._string is not a function` at start-up.

> - The `Dockerfile` pins the Playwright image to an exact version. Bumping `@playwright/test` without bumping that tag gives CI a different version from local.

Related: `playwright.config.ts` · `Makefile` · `Dockerfile` · `.nvmrc`

`playwright.config.ts` 152 lines

Decides how the suite runs: which groups of tests exist, what they share, how long they get, and what evidence a failure leaves. It does not decide what the suite talks to — that comes from `src/config`, which validates the environment before this file reads a value.

Projects

A project is the unit of organisation: a group of tests with its own settings and its own place in the dependency order. That is what lets "log in once, then run everything else" be declarative rather than a global variable.

PROJECT	TEST DIRECTORY	SETTINGS THAT DIFFER	WHY
`setup`	`src/hooks/`	Longer timeout	Captures a session and writes it to `storage/`. Everything else depends on it.
`api`	`tests/api/`	Defaults	The functional suite: REST, GraphQL, streaming.
`contract`	`tests/contract/`	Shorter timeout	A different question — conformance, not behaviour — usually on a different trigger. Runs offline against the stub server.
`performance`	`tests/performance/`	`workers: 1`, `retries: 0`, long timeout	A percentile measured under eight parallel workers measures the load, not the endpoint.
`security`	`tests/security/`	Defaults	Hostile payloads, kept separable so a pipeline can point them at a dedicated environment.

Reporters

Six always, three conditionally. Each answers a different audience: `list` for the person watching, `html` for the person investigating, `junit` / `json` / `ctrf` for machines, and the custom summary for a pipeline gate.

The conditional tail — TYPESCRIPT

```
...(config.allure ? [['allure-playwright', { resultsDir: 'reports/allure-results'
...(process.env.GITHUB_ACTIONS ? [['github']] : []),
...(process.env.PW_BLOB_REPORT ? [['blob', { outputDir: 'blob-report' }]] : []),
```

Allure is heavy, so it is opt-in. The GitHub reporter annotates a pull request and only makes sense inside Actions. Blob reports exist to be merged across shards and are noise otherwise.

Execution settings worth understanding

SETTING	VALUE	REASONING
`workers`	8 in CI, 4 locally	API tests are IO-bound, so more workers than cores is right. The ceiling exists because the target's rate limit — not this machine — is what a large suite saturates.
`retries`	2 in CI, 0 locally	Locally a flaky test should be visible immediately. In CI a retry separates a real failure from a blip — and the summary reporter counts anything that needed one as *flaky*, so retries cannot hide rot.
`forbidOnly`	on in CI	A `test.only` left in a branch would otherwise reduce CI to one test and still go green.
`fullyParallel`	true	Tests are independent by construction. If yours are not, fix the coupling rather than turning this off.
`trace`	from `TRACE`, default `retain-on-failure`	A trace records every request and response — usually enough to diagnose without re-running.
`maxFailures`	50 in CI	When fifty tests have failed, the environment is broken; the remaining thousand add nothing.

WHEN YOU NEED TO CHANGE IT

- You add a category of test that needs its own settings.
- You need a different reporter, or a different reporter configuration.
- The suite has grown enough that shard or worker counts need retuning.
- A whole-test or hook timeout is systematically wrong.

HOW TO CHANGE IT

1. Add a project, its test folder, and a script in `package.json`.

```
{
  name: 'smoke',
  testDir: './tests/api',
  grep: /@smoke/,
  dependencies: ['setup'],
  retries: 0,
},
```

2. Add a reporter to the array. Make anything heavy conditional.
3. Prefer changing a named budget in `src/config/timeouts.ts` over a literal here, so the reasoning stays in one place.

WHY THE CHANGE BELONGS HERE

This file is the run's contract with CI. Anything that changes how the suite executes — rather than what it talks to — belongs here, where a reviewer looking at "why did CI behave differently" will find

> it.

> **WATCH OUT FOR**
>
> - `--reporter=` on the command line **replaces** this list. A run with `--reporter=list` produces no `summary.json`, no JUnit and no HTML report.
> - `dependencies` makes a project wait for another. A cycle is a start-up error, not a hang.
> - Per-project `workers` overrides the top-level value; that is how `performance` stays serial while everything else parallelises.

Related: `src/config/env.config.ts` · `src/config/timeouts.ts` · `src/reporters/summary.reporter.ts` · `src/hooks/global.setup.ts`

`tsconfig.json` 46 lines

Compiler settings and path aliases. The strictness here is not decoration: several settings turn a class of run-time failure into a compile error, which in a test framework means a bug in the framework rather than a confusing failure in somebody's test.

SETTING	EFFECT
`strict`	The whole family — `strictNullChecks`, `noImplicitAny` and the rest.
`noUncheckedIndexedAccess`	`array[0]` is `T | undefined`. Verbose, and it prevents an entire class of "cannot read property of undefined" at run time.
`noImplicitOverride`	`override` must be explicit, so a renamed base method cannot silently orphan a subclass.
`noUnusedLocals` / `noUnusedParameters`	Dead code fails the build rather than accumulating.
`noFallthroughCasesInSwitch`	The exhaustive switches over `BodyKind` stay exhaustive.
`isolatedModules`	Every file must be transpilable alone — which is exactly how Playwright transpiles them.
`module: ESNext`, `moduleResolution: Bundler`	Matches how Playwright resolves modules. Changing these breaks imports in ways the compiler will not warn about.

Path aliases

Available prefixes — TYPESCRIPT

```
@config/*     @core/*      @auth/*
@contracts/*  @protocols/* @services/*
@fixtures/*   @utils/*     @mocks/*
@types/*      @data/*
```

> **NOTE** There is no `baseUrl`: TypeScript 6 deprecates it, and `paths` entries resolve relative to the config file without it.

> **WHEN YOU NEED TO CHANGE IT**
>
> - You add a top-level folder under `src/` and want an alias for it.
> - The Node or TypeScript floor moves.

> **HOW TO CHANGE IT**
>
> 1 Add the alias to `paths`, relative to this file.
>
> ```
> "@events/*": ["./src/events/*"]
> ```
>
> 2 Confirm nothing regressed.
>
> ```
> npm run typecheck
> ```

> **WHY THE CHANGE BELONGS HERE**
>
> Aliases keep imports readable as the tree deepens; a file four levels down importing `../../../../core/http.client` is a file nobody wants to move.

> **WATCH OUT FOR**
>
> - Loosening `strict` or `noUncheckedIndexedAccess` will make hundreds of latent problems compile. That is not the same as fixing them.
> - Playwright does not read `paths` at run time from every context — the aliases work because Playwright's transform honours the config. Keep `module` and `moduleResolution` as they are.

Related: `eslint.config.mjs` · `package.json`

CI, containers and editor tooling

`.github/actions/setup/action.yml` 88 lines

Node, dependencies and browsers, in the one place every job shares. A composite action so the setup block is written once rather than copied into all five jobs.

This exists because installing dominated the pipeline. On one measured run `npm ci` took 421 seconds per job while the tests took 8 — installation was 97% of the wall time, and the suite it was there to run was a rounding error.

The npm cache was already hitting, so the cost was not fetching tarballs. It was npm rebuilding `node_modules` from them, plus an audit round-trip that every parallel job made at once. The contention shows in the spread: one shard installed in 23 seconds and the others took seven minutes for identical work.

What it does

STEP	WHY
Restore `node_modules`	Keyed on `.nvmrc` + `package-lock.json`. A hit skips installation entirely and costs about as long as untarring the directory.
Install on a miss only	`--prefer-offline --no-audit --fund=false` removes two registry round-trips; `timeout 300` bounds it so a stall fails loudly rather than silently.
Restore browsers	Only when the `browsers` input is set. Keyed on the lockfile, because that is what pins the Playwright version the binaries belong to.
`install-deps` always	The apt packages are not cached — every job starts on a clean runner — so the system libraries are installed even on a cache hit.

NOTE The job graph is what makes the cache pay: the first job has no dependencies and every other job declares `needs` on it, so the cache is populated before anything else starts. Removing that ordering would put every job back to a cold install.

WHEN YOU NEED TO CHANGE IT

- A job needs a browser it currently does not.
- Installation gets slow again.

HOW TO CHANGE IT

1. Pass the browser through the `browsers` input rather than adding a `playwright install` step to the job. A step outside this action gets no caching.

```yaml
- uses: ./.github/actions/setup
  with:
    browsers: chromium
```

2. If a cache key ever needs to change, change it here once. The key deliberately hashes the lockfile and nothing else about the source, so editing a test never invalidates the dependency cache.

WHY THE CHANGE BELONGS HERE

Five copies of a setup block drift apart. One that every job calls means a caching change is made once and is either right everywhere or wrong everywhere — which is the easier of the two to notice.

Related: `.github/workflows/api-tests.yml` · `package.json`

`.github/workflows/api-tests.yml` 162 lines

The pipeline: cheap checks first, then the suite split across four shards, then a job that merges the shard reports into one readable result — plus an offline contract job and a documentation build.

Jobs

JOB	DEPENDS ON	PURPOSE
`quality`	—	`npm run validate`. Fails in under a minute, before anything touches a network.
`test`	`quality`	The suite in a 4-way shard matrix, each shard emitting a blob report.
`merge-reports`	`test` (`if: always()`)	One HTML report from four shards. Runs even when a shard failed — especially then.
`contract`	`quality`	Contract tests against the stub server. No secrets, so it runs on forks too.
`docs`	`quality`	Rebuilds this documentation. Fails when a source file has no entry.

Triggers

- Push to `main` and every pull request.
- Nightly at 03:43 — catches drift in the target that no branch touched. Deliberately not on the hour: GitHub delays scheduled runs under load, and the top of the hour is the most contended slot.
- Manual, with an environment picker.

Secrets

Injected per step, never written to a file — YAML

```yaml
env:
  API_BASE_URL: ${{ secrets.API_BASE_URL }}
  OAUTH_CLIENT_SECRET: ${{ secrets.OAUTH_CLIENT_SECRET }}
  STRICT_CONTRACTS: 'true'
```

`STRICT_CONTRACTS` is forced on here and left off locally: a work-in-progress schema should not block a developer, but contract drift must not reach `main`.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The suite outgrows four shards.
> - A new project should run on a different trigger.
> - A new secret is needed.

> **HOW TO CHANGE IT**
>
> 1. Change the shard count in **both** places — the matrix and the `--shard` argument. They are separate values and a mismatch silently skips tests.
>
>    ```yaml
>    matrix:
>      shard: [1, 2, 3, 4, 5, 6]
>    # …
>    run: npx playwright test --shard=${{ matrix.shard }}/6
>    ```

2. Add the secret in the repository settings, then reference it in the `env:` block.
3. Keep `if: always()` on artefact uploads. The report of a failed run is the one worth having.

> **WHY THE CHANGE BELONGS HERE**
>
> Sharding is what keeps a large suite inside a reviewer's attention span; merging is what keeps the result readable. Doing one without the other gives you either a slow pipeline or four partial reports.

> **WATCH OUT FOR**
>
> - A shard-count mismatch between the matrix and the `--shard` flag means some tests never run, and nothing reports it.
> - `concurrency` cancels superseded runs on the same ref — intended, but it means a re-push loses the in-flight run's report.
> - Secrets are unavailable to workflows triggered from a fork. That is why the `contract` job needs none.

Related: `package.json` · `.nvmrc` · `src/reporters/summary.reporter.ts`

`.vscode/extensions.json` 10 lines

Recommended extensions, offered to anybody who opens the repository. Playwright, ESLint, Prettier, EditorConfig and an HTTP client for exploring an endpoint by hand.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The team adopts a tool with useful editor integration.

> **HOW TO CHANGE IT**
>
> 1. Add its marketplace identifier. Keep the list short — a long list of recommendations is ignored wholesale.

> **WHY THE CHANGE BELONGS HERE**
>
> The Playwright extension runs a single test from the gutter and opens traces inline, which is the fastest debugging loop available. It is worth recommending explicitly.

Related: `.vscode/settings.json`

`.vscode/launch.json` 26 lines

Two debug configurations: the current spec with breakpoints, and the whole suite serially. Both set `LOG_LEVEL=debug`, and the single-spec one also sets `LOG_BODIES=true` and disables the timeout.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A new debugging scenario comes up repeatedly.

HOW TO CHANGE IT

1 Add a configuration. Point `program` at Playwright's CLI rather than at `npx`, so breakpoints bind.

```
"program": "${workspaceFolder}/node_modules/@playwright/test/cli.js",
"args": ["test", "${relativeFile}", "--workers=1", "--timeout=0"]
```

WHY THE CHANGE BELONGS HERE

`--timeout=0` is the important part: without it, a test times out while you are sitting on a breakpoint, and the debugging session ends underneath you.

WATCH OUT FOR

- `--workers=1` is required. Multiple workers means multiple processes, and the debugger attaches to one of them.

Related: `.vscode/settings.json` · `src/utils/logger.ts`

.vscode/settings.json

23 lines

Shared editor behaviour: format on save, ESLint autofix, the workspace TypeScript version, and hiding generated folders from search.

WHEN YOU NEED TO CHANGE IT

- A generated folder starts polluting search results.
- A new file type needs a formatter mapping.

HOW TO CHANGE IT

1 Add the path to `files.exclude` and `search.exclude`, and keep it aligned with `.gitignore`.

WHY THE CHANGE BELONGS HERE

`typescript.tsdk` matters more than it looks: without it, the editor uses its bundled TypeScript, which can be a different version from the one the build uses — so the editor and CI disagree about whether the code compiles.

WATCH OUT FOR

- These settings only apply to people who accept the workspace TypeScript version; the editor prompts once and remembers.

Related: `.vscode/extensions.json` · `.editorconfig` · `tsconfig.json`

docker-compose.yml

39 lines

Two ready-made containerised runs: the full suite against a real environment, and the contract suite against the built-in stub server with no network at all.

SERVICE	RUNS	NEEDS
`api-tests`	Everything	Credentials passed through from the host or the runner
`contract-tests`	`--project=contract` with `TEST_ENV=mock`	Nothing — no environment, no secrets

Secrets are passed through with `${VAR:-}` rather than written into the image, so nothing credential-shaped is ever baked into a layer. Reports are mounted back to the host, so a failed containerised run leaves the same evidence a local run would.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The suite needs a companion service — a database, a WireMock instance.
> - A new secret must reach the container.

> **HOW TO CHANGE IT**
>
> 1 Add the variable to the `environment` block using the pass-through form, so an unset value is empty rather than a startup error.
>
> ```
> TENANT_ID: ${TENANT_ID:-}
> ```
>
> 2 Add a companion service and let the tests reach it by service name.
>
> ```
> services:
>   wiremock:
>     image: wiremock/wiremock:3.9.1
>     ports: ['8080:8080']
> ```

> **WHY THE CHANGE BELONGS HERE**
>
> The `contract-tests` service is the one that matters most: it proves the suite can run somewhere with no access to any real environment, which is what makes it usable on a fork's pull request.

> **WATCH OUT FOR**
>
> - Never write a literal secret here. The file is committed.
> - `network_mode: host` is what lets the container reach a service on the developer's own machine; on macOS it behaves differently from Linux.

Related: `Dockerfile` · `.env.example` · `src/mocks/mock.server.ts`

`Dockerfile`

Builds the image the suite runs in, so a local run and a CI run execute against the same runtime rather than two similar ones.

The shape DOCKERFILE

```dockerfile
FROM mcr.microsoft.com/playwright:v1.62.1-jammy
WORKDIR /work
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund
COPY . .
```

Two details do the work. The tag is **pinned to an exact Playwright version** — an unpinned tag means the image changes silently, and the first symptom is a failure nobody can reproduce. And dependencies are installed before the source is copied, so editing a test does not invalidate the slow install layer.

NOTE API tests need no browser, but the image ships one and `build-pdf.mjs` uses it to render the documentation — so the docs build needs nothing extra installed.

WHEN YOU NEED TO CHANGE IT

- The Playwright version changes — the tag must move with it.
- The suite needs a system package.

HOW TO CHANGE IT

1. Bump the tag to match `@playwright/test` in `package.json`.

   ```dockerfile
   FROM mcr.microsoft.com/playwright:v1.63.0-jammy
   ```

2. Add system packages in one `RUN` layer, before the `COPY` of the source.
3. Verify.

   ```
   docker compose run --rm api-tests --project=contract
   ```

WHY THE CHANGE BELONGS HERE

The image is where "works on my machine" is settled. Pinning it is what makes a CI failure reproducible locally.

WATCH OUT FOR

- A version mismatch between the image tag and the npm package produces confusing errors — Playwright checks that the driver and the library agree.
- Copying source before installing dependencies rebuilds the install layer on every edit and makes local iteration painfully slow.

Related: `docker-compose.yml` · `package.json` · `.github/workflows/api-tests.yml`

`Makefile`

Discoverable shortcuts. `make` on its own prints every target with its description, which is the point: a task nobody can find is a task nobody runs.

The self-documenting trick — SHELL

```
help:
	@grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) | awk 'BEGIN {FS = ":
```

Every target carries a `## description` comment, and `help` reads them out of this file. Adding a target with a comment adds it to the help automatically.

WHEN YOU NEED TO CHANGE IT

- You add a script to `package.json` that somebody would look for by name.

HOW TO CHANGE IT

1. Add the target, its `.PHONY` entry, and a `##` description.

```
orders: ## Run the orders suite
	npm run test:orders
```

2. Check it appears.

```
make
```

WHY THE CHANGE BELONGS HERE

The Makefile and `package.json` scripts serve different habits — some people type `make`, some type `npm run`. Keeping them aligned costs one line and removes a whole category of "how do I run…" questions.

WATCH OUT FOR

- Recipes must be indented with a real tab. A space-indented recipe fails with `missing separator`, which is not a helpful message.

Related: `package.json`

Documentation tooling

`scripts/docs/artifact.css`

The stylesheet for the hosted single-page manual. Same identity as the offline site, executed for a screen: a permanent dark rail that reads as an instrument panel, a paper reading column beside it, and real typefaces instead of the system stack the offline copy has to settle for.

Type

ROLE	FACE	WHY
Display	Chivo	A sturdy grotesque that holds up at large sizes without the geometric sameness of the usual choices
Body	Source Sans 3	Humanist, highly legible at length, and not Inter
Mono	JetBrains Mono	Designed for code, with a large x-height that survives being set small in a table

Three theme states, not two

An explicit choice stamps `data-theme` on the root; the default "system" setting stamps nothing. So the palette is defined three times: on bare `:root`, inside the media query guarded as `:root:not([data-theme='light'])`, and again on `:root[data-theme='dark']` so the toggle wins in both directions.

The rail keeps its own dark palette in both themes. That is a deliberate commitment: chrome that flips with the content makes the page feel unanchored, and the rail is chrome.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The hosted manual gains a component.
> - The palette or typefaces should change.

> **HOW TO CHANGE IT**
>
> 1. Add the component's styles here and mirror the structure in `docs.css`.
> 2. When adding a colour, add it to all three theme blocks. Then check both themes before publishing.

> **WHY THE CHANGE BELONGS HERE**
>
> The hosted manual is read on a screen, often on a phone, often by somebody who was sent a link. It can afford web fonts and interaction that the offline copy cannot, and it should use them.

> **WATCH OUT FOR**
>
> - `body` must set an explicit background from a token. A transparent body borrows the host page's ground and one theme becomes unreadable.
> - Only the CDN allowlist is reachable. Google Fonts works; other font hosts fail silently and fall back.

Related: `scripts/docs/docs.css` · `scripts/docs/build-docs.mjs`

scripts/docs/build-docs.mjs

676 lines

Assembles the site. It merges the authored content with the extracted API surface, **fails the build when a repository file has no documentation entry**, and emits fourteen pages

plus a single-page print sheet and the hosted manual.

The completeness gate

Why this documentation cannot drift — TYPESCRIPT

```
const collective = Object.keys(FILE_DOCS).filter((f) => f.endsWith('/'));
const coveredCollectively = (file) => collective.some((dir) => file.startsWith(di

const undocumented = repoFiles.filter((f) => !FILE_DOCS[f] && !coveredCollectivel
if (undocumented.length) process.exit(1);
```

It also fails in the other direction: an entry for a file that no longer exists is an error too, so deleting a file forces its documentation to be deleted with it. A key ending in `/` covers everything beneath it, for directories documented as a group.

Outputs

FILE	WHAT IT IS
`docs/site/<page>.html`	Fourteen standalone pages with a sidebar, an on-this-page index and a pager
`docs/site/print.html`	Everything in one document, with a cover and a contents page — the PDF's source
`docs/site/artifact.html`	The hosted manual: numbered rail, cross-document search, theme toggle, chapter switching

Generated pages

Two pages have no authored prose. **Project structure** is built from the group of every documented file, and **API index** is built from every extracted export, filterable, each linked to its definition.

WHEN YOU NEED TO CHANGE IT

- You add a source file — the build will tell you.
- You want a new page, or a different grouping of the reference pages.
- The hosted manual needs different behaviour.

HOW TO CHANGE IT

1. Add the entry in the right `scripts/docs/content/files.*.mjs`, keyed by the repository-relative path, with a `group`.
2. Add a page by adding prose to `guides.mjs` and an entry to `PAGES`.

   ```
   { id: 'migration', title: 'Migration', subtitle: 'Moving an existing suite on
   ```
3. Add a group by adding it to `GROUP_TITLES` and to a page's `groups` array. A group in neither is silently invisible.
4. Rebuild.

```
npm run docs
```

WHY THE CHANGE BELONGS HERE

The gate is what makes this document trustworthy. Documentation that *may* be complete is documentation nobody relies on; documentation that cannot build unless it is complete is documentation people use.

WATCH OUT FOR

- Keys are repository-relative paths and must match exactly — `src/core/http.client.ts`, not `./src/core/http.client.ts`.
- A group present in `GROUP_TITLES` but absent from every page's `groups` renders nowhere, with no warning.
- `.husky/` is excluded from extraction, so the hooks are documented in the `package.json` entry rather than as files of their own.

Related: `scripts/docs/content/` · `scripts/docs/render.mjs` · `scripts/docs/build-pdf.mjs`

scripts/docs/build-pdf.mjs

58 lines

Renders `print.html` to PDF using the Chromium that ships with Playwright, so the PDF and the HTML cannot diverge — they are the same document, printed.

The step that is easy to forget — TYPESCRIPT

```typescript
// Expand every collapsed API section so nothing is missing from the printed copy
await page.evaluate(() => {
  document.querySelectorAll('details').forEach((element) => element.setAttribute(
});
await page.emulateMedia({ media: 'print' });
```

A `<details>` element that is closed when the page is printed simply is not in the PDF. Opening them all first is what makes the printed API reference complete.

The output is A4 with running headers and footers, `printBackground: true` so the callout and code backgrounds survive, and a copy placed in `docs/site/` so the download link works from the local site.

WHEN YOU NEED TO CHANGE IT

- The page size, margins or running heads should change.
- The PDF is missing content that is on the site.

HOW TO CHANGE IT

1. Change the format or margins in the `page.pdf()` call.

```
format: 'Letter', margin: { top: '18mm', bottom: '18mm', left: '16mm', right:
```

2 If content is missing, look for something collapsed or hidden by CSS — the `@media print` and `body.print` rules in `docs.css` are the usual cause.

WHY THE CHANGE BELONGS HERE

Using the browser that already ships with the framework means no second rendering engine to install, and a PDF that matches what the site shows, character for character.

WATCH OUT FOR

- It needs Chromium: `npx playwright install chromium`. The CI docs job does that explicitly.
- Page breaks are controlled from `docs.css`. `page-break-inside: avoid` on a large block leaves big blank gaps and inflates the page count — which is why `.file` deliberately does not use it.

Related: `scripts/docs/build-docs.mjs` · `scripts/docs/docs.css`

scripts/docs/content/

The authored prose, split across one guides module and six file modules. This is the half of the documentation that no extractor could produce: the reasoning, the trade-offs, and the "why here rather than there".

MODULE	COVERS
`guides.mjs`	Overview, architecture, playbooks, conventions, troubleshooting, glossary
`files.root.mjs`	Root configuration and the README
`files.tooling.mjs`	CI, containers, editor settings, and this generator
`files.foundation.mjs`	`src/config`, `src/core`, `src/types`
`files.auth.mjs`	`src/auth`, `src/contracts`
`files.protocols.mjs`	`src/protocols`, `src/services`, `src/fixtures`, `src/hooks`
`files.utils.mjs`	`src/utils`
`files.support.mjs`	`src/mocks`, `src/reporters`, `src/data`, the tests folder

The shape of a file entry

Every field earns its place — TYPESCRIPT

```
'src/core/http.client.ts': {
  group: 'core',
  purpose: 'One sentence: what it is for and why it exists.',
  blocks: [ /* optional: tables, code, callouts */ ],
  changeWhen: ['The situations that bring somebody to this file'],
  changeHow: [{ text: 'A step', code: 'the code for it' }],
  why: 'Why this change belongs in this file rather than somewhere else.',
  gotchas: ['What bites people'],
  related: ['neighbouring files'],
}
```

`purpose`, `changeWhen`, `changeHow` and `why` together answer the question the reader actually has: *I need to change something — where do I go, what do I do, and why there?*

WHEN YOU NEED TO CHANGE IT

- You add or change a source file.
- A gotcha bites somebody twice — write it down the second time.

HOW TO CHANGE IT

1. Add the entry to the module matching the file's area, keyed by its repository-relative path.
2. Rebuild. The gate will tell you if anything is still missing.

```
npm run docs
```

WHY THE CHANGE BELONGS HERE

Keeping the prose beside the generator rather than in the source files means a documentation edit never touches code, so it never needs a code review — and the reasoning has room to be a paragraph rather than a comment.

WATCH OUT FOR

- A key must match the path exactly, or the gate reports the file as undocumented and the entry as orphaned — two errors for one typo.
- Content is escaped before markup is applied. Use the inline forms — backticks, `**`, `text` — rather than raw HTML.

Related: `scripts/docs/build-docs.mjs` · `scripts/docs/render.mjs`

scripts/docs/docs.css

The stylesheet for the local site and the print sheet. Inlined into every page, so the documentation opens from a `file://` URL with no network — a reference nobody can

read offline is a reference nobody reads.

The identity is an instrument panel: the subject is wire protocols, so the page is organised like a trace — monospace structural labels, one teal signal colour, and semantic bands reserved for things that actually have a status. System font stacks throughout, because a web font would be the one thing on the page that needs a network.

Three palettes

SELECTOR	APPLIES TO
`:root`	The complete light palette. Every token has a value here.
`@media (prefers-color-scheme: dark)` under `:root:not([data-theme='light'])`	Dark mode, tokens only
`body.print`	The print sheet: ink-safe colours pinned so the dark media query cannot reach a printer

CAREFUL Never give a colour its only definition inside a media query. The un-stamped default state would then have no value for it, and the page renders one theme's text on the other theme's ground.

WHEN YOU NEED TO CHANGE IT

- A new block type needs styling.
- The palette or type scale should change.
- The PDF has layout problems.

HOW TO CHANGE IT

1. Change a token rather than a rule wherever possible — the token is defined in three places and the rule in one.
2. For a PDF problem, work in `body.print` and the `@media print` block. Reproduce it with `npm run docs:pdf` rather than by guessing.
3. Mirror any structural addition into `artifact.css`.

WHY THE CHANGE BELONGS HERE

Inlining the CSS is what makes a page a genuinely self-contained artefact: one file that can be emailed, archived or opened on a machine with no network and still look right.

WATCH OUT FOR

- `body.print` re-declares the palette rather than inheriting it. A new token added to `:root` needs a print value too, or it falls back to the dark palette on paper.
- Page-break rules interact badly with large blocks; see the comment on `.file`.

Related: `scripts/docs/artifact.css` · `scripts/docs/build-docs.mjs` · `scripts/docs/highlight.mjs`

scripts/docs/extract-api.mjs

177 lines

Reads the TypeScript source with the compiler API and writes `docs/generated/api.json`: every exported class, function, interface, type and constant, with its signature and its JSDoc summary.

The reason it uses the compiler rather than a regular expression is that the extracted surface is then *the code itself*. A signature in this documentation cannot be out of date, because nobody typed it.

What it produces per file — JSON


```
{
  "path": "src/core/api.response.ts",
  "bytes": 18422,
  "lines": 512,
  "exports": [
    {
      "kind": "class",
      "name": "ApiResponse",
      "doc": "A completed HTTP exchange…",
      "members": [{ "kind": "method", "name": "expectStatus", "signature": "expec
      "privateCount": 4
    }
  ]
}
```


Private and protected members are counted but not listed: they are not part of the contract, and listing them would suggest they are.

WHEN YOU NEED TO CHANGE IT

- A TypeScript construct is being missed — an enum, a namespace, a re-export form.
- You want more detail per member, such as parameter documentation.

HOW TO CHANGE IT

1. Add a branch in the statement walk, matching the AST node kind.

```
else if (ts.isEnumDeclaration(node) && isExported(node)) {
  entry.exports.push({ kind: 'enum', name: node.name.getText(source), doc: js
}
```

2. Add the rendering for the new kind in `render.mjs` and a badge style in both stylesheets.
3. Regenerate and check the count moved.

```
npm run docs:api
```

WHY THE CHANGE BELONGS HERE

Hand-written API documentation is wrong within a month. Extracted documentation is wrong only if the extractor is wrong, and the extractor is one file.

WATCH OUT FOR

- It reads the AST, not the type checker, so an inferred return type appears as absent. That is why exported functions are required to annotate theirs.
- The walker skips `node_modules`, `.git`, `dist`, `reports`, `test-results` and `.husky`. A new generated directory needs adding, or it will be treated as undocumented source.

Related: `scripts/docs/build-docs.mjs` · `scripts/docs/render.mjs`

scripts/docs/highlight.mjs

526 lines · 3 exports

Build-time syntax highlighting for eight languages. It emits `<span class="tok-*">` markup; the colours live in the stylesheets, which is what lets each output theme the same tokens differently.

Highlighting happens during the build rather than in the browser. That is the only approach that works here: a browser-side highlighter cannot colour the PDF at all, and would add a CDN dependency to pages meant to open offline.

LANGUAGE	DETECTED BY
TypeScript	The fallback — anything not matched by another rule
HTTP	A request line or a status line
GraphQL	An operation or fragment keyword followed by a selection set
Dockerfile	An instruction keyword at the start of a line
JSON	Quoted keys with no JavaScript syntax
Env	`KEY=value` lines with no shell command
Shell	A known command at the start of a line
YAML	`key:` lines with no statement punctuation

The subtle case

Why LOOKS_LIKE_CODE exists — TYPESCRIPT

```
const LOOKS_LIKE_CODE =
  /=>|\b(?:const|let|var|function|await|async|import|export|class|interface|exten
```

A TypeScript object property such as `maxUploadMb: raw.MAX_UPLOAD_MB,` is indistinguishable from a YAML key by shape alone. Statement punctuation and JavaScript keywords separate them. The YAML detector also has to *allow* braces, because GitHub Actions files are full of `${{ }}`.

Token classes

CLASS	TOKEN
`tok-c`	comment
`tok-k`	keyword
`tok-s`	string
`tok-n`	number
`tok-f`	function or field
`tok-t`	type
`tok-p`	property or header name
`tok-o`	operator or punctuation
`tok-v`	variable
`tok-r`	regular expression
`tok-a`	flag or directive

Each lexer is a single master pattern with named groups, scanned once. One pass avoids the double-escaping bugs that come from highlighting already-escaped HTML.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A snippet is detected as the wrong language.
> - A language needs adding.
> - A token is not coloured.

> **HOW TO CHANGE IT**
>
> 1. Confirm what is actually being detected before changing anything.
>
>    ```
>    node -e "import('./scripts/docs/highlight.mjs').then(m => console.log(m.detec
>    ```
>
> 2. To add a language: write the master pattern, write the lexer, register it in `LEXERS`, add a label to `LANGUAGE_LABEL`, and add a detection rule ordered before the fallbacks.
> 3. Override detection per block when a snippet is genuinely ambiguous.
>
>    ```
>    { type: 'code', lang: 'graphql', text: '…' }
>    ```
>
> 4. Add token colours to **all** palettes: `:root`, the dark media query and `body.print` in `docs.css`; `:root`, the media query and `[data-theme="dark"]` in `artifact.css`.

> **WHY THE CHANGE BELONGS HERE**
>
> Colour is not decoration in a document that is mostly code. It is what lets a reader find the string, the type and the comment without reading every character — and it has to work in print, which rules out doing it in the browser.

> **WATCH OUT FOR**
>
> - Detection order matters. HTTP and GraphQL are checked before the `key:` heuristics, or a header line would be read as YAML.
> - A colour defined only inside a media query never applies in the un-stamped default state. Every token needs a value on bare `:root`.
> - The print palette is separate and deliberately darker; a colour that reads well on screen can be invisible on paper.

Related: `scripts/docs/render.mjs` · `scripts/docs/docs.css` · `scripts/docs/artifact.css`

API SURFACE (3 EXPORTS)

`FUNCTION` **detectLanguage(code)**

`CONST` **LANGUAGE_LABEL**

`FUNCTION` **highlight(code, lang)**

Highlights a snippet. Pass `lang` to override the detected language. Returns HTML-escaped markup ready to place inside <pre><code>.

scripts/docs/render.mjs

197 lines · 6 exports

Turns content blocks and extracted API entries into HTML. Shared by the multi-page site, the print sheet and the hosted manual, so all three are generated from one source and cannot disagree.

Block types

TYPE	RENDERS
`h2`, `h3`, `p`	Headings with slugged ids, and paragraphs with inline markup
`ul`, `ol`	Lists
`code`	A syntax-highlighted figure with a caption and a language chip
`note`, `warn`, `rule`	Callouts, each with its own colour band
`table`	A table inside its own horizontally scrolling container

TYPE	RENDERS
`steps`	A numbered procedure, each step optionally carrying code
`tree`	Pre-formatted, deliberately unhighlighted — for directory diagrams

Inline markup supports `` `code`, **bold** and *text* ``. Everything is escaped first, so content can contain angle brackets safely.

Where highlighting is applied

Not only in code blocks: step snippets, API member signatures, interface shapes and type definitions all go through the highlighter, so a signature in a reference table reads with its parameters and return type distinguished.

> **WHEN YOU NEED TO CHANGE IT**
>
> - You need a block type that does not exist.
> - The rendering of a file entry or an API item should change.

> **HOW TO CHANGE IT**
>
> 1. Add a case to `renderBlock`. The `default` branch throws, so a typo in a block type fails the build rather than silently rendering nothing.
>
> ```
> case 'diagram':
>   return `<figure class="diagram">${esc(block.text)}</figure>`;
> ```
>
> 2. Add the styles to **both** `docs.css` and `artifact.css`. They are separate files on purpose — the two outputs have different layouts.

> **WHY THE CHANGE BELONGS HERE**
>
> One renderer for three outputs is what guarantees the PDF, the site and the hosted manual say the same thing. Three renderers would drift within a release.

> **WATCH OUT FOR**
>
> - `inline()` escapes before applying markup, so raw HTML in content is shown rather than rendered. That is intentional.
> - A new block type needs styles in both stylesheets, or it will look correct in one output and unstyled in the other.

Related: `scripts/docs/highlight.mjs` · `scripts/docs/build-docs.mjs` · `scripts/docs/docs.css`

API SURFACE (6 EXPORTS)

FUNCTION **`esc(value = '')`**

FUNCTION **`inline(text = '')`**

Inline markup: `code`, **bold**, *text*.

FUNCTION **renderBlocks(blocks = [])**

FUNCTION **slug(text = '')**

FUNCTION **fileAnchor(filePath)**

FUNCTION **renderFileEntry(filePath, doc, apiEntry)**

Renders one documented file: purpose, change guidance and API.

Reference: config, core & types

The request engine and the foundation every other layer stands on

Configuration (src/config)

src/config/env.config.ts 222 lines · 5 exports

The only file in the framework that reads `process.env`. It loads the env files, validates everything against a Zod schema, and exports a frozen `config` object plus `getUser(role)`.

What it does, in order

1. Loads `.env`, then `.env.<TEST_ENV>` on top of it. Neither overrides a variable already exported in the shell — that is how CI injects secrets without touching a file.
2. Validates every variable against the schema, coercing types and applying defaults.
3. Throws once, listing every problem by name, if anything is wrong.
4. Resolves the named environment and freezes the result.

Coercion helpers

Environment variables are always strings, and an unset one is an empty string rather than `undefined`. Three `z.preprocess` helpers absorb that so the schema can describe intent rather than string handling.

booleanish — a blank value falls back rather than becoming false TYPESCRIPT

```typescript
function booleanish(fallback: boolean): z.ZodType<boolean, z.ZodTypeDef, unknown>
  return z.preprocess((value) => {
    const raw = asString(value);
    if (raw === '') return fallback;
    return ['1', 'true', 'yes', 'on'].includes(raw.toLowerCase());
  }, z.boolean());
}
```

`integerish` and `optionalString` follow the same pattern. The alternative — `.optional().default()` — does not type correctly in Zod 3 for a value arriving as `unknown`.

Credentials

getUser fails with the fix, not with a 401 TYPESCRIPT

```typescript
export function getUser(role: UserRole): UserCredentials {
  const entry = users[role];
  if (!entry.username || !entry.password) {
    throw new Error(
      `No credentials for role "${role}". Set ${role.toUpperCase()}_USERNAME and
        `${role.toUpperCase()}_PASSWORD in .env (or as CI secrets).`,
    );
  }
  return { role, username: entry.username, password: entry.password };
}
```

Throwing here rather than returning empty strings turns a configuration mistake into a readable message instead of an unexplained 401 twenty tests later. `hasUser(role)` is the non-throwing form, for tests that should skip rather than fail.

apiUrl

Applies the environment's version prefix to a relative path, passes absolute URLs through untouched, and does not double-apply a prefix that is already present.

> **WHEN YOU NEED TO CHANGE IT**
>
> - You add an environment variable.
> - A default is wrong for most people.
> - A new credential role is needed.

> **HOW TO CHANGE IT**
>
> 1. Add the field to the schema with the right helper.
>
>    ```
>    TENANT_ID: optionalString(),
>    MAX_UPLOAD_MB: integerish(25),
>    ```
>
> 2. Expose it on `config` under a name that reads well at the call site.
>
>    ```
>    tenantId: raw.TENANT_ID,
>    maxUploadMb: raw.MAX_UPLOAD_MB,
>    ```
>
> 3. Document it in `.env.example` — a variable that exists only here is a variable nobody discovers.
> 4. For a new role, extend `UserRole` and the `users` map. The compiler finds every place that has to change.

> **WHY THE CHANGE BELONGS HERE**
>
> One reader for the environment is what makes secret handling auditable: reviewing this one file tells you everything the suite can read. It also means an invalid configuration fails once, loudly, at start-up rather than as a confusing `undefined` inside a request.

WATCH OUT FOR

- `config` is frozen. A test cannot mutate the run's settings and leave a later test pointing somewhere else.
- Zod is pinned to v3 deliberately; v4's CommonJS build fails under Playwright's transform.
- A variable added to `.env.example` but not to the schema is silently ignored.

Related: `src/config/environments.ts` · `.env.example` · `src/fixtures/api.fixture.ts`

API SURFACE (5 EXPORTS)

TYPE **UserRole**

```
'admin' | 'standard' | 'readonly'
```

CONST **config**

Resolved, validated configuration. Frozen so a test cannot mutate the run's settings and leave a later test running against a different target.

FUNCTION **getUser(role: UserRole): UserCredentials**

Credentials for a role, or a precise error naming the two variables to set. Throwing here — rather than returning empty strings — turns a configuration mistake into a readable message instead of a 401 nobody can explain.

FUNCTION **hasUser(role: UserRole): boolean**

True when a role has usable credentials — use to skip rather than fail.

FUNCTION **apiUrl(pathname: string): string**

Absolute URL for a path, applying the environment's version prefix.

src/config/environments.ts

159 lines · 5 exports

The table of named targets — `demo`, `local`, `mock`, `dev`, `staging`, `production` — holding URLs, path prefixes, TLS behaviour, latency budgets and the read-only flag, plus the public APIs the demonstration suite reaches alongside them.

RULE Only non-secret values belong here. Anything that would be dangerous in a pull request — keys, passwords, tokens — is read from the process environment instead. This file is committed and reviewed like any other.

Fields

FIELD	EFFECT
`apiBaseUrl`	Root of the REST API, without a trailing slash
`graphqlUrl`, `wsUrl`	Endpoints for the non-REST protocols, when the target has them
`apiPrefix`	Version prefix applied to relative paths — `/v1`, or empty
`verifyTls`	Only ever false for local stubs
`latencyBudgetMs`	What `expectWithinLatencyBudget()` compares against
`readOnly`	**The client refuses to send any mutating verb.** Set on `production`.

The type is enforced at compile time — TYPESCRIPT

```ts
export const ENVIRONMENTS = {
  staging: { /* … */ },
} as const satisfies Record<string, EnvironmentDefinition>;

export type EnvironmentName = keyof typeof ENVIRONMENTS;
```

`satisfies` checks each entry against the interface while keeping the literal types, so `EnvironmentName` is the union of the actual keys — and the Zod enum in `env.config.ts` is built from the same list. Adding an environment makes it a valid `TEST_ENV` automatically.

WHEN YOU NEED TO CHANGE IT

- A new deployment needs testing.
- A URL, prefix or budget changes.
- An environment should become read-only.

HOW TO CHANGE IT

1. Add the entry. A missing field is a compile error, not a run-time surprise.

```ts
sandbox: {
  name: 'sandbox',
  apiBaseUrl: 'https://api.sandbox.example.com',
  apiPrefix: '/v2',
  verifyTls: true,
  latencyBudgetMs: 4000,
  readOnly: false,
},
```

2. Point at it.

```
TEST_ENV=sandbox npx playwright test
```

> **WHY THE CHANGE BELONGS HERE**
>
> Environment definitions are configuration, not secrets. Keeping them in code means they are type-checked, reviewable, and discoverable — none of which is true of a URL living in somebody's shell profile.

> **WATCH OUT FOR**
>
> - `readOnly` is a real guard, not documentation. The client throws before sending a POST, PUT, PATCH or DELETE.
> - A trailing slash on `apiBaseUrl` produces double slashes in URLs. `config.baseUrl` strips them, but keep the table clean.

Related: `src/config/env.config.ts` · `src/core/http.client.ts`

API SURFACE (5 EXPORTS)

INTERFACE EnvironmentDefinition

A deployment the suite can be pointed at.

```ts
readonly name: string;
readonly apiBaseUrl: string;
readonly graphqlUrl?: string;
readonly wsUrl?: string;
readonly apiPrefix: string;
readonly verifyTls: boolean;
readonly latencyBudgetMs: number;
readonly readOnly: boolean;
readonly requiresLiveTarget: boolean;
```

CONST ENVIRONMENTS

TYPE EnvironmentName

```ts
keyof typeof ENVIRONMENTS
```

CONST PUBLIC_APIS

Additional public APIs the demonstration suite reaches directly. These are *not* environments, because `TEST_ENV` selects exactly one target and these are used alongside it. A spec reaches them with `http.withBaseUrl(PUBLIC_APIS.jsonPlaceholder)`, which produces a derived client that keeps the run's observers — latency, recording, contract guard — attached. Each one is here because it demonstrates something the primary target cannot, and for no other reason.

CONST ENVIRONMENT_NAMES

src/config/timeouts.ts

42 lines · 1 exports

Named time budgets. Tests and helpers refer to these constants instead of writing raw numbers, so a slow environment is retuned in one place and every wait in the suite carries an explanation of what it is waiting for.

CONSTANT	VALUE	FOR
INSTANT	1s	A health check or a cached read. Anything slower is a red flag.
SHORT	5s	A normal single-resource read
MEDIUM	15s	A write, or a read that fans out to other services
LONG	30s	Report generation, bulk import, anything queued
EXTRA_LONG	120s	A large upload or download
TEST / HOOK / EXPECT	60s / 90s / 10s	Whole-test, hook and assertion budgets
POLL_TIMEOUT / POLL_INTERVAL	30s / 500ms	How `waitFor` behaves
RETRY_BASE_DELAY / RETRY_MAX_DELAY	300ms / 5s	Exponential backoff bounds
STREAM_IDLE / SOCKET_MESSAGE	15s / 10s	How long a stream or socket waits for the next message

The names are the documentation. `TIMEOUTS.MEDIUM` says "this is a write"; `15000` says nothing.

WHEN YOU NEED TO CHANGE IT

- An environment is systematically slower.
- A new category of wait appears.

HOW TO CHANGE IT

1. Adjust the named budget rather than a call site, so every wait of that kind moves together.
2. Add a constant for a genuinely new category, with a comment saying what it is waiting for.

```
/** How long a webhook receiver waits for a callback to arrive. */
WEBHOOK_DELIVERY: 45_000,
```

WHY THE CHANGE BELONGS HERE

A raw number at a call site is a decision nobody can review. A named budget is a decision with a reason attached, and it can be changed once for the whole suite.

WATCH OUT FOR

- Raising a timeout to make a flaky test pass converts a fast failure into a slow one. Find out what is slow first — the latency report is attached to every test.

Related: `playwright.config.ts` · `src/core/http.client.ts`

API SURFACE (1 EXPORT)

`CONST` **TIMEOUTS**

Named time budgets. Tests and helpers refer to these constants instead of writing raw numbers, so a slow environment is retuned in one place and every magic number in the suite carries an explanation of what it is waiting for.

Core (src/core)

src/core/api.response.ts

516 lines · 2 exports

The response wrapper — the highest-leverage class in the framework, since every assertion a test makes goes through it.

Two decisions shape it

The body is a snapshot. It is captured into a buffer before this object exists, so it can be read as text, JSON, NDJSON, XML or bytes, in any order, as many times as a test likes, after the connection has closed. That is what makes every method synchronous and chainable.

Assertions live here rather than in free functions, so a failing check can include the request, the status, the timing and a body excerpt. An assertion that only says "expected 200, got 500" costs somebody twenty minutes; one that shows the error payload costs nothing.

Readers

METHOD	RETURNS
`text()` / `buffer()`	The body as a string or bytes
`json<T>()`	Parsed JSON. Throws with a body excerpt when it is not JSON — an HTML error page where JSON was expected is a common and otherwise baffling failure.
`jsonOrNull<T>()` / `isJson()`	The non-throwing forms
`ndjson<T>()`	One parsed value per line, with the line number in any error
`xml()` / `fault()`	Parsed XML, and a SOAP fault when the body carries one
`parse(schema)`	Validated **and typed**. The method to reach for when the test needs the payload.

Parsing is memoised: a test that reads `.json()` in five assertions pays for one parse.

Navigation

Paths, not chains of optional access — TYPESCRIPT

```typescript
response.path('data.items[0].id');   // one value
response.paths('..email');           // every match, at any depth
response.has('meta.cursor');         // presence
response.fields();                   // every leaf path — useful for shape drift
```

Assertions

Fifteen, all chainable, all funnelling through one private `assert`. That is why every failure in the suite has the same anatomy.

The failure message every assertion produces — TYPESCRIPT

```
Expected status to be 201.
  Request : POST https://api.staging.example.com/v1/users
  Status  : 422 Unprocessable Entity
  Timing  : 143ms
  Actual  : 422
```

Soft mode

Report every violation, not just the first — TYPESCRIPT

```typescript
response.soft()
  .expectStatus(200)
  .expectHeader('etag')
  .expectPath('data.id')
  .expectWithinLatencyBudget();
```

`soft()` returns a new wrapper over the same snapshot whose assertions use `expect.soft`. For checking many properties of one payload, that turns four runs into one.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A recurring assertion deserves a name.
> - A new payload format needs a reader.
> - Failure messages should carry more context.

> **HOW TO CHANGE IT**
>
> 1 Add an assertion by delegating to `assert` — you get the standard message anatomy for free.
>
> ```typescript
> expectCursor(): this {
>   this.assert(this.has('nextCursor'), 'a pagination cursor', this.jsonOrNull(
> ```

```
    return this;
  }
```

2 Add a reader as a plain method over `this.snapshot.body`. It can be synchronous, because the body is already in memory.

3 To change every failure message, edit the private `assert`. One edit, whole suite.

WHY THE CHANGE BELONGS HERE

This class is where a test spends most of its time, so its ergonomics set the ergonomics of every test in the repository. Time spent here is repaid at every call site.

WATCH OUT FOR

- `json()` throws on a non-JSON body — deliberate, and the message shows the content type and an excerpt.
- `parse()` narrows the type; `json<T>()` only asserts one. Prefer `parse` when the shape matters.
- `expectSecurityHeaders()` checks a defensible baseline, not a policy. Adjust `REQUIRED_SECURITY_HEADERS` to your own.
- A response is immutable. `soft()` returns a new wrapper rather than changing this one.

Related: `src/core/http.client.ts` · `src/utils/jsonpath.utils.ts` · `src/fixtures/custom-matchers.ts`

API SURFACE (2 EXPORTS)

INTERFACE **ResponseSnapshot**

Everything captured from the wire. Immutable once the response is built.

```
readonly status: number;
readonly statusText: string;
readonly headers: HeaderMap;
readonly body: Buffer;
readonly url: string;
readonly request: RequestSpec;
readonly timing: RequestTiming;
```

CLASS ApiResponse

A completed HTTP exchange. The type parameter is the expected JSON shape. It is a convenience for autocomplete, not a guarantee — use `parse(schema)` when you need the shape actually checked at runtime.

`get status: number`	
`get statusText: string`	
`get ok: boolean`	True for 2xx. Mirrors `Response.ok` in the Fetch API.
`get url: string`	
`get headers: HeaderMap`	All response headers, keys already lower-cased.
`get request: RequestSpec`	The request that produced this response — useful in failure messages.
`get timing: RequestTiming`	
`get durationMs: number`	Round-trip duration in milliseconds.
`get size: number`	Payload size in bytes, as received.
`header(name: string): string | undefined`	Case-insensitive header lookup.
`contentType(): string`	
`cookies(): ParsedCookie[]`	Parsed `Set-Cookie` entries, with their attributes.
`cookie(name: string): ParsedCookie | undefined`	
`rateLimit(): RateLimitInfo`	Rate-limit counters, normalised across the common header spellings.
`nextPageUrl(): string | undefined`	The `next` URL from an RFC 8288 `Link` header, when present.
`buffer(): Buffer`	Raw bytes. Use for binary downloads, checksums and content-length checks.
`text(): string`	
`json(): J`	The body as JSON. Throws with a body excerpt when parsing fails — an HTML error page returned where JSON was expected is a common and otherwise very confusing failure.
`jsonOrNull(): J | undefined`	The body as JSON, or `undefined` when it is empty or not JSON at all.
`isJson(): boolean`	True when the body parses as JSON.
`ndjson(): J[]`	Newline-delimited JSON, one parsed value per non-empty line.
`xml(): UnknownRecord`	The body parsed as XML, with namespace prefixes stripped.
`fault(): SoapFault | undefined`	A SOAP fault when the body carries one — SOAP reports errors in the body.
`path(jsonPath: string): V | undefined`	A single value by path, e.g. `data.items[0].id` or `..email`.

`paths(jsonPath: string): V[]`	Every value a path matches — wildcards and recursive descent return many.
`has(jsonPath: string): boolean`	True when a path resolves to something.
`fields(): string[]`	Every leaf path in the payload — useful for detecting shape drift.
`parse(schema: S, schemaName = 'response'): z.infer<S>`	Validates and returns the body with the schema's output type. This is the method to reach for when a test needs the payload, because it makes the shape a checked fact rather than an assumption: everything after it is fully typed and cannot silently be `undefined`.
`soft(): ApiResponse<T>`	Switches to soft assertions: every following check is recorded and the test continues, so one run reports all the contract violations in a payload rather than only the first.
`expectStatus(...codes: number[]): this`	Asserts the exact status, or one of several acceptable ones.
`expectOk(): this`	Asserts any 2xx.
`expectClientError(): this`	Asserts a client error — the whole 4xx band, when the exact code varies.
`expectServerError(): this`	
`expectHeader(name: string, expected?: string | RegExp): this`	Asserts a header exists, and optionally matches a value or pattern.
`expectContentType(expected: string): this`	Asserts the media type, ignoring `charset` and other parameters.
`expectJson(): this`	Asserts the body is JSON — catches HTML error pages early.
`expectPath(jsonPath: string, expected: unknown): this`	Asserts a value at a path. Accepts a literal, a RegExp or a predicate.
`expectPathExists(jsonPath: string): this`	Asserts a path resolves to something.
`expectSubset(expected: UnknownRecord): this`	Asserts the payload contains at least these fields, ignoring extras.
`expectSchema(schema: S, schemaName = 'response'): this`	Validates against a Zod schema and reports every violation at once.
`expectFasterThan(milliseconds: number): this`	Asserts the response arrived inside an explicit budget.
`expectWithinLatencyBudget(): this`	Asserts the response met the environment's configured latency budget.
`expectEmptyBody(): this`	Asserts an empty body — the correct shape for 204 and most DELETEs.
`expectSecurityHeaders(): this`	Asserts the security headers a public API is expected to send.
`toRecord(): ExchangeRecord`	A loggable record of the exchange, with credentials redacted.
`attachTo(testInfo: TestInfo, name = this.request.label): Promise<void>`	Attaches the full exchange to the HTML report. Worth doing for any request whose failure would need investigation: the attachment survives in CI long after the process is gone.

`describe(): string`	One-line summary used in step titles and logs.

src/core/base.service.ts

126 lines · 1 exports

The base class for service objects: path joining, request helpers scoped to `basePath`, reporter steps, and access to the cleanup registry.

A service object is to an API suite what a page object is to a UI suite. When `/v1/users` becomes `/v1/accounts`, exactly one file changes — not every test that happened to mention users.

What a subclass provides — TYPESCRIPT

```typescript
export class OrderService extends BaseService {
  protected readonly basePath = '/orders';

  async create(overrides: Partial<Order> = {}): Promise<Order> {
    return this.step('create an order', async () => {
      const response = await this.post().json(buildOrder(overrides)).expectStatus
      const order = response.parse(OrderSchema, 'order');
      return this.track(order, `order ${order.id}`, () => this.remove(order.id));
    });
  }
}
```

What the base provides

MEMBER	DOES
`get, post, put, patch, del`	A builder for a path relative to `basePath`, pre-labelled for the report
`path(sub)`	Joins `basePath` and a sub-path, tolerating a leading slash on either
`step(title, body)`	Wraps a multi-request operation in one reporter step, so the report reads as intent rather than as four anonymous calls
`track(value, description, remove)`	Registers a deletion and passes the value through, so creation and cleanup are one expression
`send(builder)`	The escape hatch for a call that needs the response itself

RULE A service method returns a domain value and never asserts. Assertions belong in tests, so a negative-path test can reuse the service instead of fighting it. Where a test needs the response — a `Location` header, a status — expose a `raw*` method rather than forcing the test to bypass the service.

> **WHEN YOU NEED TO CHANGE IT**
>
> - Every service needs a capability — a shared header, a common retry policy, a standard pagination reader.

> **HOW TO CHANGE IT**
>
> 1 Add a protected method here rather than repeating it in each service.

```
protected async page<T>(sub: string, schema: z.ZodTypeAny): Promise<T[]> {
  return followOffset(async (offset, limit) => {
    const response = await this.get(sub).query({ offset, limit }).send();
    return response.parse(schema).items as T[];
  });
}
```

> **WHY THE CHANGE BELONGS HERE**
>
> Anything every service needs belongs here; anything one service needs belongs in that service. Keeping the base small is what stops it becoming a dumping ground that every service drags around.

> **WATCH OUT FOR**
>
> - `del` rather than `delete` — `delete` is a reserved word and cannot be a method name in this position.
> - `step()` falls back to calling the body directly when not inside a test, so services work from setup scripts too.
> - Each service gets its own registry unless one is passed in. The fixture passes the shared one, so a test's cleanup runs together.

Related: `src/services/template.service.ts` · `src/utils/cleanup.registry.ts` · `src/core/request.builder.ts`

API SURFACE (1 EXPORT)

CLASS **BaseService**

src/core/errors.ts

170 lines · 11 exports

The error hierarchy. Every failure the framework raises itself is one of these, so a test can distinguish "the API said no" from "the framework is misconfigured" from "the payload does not match its contract" — three problems with three different owners.

ERROR	MEANS	OWNER
`ConfigurationError`	The environment or a call is wrong. Raised before any request.	Whoever set the configuration
`HttpError`	A response arrived that `expectStatus` did not accept.	Usually the API

ERROR	MEANS	OWNER
`RequestTimeoutError`	No response within the timeout.	The API, or the network
`TransportError`	DNS, TLS, reset — no response at all.	Infrastructure
`SchemaValidationError`	A payload does not match its schema.	The API, or the schema
`ContractViolationError`	A response disagrees with the OpenAPI document.	The API, or the document
`AuthenticationError`	A token could not be obtained or refreshed.	Configuration, or the identity provider
`PollTimeoutError`	A protocol client waited for a frame that never arrived.	The API, or the expectation
`ReadOnlyEnvironmentError`	A write was attempted on a read-only environment.	The test

Every message is written to be actionable on its own, because in CI the message is often all anyone sees.

A message that names its own fix — TYPESCRIPT

```
Refusing to send POST https://api.example.com/v1/users: the "production"
environment is marked read-only in src/config/environments.ts. Point TEST_ENV
at a writable environment, or mark the test @read-only.
```

Two helpers live here because every error message needs them: `truncate` keeps a large body from filling a terminal, and `safeJson` stringifies anything without throwing on cycles or BigInt.

WHEN YOU NEED TO CHANGE IT

- A new failure mode deserves its own type.
- A message could be more actionable.

HOW TO CHANGE IT

1. Extend `FrameworkError` so `instanceof FrameworkError` keeps working.

```
export class RateLimitError extends FrameworkError {
  constructor(readonly retryAfterMs: number, url: string) {
    super(`Rate limited on ${url}; the server asked for ${retryAfterMs}ms. Lo
      `WORKERS, or seed with mapWithConcurrency.`);
  }
}
```

2. Write the message as advice, not as a description. "Set X in .env" beats "X is undefined".

WHY THE CHANGE BELONGS HERE

A typed hierarchy lets calling code react differently to different failures, and the shared base means a broad catch still works. The message quality is the real product: it is what somebody reads at 3am.

WATCH OUT FOR

- `Error.captureStackTrace(this, new.target)` keeps the stack pointing at the caller rather than at the constructor.
- Pass `{ cause: error }` when wrapping, or the original stack is lost — the `preserve-caught-error` lint rule enforces it.

Related: `src/core/http.client.ts` · `src/core/api.response.ts`

API SURFACE (11 EXPORTS)

CLASS **ConfigurationError** `extends FrameworkError`

The environment or `.env` file is wrong. Raised before any request is sent.

CLASS **HttpError** `extends FrameworkError`

A response arrived, but not one the caller was willing to accept.

```
status: number
method: HttpMethod
url: string
body: string
expected: number[]
```

CLASS **RequestTimeoutError** `extends FrameworkError`

The request never completed within its timeout.

CLASS **TransportError** `extends FrameworkError`

The transport failed: DNS, TLS, connection reset, unreachable host.

CLASS **SchemaValidationError** `extends FrameworkError`

A payload did not match the schema it was validated against.

CLASS **ContractViolationError** `extends FrameworkError`

The response disagreed with the OpenAPI document for that operation.

CLASS **AuthenticationError** `extends FrameworkError`

A token could not be obtained or refreshed.

CLASS **PollTimeoutError** `extends FrameworkError`

A `waitFor`-style helper gave up before its condition became true.

CLASS **ReadOnlyEnvironmentError** `extends FrameworkError`

A mutating verb was attempted against an environment marked read-only.

FUNCTION **`truncate(text: string, max: number): string`**

Keeps error messages readable when a server returns a very large body.

FUNCTION **`safeJson(value: unknown): string`**

`JSON.stringify` that never throws on cycles or BigInt.

src/core/http.client.ts

480 lines · 2 exports

The HTTP engine. Everything that must happen on every request lives here exactly once: URL resolution, credential injection, the read-only guard, retry with backoff, timing capture, logging, reporter steps and body capture.

It wraps Playwright's `APIRequestContext` rather than `fetch`, so requests share the run's proxy and TLS configuration, appear in traces, and are recorded in the HTML report alongside everything else the test did.

The retry structure

This is the single most important detail in the file. **Only the transport call is inside the try block.** Everything that happens once a response has arrived — recording it, enforcing the expected status, notifying the contract guard — sits outside it.

The shape that makes verdicts un-retryable TYPESCRIPT

```typescript
let snapshot: ResponseSnapshot;
try {
  const raw = await this.transport(spec, headers);
  snapshot = { /* status, headers, body, timing */ };
} catch (error) {
  if (attempt >= maxAttempts) break;
  await this.backoff(attempt, {}, spec);
  continue;
}

if (this.shouldRetry(spec, snapshot.status) && attempt < maxAttempts) {
  await this.backoff(attempt, snapshot.headers, spec);
  continue;
}

const response = new ApiResponse<T>(snapshot);
this.record(response);                    // observers — never retried
this.enforceExpectedStatus(spec, response);
return response;
```

NOTE Without that structure, a contract violation thrown by an observer would be caught by the retry handler and tried three times — three identical failures, and a report that suggests a transient problem where there is a real one.

What gets retried

CONDITION	RETRIED?
DNS, TLS, connection reset, timeout	Yes — a transport fault
408, 425, 429, 500, 502, 503, 504 on GET/HEAD/OPTIONS/PUT/DELETE	Yes — transient, and the verb is idempotent
The same statuses on POST/PATCH **with** an `Idempotency-Key`	Yes — the server promised to de-duplicate
The same statuses on POST/PATCH **without** one	No — a retry could create two resources
Any 4xx that is not 408 or 425	No — that is an answer
A failing contract check	No — structurally impossible, see above

Backoff

Exponential, capped at `RETRY_MAX_DELAY`, with jitter so parallel workers do not re-collide on the same rate limit — but a server-supplied `Retry-After` always wins, because the server knows better than the heuristic.

Derived clients

`withAuth`, `withHeaders` and `withBaseUrl` return copies. A copy **inherits the observers**, so a test that switches auth mid-way does not silently stop recording or validating.

The read-only guard

Checked before anything is sent TYPESCRIPT

```typescript
private guardReadOnly(spec: RequestSpec): void {
  if (!config.readOnly) return;
  if (!MUTATING_METHODS.includes(spec.method)) return;
  throw new ReadOnlyEnvironmentError(spec.method, spec.url, config.env);
}
```

A production smoke suite that accidentally POSTs is the kind of mistake that only happens once. The framework is the right place to make sure it happens zero times.

WHEN YOU NEED TO CHANGE IT

- A policy should apply to every request — a header, a correlation id, a circuit breaker.
- The retry rules need to change.
- A new body encoding is added.
- You need a new observer hook.

HOW TO CHANGE IT

1. For a per-request policy, edit `resolveHeaders` or `transport`. Both run for every request, which is the point of having them.
2. For retry behaviour, edit `RETRYABLE_STATUSES` or `shouldRetry`. Keep the idempotency reasoning intact.

   ```typescript
   const RETRYABLE_STATUSES = new Set([408, 425, 429, 500, 502, 503, 504, 522]);
   ```

3. For a body encoding, add a case to the switch in `transport` — the compiler will already be pointing at it.
4. For a new hook, follow `onResponse`: push to a list, return an unsubscribe, and copy the list in `derive()`.

WHY THE CHANGE BELONGS HERE

One engine means a policy is written once and cannot be forgotten. Spread across service objects, "always send a correlation id" becomes "send it wherever somebody remembered".

WATCH OUT FOR

- `failOnStatusCode: false` is always set. The framework never throws on status by itself — `expectStatus` and the test's assertions decide what is acceptable.
- Multipart deliberately deletes any `content-type`: the boundary is generated by the transport and a hand-written header will not match it.

- A header set explicitly on a request always wins over the auth provider, so a test can override credentials without swapping clients.
- `test.step` only works inside a running test; `inTest()` detects that so the client also works from a hook or a script.

Related: `src/core/request.builder.ts` · `src/core/api.response.ts` · `src/core/errors.ts`

API SURFACE (2 EXPORTS)

INTERFACE **AuthProvider**

Supplies credentials for outgoing requests. Implemented by `src/auth`.

```
readonly name: string;
headers(): Promise<HeaderMap>;
invalidate?(): void;
```

CLASS **HttpClient** implements RequestSender

`get(path: string): RequestBuilder<T>`	
`post(path: string): RequestBuilder<T>`	
`put(path: string): RequestBuilder<T>`	
`patch(path: string): RequestBuilder<T>`	
`delete(path: string): RequestBuilder<T>`	
`head(path: string): RequestBuilder<T>`	
`options(path: string): RequestBuilder<T>`	
`request(method: HttpMethod, path: string): RequestBuilder<T>`	Any verb, for tests that iterate over methods (CORS, 405 checks).
`withAuth(auth: AuthProvider | undefined): HttpClient`	A copy of this client that authenticates differently.
`withHeaders(headers: HeaderMap): HttpClient`	A copy with extra default headers — tenant ids, feature flags, locales.
`withBaseUrl(baseUrl: string): HttpClient`	A copy pointed at a different host — a second service, or a mock server.
`onExchange(listener: ExchangeListener): () => void`	Subscribes to every exchange this client completes.
`onResponse(listener: ResponseListener): () => void`	Subscribes to every response with its body intact. This is what automatic contract validation hooks into: it needs the real payload, which the redacted exchange record deliberately does not carry.
`resolveUrl(pathname: string): string`	
`defaults(): { headers: HeaderMap; timeout: number; retries: number }`	
`dispatch(spec: RequestSpec): Promise<ApiResponse<T>>`	Sends one request, applying every cross-cutting policy. Wrapped in a reporter step so the HTML report reads as a narrative of what the test did, with the status and duration on the step title.

src/core/request.builder.ts

400 lines · 2 exports

The fluent builder. It lets a call name only the dimensions it cares about, in any order, and lets a new dimension be added without touching a single existing test.

Why a builder

A request has more than a dozen independent dimensions. As one options object, every call site has to be read in full to find the two fields that matter, and a thirteenth dimension is a breaking change for everybody.

Thenable

Nothing is sent until the builder is awaited — TYPESCRIPT

```ts
await http.get('/users');                              // sends
await http.get('/users').query({ page: 2 }).send();    // also sends
const spec = http.get('/users').build();               // sends nothing
```

The surface

AREA	METHODS
URL	`param`, `params`, `query`, `queryParam`, `arrays`
Headers	`header`, `headers`, `withoutHeader`, `accept`, `contentType`, `bearer`, `basic`, `idempotencyKey`, `traceId`
Body	`json`, `form`, `multipart`, `file`, `fileFromBuffer`, `field`, `text`, `binary`
Policy	`timeout`, `retries`, `expectStatus`, `anonymous`, `noRedirect`, `as`, `meta`
Terminal	`build`, `send`, `then`

Details that prevent real bugs

- **An unfilled placeholder throws**, naming the parameter, rather than sending a request to a literal `/users/{id}`.
- **`arrays(format)`** chooses between `?tag=a&tag=b`, `?tag=a,b` and `?tag[]=a`. Servers disagree, and getting it wrong produces a silently empty filter rather than an error.
- **`idempotencyKey()` generates one when none is given**, which is what makes retrying a POST safe — and what `shouldRetry` looks for.
- **`assertBodyUnset`** throws if two body kinds are set, instead of silently sending one of them.
- **`anonymous()`** suppresses credential injection, so the 401 path can be tested without building a second client.

> WHEN YOU NEED TO CHANGE IT
>
> - A request needs a dimension that does not exist.
> - A new body encoding is added.
> - Query encoding needs another format.

> HOW TO CHANGE IT
>
> 1 Add a private field, a chainable method returning `this`, and the field in `build()`.

```
  private tenantId: string | undefined;

  tenant(id: string): this {
    this.tenantId = id;
    return this;
  }
```

2 Add it to `RequestSpec` in `src/types/index.ts` so the client can act on it. The compiler will point at every place that must keep up.

WHY THE CHANGE BELONGS HERE

Optionality is the whole design. A test that cares about nothing but the path writes `http.get('/users')`; a test that cares about eight things writes eight calls. Neither pays for the other.

WATCH OUT FOR

- A builder is single-use — it carries mutable state. Reusing one after `send()` re-sends the same request.
- `expectStatus` controls whether the *client* raises. Assertions in the test are still where the status should be checked.
- `RequestSender` is declared here rather than imported, to avoid an import cycle with the client.

Related: `src/core/http.client.ts` · `src/types/index.ts`

API SURFACE (2 EXPORTS)

INTERFACE **RequestSender**

Implemented by the HTTP client. Declared here to avoid an import cycle.

```
dispatch<T>(spec: RequestSpec): Promise<ApiResponse<T>>;
resolveUrl(path: string): string;
defaults(): { headers: HeaderMap; timeout: number; retries: number };
```

CLASS **RequestBuilder** `implements PromiseLike<ApiResponse<T>>`

`params(values: PathParams): this`	Fills `{id}` or `:id` placeholders in the path.
`param(name: string, value: string | number): this`	Fills a single path placeholder.
`query(values: QueryParams): this`	Merges query-string values. `undefined` and `null` entries are dropped.
`queryParam(name: string, value: QueryValue): this`	
`arrays(format: ArrayFormat): this`	How repeated query values are encoded. Servers disagree, and getting this wrong produces a silently empty filter rather than an error.
`headers(values: HeaderMap): this`	
`header(name: string, value: string): this`	
`withoutHeader(name: string): this`	Removes a header the client set by default.
`accept(mediaType: string): this`	
`contentType(mediaType: string): this`	
`bearer(token: string): this`	Sets `Authorization: Bearer …` for this request only.
`basic(username: string, password: string): this`	Sets HTTP Basic credentials for this request only.
`idempotencyKey(key = crypto.randomUUID()): this`	Attaches an idempotency key. Generated when omitted, so a retried POST cannot create two resources — the failure mode that makes teams give up on retrying writes altogether.
`traceId(id: string): this`	Adds a correlation id so a failing test can be found in server logs.
`json(body: unknown): this`	JSON body. Sets `Content-Type: application/json` unless already set.
`form(fields: Record<string, string | number | boolean>): this`	`application/x-www-form-urlencoded` body.
`multipart(parts: MultipartPart[]): this`	`multipart/form-data` body, built part by part.
`file(name: string, filePath: string, mimeType?: string): this`	Adds one file part, creating a multipart body if there is not one yet.
`fileFromBuffer(name: string, buffer: Buffer, fileName: string, mimeType = 'application/octet-stream'): this`	Adds one in-memory file part — for generated or oversized payloads.
`field(name: string, value: string): this`	Adds one plain field to a multipart body.
`text(body: string, mediaType = 'text/plain'): this`	Raw text body — XML, CSV, plain text, or a deliberately malformed payload.

`binary(body: Buffer, mediaType = 'application/octet-stream'): this`	Raw bytes — image uploads, protobuf, gzip payloads.
`timeout(milliseconds: number): this`	
`retries(count: number): this`	Overrides the retry count for this request. Zero disables retrying.
`expectStatus(...codes: number[]): this`	Statuses the client accepts without raising. Assertions in the test are still the place to *check* the status — this only decides whether the client itself treats the response as a transport-level failure.
`anonymous(): this`	Sends no credentials, so the unauthenticated path can be tested.
`noRedirect(): this`	Returns the 3xx itself instead of following it — for redirect assertions.
`as(label: string): this`	Names the request in step titles, logs and report attachments.
`meta(key: string, value: string | number | boolean): this`	Attaches metadata carried into the exchange log.
`build(): RequestSpec`	Resolves everything into an immutable spec without sending it.
`send(): Promise<ApiResponse<T>>`	Sends the request and resolves with the captured response.
`then(onfulfilled?: ((value: ApiResponse<T>) => TResult1 | PromiseLike<TResult1>) | null, onrejected?: ((reason: unknown) => TResult2 | PromiseLike<TResult2>) | null): PromiseLike<TResult1 | TResult2>`	Makes the builder awaitable, so `await api.get('/x')` sends the request.

Types (src/types)

src/types/index.ts

131 lines · 13 exports

The shared vocabulary. The builder, the client, the response, the auth providers and the reporters all describe a request the same way, so changing the shape of a request is one edit here plus the compiler listing everything that must keep up.

TYPE	ROLE
`HttpMethod`, `MUTATING_METHODS`	The verbs, and which of them the read-only guard blocks
`JsonValue`, `JsonObject`, `JsonArray`	A precise recursive JSON type — better than `any` for parsed payloads
`QueryValue`, `QueryParams`	Query values, including arrays
`HeaderMap`	Headers, compared case-insensitively throughout

TYPE	ROLE
`PathParams`	Values substituted into `/users/{id}`
`MultipartPart`	One part of a multipart upload: inline value, file path, or buffer
`BodyKind`	**A discriminated union.** Adding a member makes the compiler list every switch that must handle it.
`RequestSpec`	A fully resolved request — the boundary between the builder and the client
`RequestTiming`, `ExchangeRecord`	What the recorder, the latency collector and the reporters consume
`ValidationResult`	One shape for every validator, so assertions do not care which kind of schema ran
`Page<T>`	A normalised page of results, whatever convention the API uses
`Awaitable<T>`, `PartialBy`, `UnknownRecord`	Small utilities used across the framework

`UnknownRecord` deserves a mention: it is `Record<string, unknown>`, and it is what makes "no `any`" practical. A parsed payload is genuinely unknown, and saying so forces the narrowing that catches real bugs.

WHEN YOU NEED TO CHANGE IT

- A request or response gains a dimension.
- A shape is being described in two places.

HOW TO CHANGE IT

1. Add the field to `RequestSpec`, then follow the compiler. It will name the builder, the client and the log renderer.
2. Add a member to `BodyKind` and the exhaustive switches become errors until they handle it — that is the point of the union.

WHY THE CHANGE BELONGS HERE

Types are the cheapest coordination mechanism available. A shared vocabulary means a change is checked rather than searched for, and nothing is missed because nothing *can* be missed.

WATCH OUT FOR

- Most fields are `readonly`. A `RequestSpec` is meant to be built once and then treated as a fact.
- `HeaderMap` keys are lower-cased by convention, not by the type. Everything that produces one lower-cases it; anything new must too.

Related: `src/core/request.builder.ts` · `src/core/http.client.ts`

API SURFACE (13 EXPORTS)

TYPE **HttpMethod**

Every verb the client can issue.

```
'GET' | 'POST' | 'PUT' | 'PATCH' | 'DELETE' | 'HEAD' | 'OPTIONS'
```

CONST **MUTATING_METHODS**

Verbs that must not run against a read-only environment.

TYPE **QueryValue**

A single query-string value. Arrays are repeated as `?tag=a&tag=b`.

```
string | number | boolean | null | undefined | (string | number)[]
```

TYPE **QueryParams**

```
Record<string, QueryValue>
```

TYPE **HeaderMap**

Header names are compared case-insensitively throughout the framework.

```
Record<string, string>
```

TYPE **PathParams**

Values substituted into a templated path such as `/users/{id}`.

```
Record<string, string | number>
```

INTERFACE **MultipartPart**

One part of a `multipart/form-data` upload.

```
readonly name: string;
readonly value?: string;
readonly filePath?: string;
readonly buffer?: Buffer;
readonly fileName?: string;
readonly mimeType?: string;
```

TYPE **BodyKind**

How the request body is encoded on the wire.

```
'none' | 'json' | 'form' | 'multipart' | 'text' | 'binary'
```

INTERFACE **RequestSpec**

A fully resolved request, ready to be sent. Produced by the request builder.

```
readonly method: HttpMethod;
readonly url: string;
readonly headers: HeaderMap;
readonly bodyKind: BodyKind;
readonly json?: unknown;
readonly form?: Record<string, string | number | boolean>;
readonly multipart?: MultipartPart[];
readonly text?: string;
readonly binary?: Buffer;
readonly timeout: number;
readonly retries: number;
readonly expectStatus?: number[];
readonly label: string;
readonly anonymous: boolean;
readonly followRedirects: boolean;
readonly meta: Record<string, string | number | boolean>;
```

INTERFACE **RequestTiming**

Wall-clock timings captured for every request.

```
readonly startedAt: number;
readonly durationMs: number;
readonly attempts: number;
```

INTERFACE **ExchangeRecord**

One completed exchange, as written to the run log and the HTML report.

```
readonly method: HttpMethod;
readonly url: string;
readonly status: number;
readonly requestHeaders: HeaderMap;
readonly responseHeaders: HeaderMap;
readonly requestBody?: string;
readonly responseBody?: string;
readonly timing: RequestTiming;
readonly label: string;
```

INTERFACE **ValidationResult**

Outcome of a schema or contract check.

```
readonly valid: boolean;
readonly errors: string[];
readonly value?: unknown;
```

TYPE **UnknownRecord**

A plain object with unknown values — safer than `any` for parsed payloads.

```
Record<string, unknown>
```

Reference: authentication & contracts

Credentials, tokens, schemas and OpenAPI conformance

Authentication (src/auth)

src/auth/jwt.ts 64 lines · 1 exports

JWT inspection — decoding only. Reads the header, claims and (unverified) signature so a test can assert on scopes, expiry and subject.

> **RULE** Decode-only, deliberately. Verifying a signature needs the issuer's key and is the API's job, not the suite's. What a test legitimately needs is to read the claims it was given. **Never treat a decoded token as trusted.**

FUNCTION	RETURNS
`decodeJwt(token)`	Header, claims and signature segment

One function, because one is what the suite uses. Named accessors for claims, expiry and scopes existed here and were never called; they were removed rather than left as surface nobody exercises.

> **NOTE** `exp` is in **seconds** since the epoch, not milliseconds. That off-by-a-thousand is the classic JWT bug — convert at the point of use, and remember it when adding an expiry helper.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A test needs a claim the helpers do not expose.

> **HOW TO CHANGE IT**
>
> 1. The decoded `claims` object is an open record, so custom claims are already reachable.
>
> ```
> const tenant = decodeJwt(token).claims['https://example.com/tenant'];
> ```
>
> 2. Add a named helper only for a claim used repeatedly across the suite.

> **WHY THE CHANGE BELONGS HERE**
>
> Asserting that a token carries the right scopes and expires when it should is a real test, and it needs no cryptography. Keeping verification out keeps the boundary honest.

> **WATCH OUT FOR**
>
> - A leading `Bearer` is stripped, so a header value can be passed directly.
> - base64url differs from base64 in two characters and drops padding; `decodeSegment` handles both.

> - A malformed token throws `ConfigurationError` with a specific message rather than a generic parse error.

Related: `src/auth/oauth2.auth.ts` · `src/core/errors.ts`

API SURFACE (1 EXPORT)

`FUNCTION` **`decodeJwt(token: string): DecodedJwt`**

Splits and base64url-decodes a token. Throws when it is not a JWT.

src/auth/oauth2.auth.ts

213 lines · 1 exports

OAuth 2.0 token acquisition: client credentials, resource-owner password, and refresh. Tokens are cached in the shared store, so a suite authenticates once per role rather than once per test.

Why these three grants

- **Client credentials** — the suite acting as itself, for machine-to-machine APIs.
- **Password** — the suite acting as a named user, which is what an authorisation matrix needs.
- **Refresh** — extending a session without re-authenticating.

Authorisation-code flows need a browser and belong in the UI suite, which can perform the flow and hand the resulting token to `BearerAuth`.

Both entry points are static, so the grant is visible at the call site — TYPESCRIPT

```typescript
OAuth2Auth.clientCredentials(request, { scope: 'orders:read' });
OAuth2Auth.password(request, username, password, { store: tokens });
```

The option that saves the most time

clientAuth — TYPESCRIPT

```typescript
// Default: credentials in an Authorization: Basic header.
OAuth2Auth.clientCredentials(request);

// Some providers require them in the form body instead.
OAuth2Auth.clientCredentials(request, { clientAuth: 'body' });
```

Providers differ, and sending the wrong one produces `invalid_client` — which reads exactly like a wrong secret. If credentials are definitely correct and the token endpoint still refuses them, try the other form before anything else.

Caching

Keyed by token URL, grant, subject and scope, so two roles never share a token. Renewal happens 30 seconds before real expiry, because a token that expires mid-flight produces a 401 nobody can reproduce.

WHEN YOU NEED TO CHANGE IT

- The provider needs an extra parameter.
- A grant is needed that is not here.
- Errors need more context.

HOW TO CHANGE IT

1. Most providers only need an extra parameter, which needs no code change.

```
OAuth2Auth.clientCredentials(request, {
  extra: { audience: 'https://api.example.com', resource: 'orders' },
});
```

2. For a new grant, add a static factory and a branch in `fetchToken`. Keep the cache key discriminating enough that grants cannot collide.

WHY THE CHANGE BELONGS HERE

A token endpoint is usually rate-limited harder than the API itself, so a hundred parallel tests each fetching their own token start failing with 429s that look like product bugs. Caching is not an optimisation here; it is what makes a parallel suite viable.

WATCH OUT FOR

- A non-JSON response from the token endpoint is reported with an excerpt — usually an HTML error page from a proxy, not a credential problem.
- `invalidate()` drops the cached token, which is how a token-expiry test forces a refresh.
- The store is per worker. Eight workers fetch eight tokens, not one — pass a persisting store if that matters.

Related: `src/auth/token.store.ts` · `src/fixtures/api.fixture.ts` · `src/config/env.config.ts`

API SURFACE (1 EXPORT)

CLASS **OAuth2Auth** implements AuthProvider

`name: string`	
`clientCredentials(request: APIRequestContext, options: OAuth2Options = {}): OAuth2Auth`	Machine-to-machine: the suite itself is the client.
`password(request: APIRequestContext, username: string, password: string, options: OAuth2Options = {}): OAuth2Auth`	Resource-owner password grant: act as a named user.
`headers(): Promise<HeaderMap>`	
`invalidate(): void`	Forces the next request to fetch a new token.
`token(): Promise<CachedToken>`	The cached or freshly issued token, including its expiry.
`refresh(refreshToken: string): Promise<CachedToken>`	Exchanges a refresh token for a new access token.

src/auth/static.auth.ts

79 lines · 3 exports

The three credential schemes that need no network call: `NoAuth`, `BasicAuth` and `ApiKeyAuth`. Grouped in one file because each is a handful of lines and splitting them would cost more in imports than it saves in navigation. Bearer tokens are applied through `.bearer(token)` on the request builder rather than through a strategy class.

PROVIDER	SENDS	USE FOR
`NoAuth`	Nothing	Asserting the 401 path explicitly
`BasicAuth`	`Authorization: Basic …`	Simple username/password APIs
`BearerAuth`	`Authorization: Bearer …`	A token captured elsewhere — including by the UI suite
`ApiKeyAuth`	A header, or a query parameter	Key-based APIs

BearerAuth accepts a function, which is what makes it useful — TYPESCRIPT

```typescript
// A fixed token:
new BearerAuth(token);

// Or a source read at request time, so a refresh is picked up automatically:
new BearerAuth(() => readCapturedToken());
```

`ApiKeyAuth` supports the query-string form because some APIs require it — but a key in a query string ends up in server access logs and browser history, so the header form is the default and the better choice wherever the API allows it.

WHEN YOU NEED TO CHANGE IT

- An API uses a header name or prefix these do not cover.

HOW TO CHANGE IT

1. Most variation is already an option.

```
new ApiKeyAuth(key, { in: 'header', name: 'x-tenant-key', prefix: 'ApiKey ' }
```

2. For a genuinely different scheme, add a file implementing `AuthProvider` rather than adding a fifth mode here.

WHY THE CHANGE BELONGS HERE

These are the schemes with no state and no I/O. Keeping them together and keeping them tiny makes the point that a provider is a small thing — which is what encourages writing another one instead of special-casing a test.

WATCH OUT FOR

- `ApiKeyAuth.queryParams()` exists for the query form, but the client only injects headers. A query key has to be merged by the caller — which is another reason to prefer the header.
- A header set explicitly on a request always beats the provider.

Related: `src/core/http.client.ts` · `src/fixtures/api.fixture.ts`

API SURFACE (3 EXPORTS)

CLASS **NoAuth** `implements AuthProvider`

Sends no credentials. Use to assert the 401 path explicitly.

name

```
headers(): Promise<HeaderMap>
```

CLASS **BasicAuth** `implements AuthProvider`

HTTP Basic — `Authorization: Basic base64(user:pass)`.

name

```
headers(): Promise<HeaderMap>
```

CLASS **ApiKeyAuth** `implements AuthProvider`

API-key authentication. Query-string keys are supported because some APIs require them, but they end up in server access logs and browser history — the header form is preferred wherever the API allows it.

name	
`headers(): Promise<HeaderMap>`	
`queryParams(): Record<string, string>`	Query parameters to merge, for the `in: 'query'` form.

src/auth/token.store.ts

111 lines · 3 exports

Caches access tokens, with an expiry skew, and optionally persists them to a file so separate worker processes can share one.

The pattern every provider uses — TYPESCRIPT

```ts
async token(): Promise<CachedToken> {
  return this.store.getOrCreate(this.cacheKey(), () => this.fetchToken());
}
```

A token within 30 seconds of expiry is treated as already expired and discarded. That skew is the difference between a reliable suite and one that occasionally 401s for no discoverable reason.

CAREFUL A persisted token file is a **live credential**. It is written with mode `0600`, lives under `storage/`, and that whole directory is git-ignored. Global teardown clears the in-memory store so a token does not linger on a shared build agent.

WHEN YOU NEED TO CHANGE IT

- The skew is wrong for a provider issuing very short tokens.
- Workers should share one token.

HOW TO CHANGE IT

1. Adjust `EXPIRY_SKEW_MS` — raise it for short-lived tokens, never lower it below a few seconds.
2. To share across workers, construct the store with a path. The file is git-ignored.

```ts
new TokenStore(fromRoot('storage', `tokens-${config.env}.json`));
```

WHY THE CHANGE BELONGS HERE

Two costs justify a cache: token endpoints are rate-limited, and a token round trip on every request roughly doubles a suite's runtime. The skew exists because the alternative failure is unreproducible.

> **WATCH OUT FOR**
>
> - A corrupt cache file is ignored with a warning rather than failing the run. The worst case is one extra token request.
> - The exported `tokenStore` is per process. Workers do not share memory.

Related: `src/auth/oauth2.auth.ts` · `src/hooks/global.teardown.ts` · `.gitignore`

API SURFACE (3 EXPORTS)

INTERFACE **CachedToken**

A token with the moment it stops being usable.

```
readonly value: string;
readonly expiresAt: number;
readonly refreshToken?: string;
readonly type: string;
```

CLASS **TokenStore**

`get(key: string): CachedToken | undefined`	A cached token for a key, or `undefined` when absent or near expiry.
`set(key: string, token: CachedToken): void`	
`getOrCreate(key: string, produce: () => Promise<CachedToken>): Promise<CachedToken>`	Returns the cached token, or produces and caches a new one.
`invalidate(key?: string): void`	Drops one key, or everything when no key is given.
`expiryFromSeconds(seconds: number | undefined, fallbackSeconds = 3600): number`	Convenience for building `expiresAt` from an `expires_in` in seconds.

CONST **tokenStore**

Process-wide store. One per worker; workers do not share memory.

Contracts (src/contracts)

src/contracts/json-schema.ts — 77 lines · 1 exports

JSON Schema validation via Ajv, for schemas the team was *given* rather than wrote — an OpenAPI document, a partner's published contract, a schema from another repository.

Both validators report violations in the same `ValidationResult` shape, so an assertion does not care whether a Zod schema or a JSON Schema ran.

Why Ajv is configured this way TYPESCRIPT

```typescript
const ajv = new Ajv({ allErrors: true, strict: false, allowUnionTypes: true, verb
addFormats(ajv);
```

- `strict: false` — real-world OpenAPI documents carry vendor extensions and keywords Ajv does not know. Failing on those would reject valid specifications.
- `allErrors: true` — collecting every violation is the whole point of a contract check, rather than reading the first mismatch and re-running.
- `addFormats` — without it, `date-time`, `email` and `uri` are silently ignored, and a schema appears to pass when it never checked anything.

Error formatting

Ajv's raw output puts the path in `instancePath`, the reason in `message` and the useful specifics in `params`. `formatAjvErrors` joins them into a sentence.

Before and after TYPESCRIPT

```typescript
// Ajv: { instancePath: '/items/0/price', message: 'must be number', params: { ty
// Formatted:
items.0.price must be number (expected number)
```

> **WHEN YOU NEED TO CHANGE IT**
>
> - A keyword needs custom handling.
> - An error message could be more specific.

> **HOW TO CHANGE IT**
>
> 1. Add a case to `describeParams` for a keyword whose `params` carry something useful.
> 2. Register shared schemas by `$id` so others can `$ref` them.
>
> ```typescript
> addSchema(commonDefinitions, 'https://example.com/schemas/common.json');
> ```

> **WHY THE CHANGE BELONGS HERE**
>
> Zod covers what the team writes; JSON Schema covers what the team is given. Both matter, and a suite that can only do one of them cannot check conformance against a published contract.

> **WATCH OUT FOR**
>
> - Compiled validators are cached by object identity in a `WeakMap`. Building a fresh schema object on every call recompiles every time — hoist it.
> - `strict: false` means a typo in a keyword name is ignored rather than reported. Validate the schema itself if it is hand-written.

Related: `src/contracts/openapi.ts` · `src/fixtures/custom-matchers.ts`

API SURFACE (1 EXPORT)

FUNCTION

```
validateJsonSchema(schema: object, value: unknown): ValidationResult
```

Validates a value against a JSON Schema document.

src/contracts/openapi.ts

229 lines · 1 exports

Validates responses against an OpenAPI document: finds the operation for a method and path, resolves local `$ref`s, picks the schema for the status, and hands it to Ajv.

A hand-written schema records what the test author believed. The document records what the API *promised*. Only the second catches the case that hand-written schemas never do: the API and its published contract have drifted, and every consumer who trusted the document is broken.

METHOD	DOES
`fromFile` / `fromObject`	Loads a document from disk, or one fetched from the API itself
`list()`	Every documented operation
`find(method, url)`	The operation matching a real request
`validate(method, url, status, payload)`	A `ValidationResult`
`assert(...)`	The throwing form, raising `ContractViolationError`
`uncovered(calls)`	Documented operations the suite never exercised — the contract-coverage gap

Status resolution

It looks for the exact code, then the `4XX` band, then `default` — which is how a document describes "every other status", usually the error shape.

Deliberately small

Only in-document `$ref`s are followed. A specification that references another file should be bundled first — resolving across files here would mean inventing a resolver whose behaviour differs subtly from the team's own tooling, which is worse than not having one.

Bundle first — SHELL

```
npx @redocly/cli bundle openapi.yaml -o src/data/openapi.json
```

WHEN YOU NEED TO CHANGE IT

- The document uses a construct that is not handled.
- You want request-side validation too.

HOW TO CHANGE IT

1. Extend `collectResponses` for a media type other than JSON, or `collectOperations` for a construct such as `webhooks`.
2. For request validation, the parameters are already collected — add a `validateRequest` that checks them against `RequestSpec`.

WHY THE CHANGE BELONGS HERE

Contract conformance is the one check that catches a breaking change *before* a consumer does. Making it cheap — drop the document in `src/data/` and it is picked up automatically — is what makes it get used.

WATCH OUT FOR

- An external `$ref` throws with an explicit message telling you to bundle. That is intentional, not a limitation to work around.
- The `$ref` resolver has a depth limit of 50, so a circular reference terminates instead of hanging.
- An undocumented operation is reported as a validation failure. That is usually a real finding: the suite is calling something the document does not describe.

Related: `src/contracts/json-schema.ts` · `src/fixtures/api.fixture.ts` · `src/data/README.md`

API SURFACE (1 EXPORT)

CLASS OpenApiContract

`fromFile(specPath: string): OpenApiContract`	Loads a document from a `.json` file.
`fromObject(document: UnknownRecord): OpenApiContract`	Loads a document already parsed — for a spec fetched from the API itself.
`list(): readonly OperationDescriptor[]`	Every operation the document declares.
`find(method: HttpMethod, url: string): OperationDescriptor | undefined`	Finds the operation matching a real request.
`validate(method: HttpMethod, url: string, status: number, payload: unknown): ValidationResult`	Validates a payload against the documented schema for a status. Falls back to the `default` response, which is how a document describes "every other status" — usually the error shape.
`assert(method: HttpMethod, url: string, status: number, payload: unknown): void`	Validates, and throws a contract violation when the payload disagrees.
`uncovered(exercised: { method: HttpMethod; url: string }[]): OperationDescriptor[]`	Operations the suite never touched — the contract-coverage gap.

src/contracts/schema.registry.ts

97 lines · 3 exports

Maps method + path pattern + status to a schema. This is what lets the contract guard find the right schema from a real request, and what gives contract coverage a denominator.

Registration, at module scope beside the service — TYPESCRIPT

```
registerSchemas([
  { name: 'order', method: 'GET', pathPattern: '/orders/{id}', status: '2xx', sch
  { name: 'order-created', method: 'POST', pathPattern: '/orders', status: 201, s
  { name: 'order-error', method: 'POST', pathPattern: '/orders', status: '4xx', s
]);
```

Path matching

`/orders/{id}` matches `/orders/42` — otherwise every identifier would need its own registration. It also tolerates a leading version segment, so one registration works whether the environment's prefix is `/v1` or empty.

What matches what TYPESCRIPT

```typescript
matchesPath('/orders/{id}', '/orders/42')      // true
matchesPath('/orders/{id}', '/v1/orders/42')   // true — version prefix skipped
matchesPath('/orders/{id}', '/orders')         // false — different segment count
```

Status patterns

A status is an exact code or one of `2xx`, `4xx`, `5xx`. The band form is what makes a single error-shape registration cover every failure of an endpoint.

WHEN YOU NEED TO CHANGE IT

- You add an endpoint.
- The path-matching rules need to change.

HOW TO CHANGE IT

1. Register beside the schema, in the service file, so the two cannot drift apart.
2. Check registration took effect.

```
node -e "import('./src/contracts/schema.registry.ts')" # or assert findSchema
```

WHY THE CHANGE BELONGS HERE

Automatic validation needs a way to get from "a response arrived from `/v1/orders/42`" to "here is the schema that describes it". Without the registry, `STRICT_CONTRACTS` could not exist and every schema check would have to be written by hand in a test.

WATCH OUT FOR

- Registration happens on import. A service nobody imports registers nothing — which is fine, since nothing is calling it either.
- `findSchema` returns the **first** match, so a specific pattern should be registered before a general one.
- `clearSchemas()` exists for the framework's own tests. Calling it in a suite silently disables the guard.

Related: `src/fixtures/api.fixture.ts` · `src/contracts/schemas.ts` · `src/services/template.service.ts`

API SURFACE (3 EXPORTS)

FUNCTION `registerSchemas(entries: RegisteredSchema[]): void`

Registers many at once — the usual form for a service's contract file.

FUNCTION

findSchema(method: HttpMethod, url: string, status: number): RegisteredSchema | undefined

The schema for a request/status pair, or `undefined` when none is registered.

FUNCTION **matchesPath(pattern: string, pathname: string): boolean**

Compares a `{param}` pattern with a concrete path, ignoring any version prefix so one registration works across `/v1/users` and `/users`.

src/contracts/schemas.ts

37 lines · 3 exports

Reusable schema building blocks: identifiers, timestamps, money, pagination envelopes, and the standard error shapes.

Zod rather than raw JSON Schema for anything hand-written, because it produces a TypeScript type as a by-product: `response.parse(User)` returns a fully typed value, so the test body cannot drift from the contract.

EXPORT	DESCRIBES
`uuid`, `identifier`	An RFC 4122 id, and the union of string-or-number ids most APIs actually use
`isoDateTime`, `isoDate`	An instant with an offset, and a bare calendar date
`email`, `url`	Format-checked strings
`minorUnits`, `currency`	Money as an integer — never a float — and an ISO 4217 code
`timestamps`	The `createdAt`/`updatedAt` pair, spread into a resource schema
`offsetPage`, `cursorPage`, `dataEnvelope`	The three common wrapper shapes, as generic functions
`problemDetails`	RFC 9457 — the standard error body
`errorEnvelope`, `validationErrors`	Looser shapes for APIs that predate the standard
`healthCheck`	What a `/health` endpoint should return

Composing a resource schema TYPESCRIPT

```ts
export const OrderSchema = z.object({
  id: identifier,
  currency,
  totalMinor: minorUnits,
  ...timestamps,
});

export const OrderPageSchema = cursorPage(OrderSchema);
```

NOTE `problemDetails` is worth asserting even on a happy-path test. An API that returns a bare string on error is one your consumers cannot handle, and this is the cheapest place to catch that.

WHEN YOU NEED TO CHANGE IT

- A shape is being written out in more than one service.
- The API uses a convention not covered here.

HOW TO CHANGE IT

1. Add the primitive or the envelope factory. Annotate the return type of a generic factory explicitly — the inferred Zod type is unreadable and the lint rule requires it.

```ts
export function keysetPage<T extends z.ZodTypeAny>(
  item: T,
): z.ZodObject<{ items: z.ZodArray<T>; after: z.ZodOptional<z.ZodString> }> {
  return z.object({ items: z.array(item), after: z.string().optional() });
}
```

WHY THE CHANGE BELONGS HERE

Every API repeats the same handful of shapes. Defining them once means a change to "what a valid timestamp looks like" is one edit, and it keeps each service's own schema short enough to read.

WATCH OUT FOR

- Zod objects are non-strict by default: an unexpected extra field passes. Use `.strict()` when the test is specifically about the API not returning more than it promised.
- `z.string().datetime({ offset: true })` accepts an offset; without the option only `Z` is allowed.

Related: `src/contracts/schema.registry.ts` · `src/services/template.service.ts`

API SURFACE (3 EXPORTS)

`CONST` **identifier**

An identifier that may be a UUID or a numeric string — very common.

CONST **isoDateTime**

ISO 8601 instant, e.g. `2026-08-31T09:15:00Z`.

FUNCTION

```
offsetPage(item: T): z.ZodObject<{ items: z.ZodArray<T>; total: z.ZodNumber;
offset: z.ZodNumber; limit: z.ZodNumber; }>
```

Offset/limit pagination, the most common REST convention.

Reference: protocols, services & fixtures

GraphQL, streaming, sockets, service objects and how the framework reaches your tests

Protocol clients (src/protocols)

src/protocols/graphql.client.ts 227 lines · 1 exports

GraphQL client: queries, mutations, batching, introspection, and a response wrapper whose assertions understand GraphQL's failure model.

RULE GraphQL answers `200 OK` and puts the failure in an `errors` array in the body. A test that only checks the status will pass while the query returned nothing at all. `expectNoErrors()` exists to make that impossible to forget.

The shape of a GraphQL test — TYPESCRIPT

```ts
const result = await graphql.query<{ viewer: { id: string } }>(
  `query Viewer { viewer { id name } }`,
);

result.expectNoErrors().expectData();
expect(result.path('viewer.name')).toBe('Ada');
```

And the negative path — TYPESCRIPT

```ts
const denied = await graphql.mutate(`mutation Delete { deleteAccount { ok } }`);
denied.expectError(/not permitted/).expectErrorCode('FORBIDDEN');
```

Partial success

In GraphQL, `data` and `errors` can both be present — a field resolver failed while the rest succeeded. The wrapper exposes both rather than choosing for you, because which one matters depends on the test.

Batching and introspection

`batch()` sends several operations in one HTTP request, which is how a GraphQL API is usually rate-limited and load-tested — and some servers behave differently under it. `introspect()` fetches the schema, for detecting drift between deployments.

> WHEN YOU NEED TO CHANGE IT
>
> - The API uses persisted queries or another transport convention.

- Errors carry extensions worth asserting on directly.
- Subscriptions are needed.

HOW TO CHANGE IT

1. For persisted queries, send the hash instead of the document in `execute`.

```
.json({ extensions: { persistedQuery: { version: 1, sha256Hash: hash } }, var
```

2. For subscriptions, use `WebSocketClient` with the `graphql-transport-ws` sub-protocol; the framing is a socket concern, not a GraphQL one.

```
await socket(config.wsUrl, { protocols: ['graphql-transport-ws'] });
```

WHY THE CHANGE BELONGS HERE

GraphQL is REST-shaped on the wire and completely different in its error model. Giving it its own client is what stops REST-shaped assertions from silently passing on a failed query.

WATCH OUT FOR

- The operation name is extracted from the document for logs and step titles; an anonymous operation shows as `anonymous`.
- `batch()` returns one wrapper per operation, in order — a server that reorders them would break that assumption, and none do.
- A GraphQL endpoint that returns non-JSON produces an empty envelope rather than throwing; check `result.http.status` when nothing makes sense.

Related: `src/core/http.client.ts` · `src/protocols/websocket.client.ts`

API SURFACE (1 EXPORT)

CLASS **GraphQlClient**

`query(document: string, variables: UnknownRecord = {}, operationName?: string): Promise<GraphQlResponse<T>>`	Executes a query.
`mutate(document: string, variables: UnknownRecord = {}, operationName?: string): Promise<GraphQlResponse<T>>`	Executes a mutation. Identical to `query` on the wire; kept separate so the test reads as what it intends and so a read-only environment guard can tell them apart.
`batch(operations: { document: string; variables?: UnknownRecord; operationName?: string }[]): Promise<GraphQlResponse[]>`	Sends several operations in one HTTP request. Batching is how a GraphQL API is usually rate-limited and load-tested, and some servers behave differently under it — worth being able to exercise.
`introspect(): Promise<UnknownRecord | null | undefined>`	Fetches the schema via introspection — used for schema-drift checks.

src/protocols/sse.client.ts

213 lines · 2 exports

Server-Sent Events and NDJSON streaming. Implements the `text/event-stream` framing rules and exposes a wait-for-event API.

> **NOTE** This is the one place the framework does not use Playwright's request context. `APIRequestContext` buffers the whole response before returning it, which is exactly wrong for a stream that stays open — the request would simply never resolve. Node's global `fetch` exposes the body as a readable stream, so these two clients use it. That is a deliberate, documented exception.

The framing rules it implements

- Fields are `field: value`; an event is terminated by a blank line.
- Repeated `data:` lines accumulate, joined by newlines.
- A line starting with `:` is a comment — servers send these as keep-alives, and treating one as an event would confuse every waiter.
- `\r\n` is tolerated, because proxies rewrite line endings.

Using it — TYPESCRIPT

```ts
const stream = await sse(`${config.baseUrl}/v1/events`);

const started = await stream.waitForEvent('job.started');
const finished = await stream.waitForEvent('job.finished');

expect(SseClient.json<{ id: string }>(finished).id).toBe(jobId);
```

Events received before a wait begins are searched first, so a test that sends a request and then starts listening does not miss the event that arrived in between.

NDJSON

`readNdjsonStream` covers the other streaming convention — log tails, bulk exports, token streams — with a signature that mirrors the SSE client so switching between them costs nothing.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The server uses non-standard framing.
> - Reconnection with `Last-Event-ID` needs to be automatic.

> **HOW TO CHANGE IT**
>
> 1. Framing lives entirely in `parseFrame`; nothing else needs to change.
> 2. For reconnection, catch the read loop ending and re-connect with the last id — the option already exists.

```
    new SseClient(url, { lastEventId: stream.events.at(-1)?.id });
```

> **WHY THE CHANGE BELONGS HERE**
>
> A stream test that misses an event is a flaky test, and the miss is almost always a race between connecting and listening. Buffering from the moment the stream opens removes the race entirely.

> **WATCH OUT FOR**
>
> - **Always close the stream.** An open stream keeps the Node event loop alive and the worker never exits. The `sse` fixture closes everything on teardown, which is why it exists.
> - If events never arrive, a proxy is probably buffering. Confirm with `curl -N`; the framework cannot fix a proxy.
> - The idle timeout is per-event, not per-stream: a stream that sends one event a minute is fine.

Related: `src/fixtures/api.fixture.ts` · `src/config/timeouts.ts`

API SURFACE (2 EXPORTS)

INTERFACE SseOptions

```
readonly headers?: HeaderMap;
readonly idleTimeout?: number;
readonly lastEventId?: string;
```

CLASS SseClient

`get events: readonly ServerSentEvent[]`	Every event received so far.
`connect(): Promise<void>`	Opens the stream. Resolves as soon as the response headers arrive.
`waitFor(predicate: (event: ServerSentEvent) => boolean, description = 'a matching event', timeout = this.options.idleTimeout ?? TIMEOUTS.STREAM_IDLE): Promise<ServerSentEvent>`	Waits for the next event matching a predicate.
`waitForEvent(name: string, timeout?: number): Promise<ServerSentEvent>`	Waits for an event by name.
`collect(count: number, timeout?: number): Promise<ServerSentEvent[]>`	Collects `count` events, then resolves.
`json(event: ServerSentEvent): T`	Parses an event's `data` as JSON.
`close(): void`	Closes the stream. Always call this — an open stream keeps a test alive.

src/protocols/websocket.client.ts

208 lines · 2 exports

WebSocket client built on Node's global `WebSocket`, with every received frame buffered from the moment it connects.

> **RULE** The buffering is the design point. A test that sends a request and *then* starts listening will otherwise miss a reply that arrived first — the single most common source of flaky WebSocket tests. `waitFor` searches the buffer before it waits, so ordering does not matter.

Request and correlated reply, in one call — TYPESCRIPT

```ts
const socket = await openSocket();
const reply = await socket.request<{ id: number; status: string }>(
  { type: 'subscribe', id: 7 },
  (message) => message.id === 7,
);
expect(reply.status).toBe('ok');
```

METHOD	DOES
`connect(timeout)`	Opens and resolves on handshake; sends `onOpenSend` if configured
`send(payload)`	Objects are JSON-encoded, strings go as-is
`waitFor(predicate)`	Searches the buffer, then waits
`waitForJson(matches)`	The same, parsing each frame, returning it parsed
`request(payload, matches)`	Send and await the correlated reply
`close(code, reason)`	Closes, with a 2s escape hatch for a server that ignores the frame

No dependency: `WebSocket` has been global in Node since 22, which is what `.nvmrc` pins. A library here would be a second implementation to keep current.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A sub-protocol needs negotiating.
> - Reconnection should be automatic.
> - Binary frames are needed.

> **HOW TO CHANGE IT**
>
> 1 Sub-protocols are an option.
>
> ```ts
> new WebSocketClient(url, { protocols: ['graphql-transport-ws'], onOpenSend: {
> ```

2 For binary, widen `SocketMessage.data` and stop coercing with `String(event.data)` in the message listener.

WHY THE CHANGE BELONGS HERE

Socket tests fail for two reasons: a missed message and a leaked connection. The buffer fixes the first; the `socket` fixture closing everything on teardown fixes the second.

WATCH OUT FOR

- Node 22 or newer is required for the global `WebSocket`. Older Node fails at run time, not at install.
- `close()` resolves after two seconds even if the server never acknowledges, so a misbehaving server cannot hang the run.
- `closure` records the close code and reason — useful when a server closes for a policy reason rather than replying.

Related: `src/fixtures/api.fixture.ts` · `.nvmrc` · `src/config/timeouts.ts`

API SURFACE (2 EXPORTS)

INTERFACE **WebSocketOptions**

```
readonly protocols?: string[];
readonly messageTimeout?: number;
readonly onOpenSend?: unknown;
```

CLASS **WebSocketClient**

`get messages: readonly SocketMessage[]`	Every message received so far, oldest first.
`get closure: { code: number; reason: string } | undefined`	How the socket closed, once it has.
`get isOpen: boolean`	
`connect(timeout = TIMEOUTS.SOCKET_MESSAGE): Promise<void>`	Opens the socket and resolves once the handshake completes.
`send(payload: unknown): void`	Sends a frame. Objects are JSON-encoded; strings go as-is.
`waitFor(predicate: (message: SocketMessage) => boolean, description = 'a matching message', timeout = this.options.messageTimeout ?? TIMEOUTS.SOCKET_MESSAGE): Promise<SocketMessage>`	Waits for a message matching a predicate, searching already-received messages first so a reply that arrived early is not missed.
`waitForJson(matches: (value: T) => boolean, description = 'a matching JSON message', timeout?: number): Promise<T>`	Waits for a JSON message whose fields match, and returns it parsed.
`request(payload: unknown, matches: (value: T) => boolean, timeout?: number): Promise<T>`	Sends a frame and waits for the correlated reply.
`close(code = 1000, reason = 'test complete'): Promise<void>`	Closes the socket. Safe to call more than once.

Service objects (src/services)

src/services/object.service.ts 371 lines · 5 exports

The service object for **https://api.restful-api.dev/objects** — the live CRUD resource the lifecycle suite runs against. It is the framework's worked example against a real API.

What the real API actually does

Everything in this file was established by **calling the API**, not by reading its documentation, because the two disagree in several places. Each quirk is encoded in a schema and asserted by a test, so if the API changes the suite says so.

OPERATION	REALITY	WHAT A NAIVE SCHEMA WOULD GET WRONG
`POST /objects`	**200**, not 201, with `{ id, name, createdAt, data }`	Expecting 201 — the client would raise on every create
`GET /objects/{id}`	`{ id, name, data }` — **no timestamps at all**	A single Object schema requiring `createdAt` fails on every read
`GET /objects`	Only the 13 seeded objects; created ones are never listed	Asserting a created object appears in the list
`GET /objects?id=1&id=2`	Repeated parameters filter; a comma-joined list is **ignored**	A filter that silently returns everything
`PUT` / `PATCH`	200 with `updatedAt` and no `createdAt`	Reusing the create schema
`DELETE`	**200 with a message body**, not 204; a second delete is 404	Expecting 204 and an empty body
Errors	`{ error: string }` — **not** RFC 9457 problem details	Asserting the standard shape
Malformed JSON	400 `{ error: "Invalid request body" }`	
Validation	**None.** An empty body is accepted and stored with `name: null`	Assuming `name` is always a string

Why three schemas, not one

The timestamps are what force the split — TYPESCRIPT

```typescript
const objectBase = z.object({ id: z.string().min(1), name: z.string().nullable(),

export const ApiObjectSchema = objectBase;                                    //
export const CreatedObjectSchema = objectBase.extend({ createdAt: z.number() });
export const UpdatedObjectSchema = objectBase.extend({ updatedAt: z.number() });
```

`.passthrough()` matters as much as the split. A strict schema turns "the API added a field" — which breaks nobody — into a suite-wide failure, and teams respond to that by deleting the schema check entirely.

The quota guard

A refusal is not a failure — TYPESCRIPT

```typescript
static isQuotaExhausted(response: ApiResponse<unknown>): boolean {
  return response.status === 405 && /daily request limit/i.test(response.text());
}
```

The API allows 50 anonymous requests per 24 hours. A test that fails because somebody else used the day's quota tells nobody anything, so the live suite detects the refusal and skips with a reason.

Two design points worth copying

Creation and cleanup are one expression — TYPESCRIPT

```typescript
const created = response.parse(CreatedObjectSchema, 'object-created');
return this.track(created, `object ${created.id}`, () => this.remove(created.id))
```

And delete tolerates "already gone" — TYPESCRIPT

```typescript
await this.del('/{id}').param('id', id).expectStatus(200, 404).send();
this.cleanup.forget(`object ${id}`);
```

Cleanup runs after tests that may have deleted the object themselves. If a second delete failed, every such test would go red in teardown for no reason.

WHEN YOU NEED TO CHANGE IT

- The API changes — the schemas and the table above are where that is recorded.
- You point the framework at your own API: this is the file to copy.
- A new operation is needed.

HOW TO CHANGE IT

1. Change a schema and the registration together; they sit side by side for that reason.
2. When adapting to your own API, keep the shape: schema first, type inferred, registration, then intention-revealing methods that never assert.
3. Run the live suite within quota after any change, so the stub and the API are checked against each other.

WHY THE CHANGE BELONGS HERE

This file is where all knowledge of one real API lives. When the endpoint moves or its payload changes, exactly one file changes — not every test that happened to mention objects.

WATCH OUT FOR

- `.arrays('repeat')` on the list call is load-bearing. The comma form produces a request the API answers 200 to and filters nothing.
- The idempotency key on create is what makes the client willing to retry a POST at all.
- Schemas register on import, so a service nobody imports registers nothing.

Related: `tests/api/objects.crud.spec.ts` · `src/mocks/objects.stub.ts` · `src/contracts/schema.registry.ts`

API SURFACE (5 EXPORTS)

CONST **ApiObjectSchema**

A read: `GET /objects/{id}`. Carries no timestamps.

CONST **CreatedObjectSchema**

A create response: the only one that carries `createdAt`.

CONST **DeleteAcknowledgementSchema**

The delete acknowledgement. Note it is a body, not a 204.

CONST **ObjectErrorSchema**

The API's error shape. Not RFC 9457. Recorded faithfully so tests assert what is really returned; see `tests/api/objects.negative.spec.ts` for the assertion that this is the shape, and the documentation for why we do not pretend otherwise.

CLASS **ObjectService** extends BaseService

`isQuotaExhausted(response: ApiResponse): boolean`	The live API allows **50 anonymous requests per 24 hours**, then answers `405` with an explanatory body until the window resets. That is not a failure of the API or of the suite, so it must not be reported as one: a test that fails because somebody else already used the day's quota tells nobody anything. Detecting it lets the live suite skip with a clear reason and hand the coverage to the stubbed suite.
`quotaAvailable(): Promise<boolean>`	One cheap request that answers "is there quota left?". Costs a request itself, so callers should cache the answer for the whole worker rather than asking per test.
`create(input: NewObject): Promise<CreatedObject>`	Creates an object and registers its deletion. Three things are happening in the chain below, and each is deliberate. `.expectStatus(200)` — the API answers 200, not the 201 a REST purist would expect. Encoding the *real* status here means the client raises immediately if the API ever starts answering something else, instead of the failure surfacing as a confusing schema mismatch later. `.idempotencyKey()` — generates a key so this POST is safe to retry. The HTTP client only retries a non-idempotent verb when it sees this header; without it a retried create could produce two objects. `.as('create object')` — names the call in the HTML report, the log line and the recording, so a failure is identifiable without reading the URL.
`find(id: string): Promise<ApiObject | undefined>`	Reads one object, or `undefined` when the API answers 404.
`require(id: string): Promise<ApiObject>`	Reads one object, failing with a clear message when it is absent.
`list(ids: string[] = []): Promise<ApiObject[]>`	Lists objects. `.arrays('repeat')` produces `?id=1&id=2` rather than `?id=1,2`. This API requires the repeated form; the comma form silently returns everything, which is the worst kind of wrong — a passing test that filtered nothing.
`replace(id: string, input: NewObject): Promise<UpdatedObject>`	Replaces an object wholesale (PUT). The response carries `updatedAt` and not `createdAt`, which is why this validates against a different schema from `create`.
`update(id: string, changes: Partial<NewObject>): Promise<UpdatedObject>`	Applies a partial update (PATCH).
`remove(id: string): Promise<void>`	Deletes an object, tolerating "already gone". 404 is accepted because this runs from the cleanup registry, after tests that may have deleted the object themselves. If a second delete failed, every such test would fail in teardown for no reason. `cleanup.forget` then removes the registration, so teardown does not try again for a resource this call already dealt with.
`rawCreate(payload: unknown): Promise<ApiResponse>`	The raw create response, for tests that assert on status, headers or timing rather than on the resulting object. Every service should expose one of these. Without it, a test that needs to check a header ends up bypassing the service and hard-coding the path the service exists to own. Note it does

	not register cleanup: the caller may deliberately be sending something that will not create anything.
`rawFind(id: string): Promise<ApiResponse>`	The raw read response — used by the negative and contract suites.

src/services/post.service.ts

138 lines · 3 exports

The service object for **https://jsonplaceholder.typicode.com/posts** — reads and real `Link` header pagination over a stable 100-record dataset.

It exists to exercise the two things the CRUD target cannot demonstrate: genuine RFC 8288 pagination, and a dataset large and stable enough that exact counts can be asserted without flakiness.

Reached through a derived client

A different host, in the fixture — TYPESCRIPT

```typescript
posts: new PostService(http.withBaseUrl(PUBLIC_APIS.jsonPlaceholder), {
  cleanup,
  logger: log.child('posts'),
}),
```

`withBaseUrl` derives rather than constructs, which keeps the run's observers — latency collection, exchange recording, the contract guard — attached. A freshly built `HttpClient` would silently lose all three.

The pagination walk

The server hands back the next URL; the walker just follows it — TYPESCRIPT

```typescript
return followLinkHeader<Post>(
  first,
  (url) => this.http.get(url).expectStatus(200).as('next page').send(),
  (response) => response.parse(PostListSchema, 'post-list'),
);
```

This is the one pagination style where the client needs no knowledge of the API's own parameter names. `followLinkHeader` stops when no `rel="next"` arrives and has a hard page ceiling, so a server that always advertises another page produces a clear failure rather than a hang.

NOTE The write method is called `simulateCreate`, not `create`. This API answers 201 with an echoed body and persists nothing; a method named `create` that does not create is a trap for the next person.

WHEN YOU NEED TO CHANGE IT

- Your API paginates differently.
- The dataset size changes.

HOW TO CHANGE IT

1. Swap the walker: `followCursor` for opaque cursors, `followOffset` for offset/limit. All three take the same shape of callback.
2. Keep the total in a constant like `TOTAL_POSTS` so a change is one edit.

WHY THE CHANGE BELONGS HERE

One API rarely demonstrates everything. Having a second service on a second host, reached through a derived client, is also the pattern any real project needs the moment it talks to more than one service.

WATCH OUT FOR

- The `Link` URLs are absolute and come from the server, so they pass through `resolveUrl` untouched — which is what makes the walker host-agnostic.

Related: `tests/api/posts.pagination.spec.ts` · `src/utils/pagination.utils.ts` · `src/config/environments.ts`

API SURFACE (3 EXPORTS)

`CONST` **PostSchema**

`CONST` **PostListSchema**

CLASS **PostService** extends BaseService

`TOTAL_POSTS`	The total number of posts the dataset contains. Stable, and asserted on.
`find(id: number): Promise<Post | undefined>`	Reads one post, or `undefined` for 404.
`all(): Promise<Post[]>`	Every post, unpaginated — the dataset is small enough that this is fine.
`page(pageNumber: number, limit: number): Promise<ApiResponse>`	One page, using the API's `_page`/`_limit` convention.
`walkAllPages(limit = 10): Promise<Post[]>`	Walks every page by following the `Link` header to the end. This is the pagination style that needs no knowledge of the API's own parameter names: the server hands back the next page's full URL, and the walker just follows it. `followLinkHeader` stops when no `rel="next"` arrives, and has a hard page ceiling so a server bug cannot loop forever.
`simulateCreate(input: { title: string; body: string; userId: number }): Promise<Post>`	Posts a new record and returns the response. Named `simulateCreate` rather than `create` on purpose: this API answers 201 with an echoed body but persists nothing, and a method called `create` that does not create is a trap for the next person.
`rawPage(pageNumber: number, limit: number): Promise<ApiResponse>`	The raw list response, for tests asserting on pagination headers.

src/services/template.service.ts

153 lines · 1 exports

A worked example of a service object — copy it to start a new one. Deliberately complete rather than minimal, because every question that comes up when writing the *second* service object is answered in it.

What it demonstrates

PATTERN	WHERE
Schema first, type inferred from it	`UserSchema` / `type User = z.infer<…>`
Registering schemas for the contract guard	`registerSchemas([...])`
Creation that registers its own cleanup	`create()`
A read that returns `undefined` for 404	`find()` — with `require()` as the throwing form
A list that follows pagination to the end	`list()`
A delete that tolerates "already gone"	`remove()` — accepts 204, 200 **and** 404

PATTERN	WHERE
An escape hatch for tests that need the response	`rawCreate()`

Schema first — so the runtime check and the compile-time type cannot disagree — TYPESCRIPT

```typescript
export const UserSchema = z.object({
  id: identifier,
  email: z.string().email(),
  role: z.enum(['admin', 'standard', 'readonly']),
  createdAt: isoDateTime,
});

export type User = z.infer<typeof UserSchema>;
```

Creation and cleanup in one expression — TYPESCRIPT

```typescript
const user = response.parse(UserSchema, 'user');
return this.track(user, `user ${String(user.id)}`, () => this.remove(user.id));
```

NOTE `remove()` accepts 404 because cleanup runs after tests that may already have deleted the resource themselves. "Already gone" has to count as success, or every such test fails in teardown.

WHEN YOU NEED TO CHANGE IT

- You are writing a new service — copy this.
- A pattern here proves wrong and should be corrected everywhere.

HOW TO CHANGE IT

1. Copy, rename, change `basePath`, and replace the schema with the real resource.
2. Add it to the `api` fixture in `src/fixtures/api.fixture.ts`.
3. Register the schemas so the contract guard can find them.

WHY THE CHANGE BELONGS HERE

The second service object is where a codebase's conventions are actually set. Making the first one exemplary — with its reasoning in comments — is cheaper than reviewing the next ten.

WATCH OUT FOR

- `remove()` calls `cleanup.forget()` so a resource deleted by the test is not deleted again in teardown.
- Schemas register on import. A service nobody imports registers nothing.

> - Service methods never assert. If you find yourself wanting one to, the assertion belongs in the test.

Related: `src/core/base.service.ts` · `src/contracts/schema.registry.ts` · `src/utils/cleanup.registry.ts`

API SURFACE (1 EXPORT)

CLASS **UserService** extends BaseService

`create(overrides: NewUser = {}): Promise<User>`	Creates a user and registers its deletion. The cleanup is registered inside the method rather than left to the test, so a resource cannot be created without also being cleaned up.
`find(id: User['id']): Promise<User | undefined>`	Reads one user. Returns `undefined` for 404 rather than throwing.
`require(id: User['id']): Promise<User>`	Reads one user, failing when it is absent.
`list(filter: { role?: User['role']; active?: boolean } = {}): Promise<User[]>`	Every user matching a filter, following pagination to the end.
`update(id: User['id'], changes: Partial<NewUser>): Promise<User>`	Applies a partial update.
`remove(id: User['id']): Promise<void>`	Deletes a user, tolerating a 404. Cleanup runs after tests that may already have deleted the resource themselves, so "already gone" has to count as success.
`rawCreate(payload: UnknownRecord): Promise<ApiResponse>`	The raw response, for tests that assert on status, headers or timing. Every service should expose one of these. Without it, a test that needs to check a `Location` header ends up bypassing the service entirely and hard-coding the path it was supposed to encapsulate.

Fixtures (src/fixtures)

src/fixtures/api.fixture.ts — 373 lines · 1 exports

The fixtures every test imports. A test opens with the things it needs and nothing else; everything behind that — building an authenticated client, seeding deterministic data, collecting latency, recording exchanges, deleting what the test created — is set up and torn down here.

Worker-scoped

FIXTURE	IS
`tokens`	The token cache shared by every test in the worker

FIXTURE	IS
`workerRequest`	A request context outliving individual tests, for worker-level setup
`mockServer`	The stub server. Started on port 0 so parallel workers never collide — and only started if a test asks for it.
`contract`	The OpenAPI document, if `src/data/openapi.json` exists

Test-scoped

FIXTURE	IS
`data`	Faker, seeded from the test's own title path
`log`	A logger prefixed with the test's name
`cleanup`	The registry, drained after the test — including when it failed
`auth`	The credential source chosen from what the environment supplies
`http`	The authenticated client, with latency and recording attached
`anonymousHttp`	An unauthenticated client, for the 401 path
`httpAs(role)`	A client authenticated as a named role, memoised per role
`api`	Every service object, sharing one cleanup registry
`graphql`	The GraphQL client
`latency`	Per-route timings, attached to the test as `latency.json`
`recorder`	Every exchange; saved to `reports/recordings/` **only when the test fails**
`sse` / `socket`	Factories that open a stream or socket and close it on teardown
`contractGuard`	Automatic schema validation. `auto: true`, so it applies without being named.

Deterministic data

Seeded from the test itself — TYPESCRIPT

```typescript
data: async ({}, use, testInfo) => {
  await use(seededFaker(testInfo.titlePath.join(' > ')));
},
```

The same test always gets the same data across runs and machines, while two different tests never collide. Random enough to catch a uniqueness bug, reproducible enough to debug one.

The contract guard

auto: true — because contract drift is found by the tests nobody wrote TYPESCRIPT

```
contractGuard: [
  async ({ http, log }, use): Promise<void> => {
    if (config.strictContracts) {
      http.onResponse((response) => { /* validate against the registered schema *
    }
    await use(undefined);
  },
  { auto: true },
],
```

Choosing credentials

`defaultAuthProvider` picks in order: OAuth2 if a token endpoint is configured, then an API key, then Basic credentials, then nothing. That ordering is what makes moving an environment from API keys to OAuth a `.env` change.

WHEN YOU NEED TO CHANGE IT

- Tests repeatedly build the same thing themselves.
- You add a service, or a client that holds a connection.
- Something must happen for every test.

HOW TO CHANGE IT

1. Declare the type in `ApiFixtures` or `WorkerFixtures`, then implement it. `use` is the boundary between setup and teardown.

```
tenant: async ({ http }, use) => {
  const created = await http.post('/tenants').json({ name: uniqueId('t') }).s
  await use(created.path<string>('id'));
  await http.delete(`/tenants/${created.path<string>('id')}`).send();
},
```

2. Choose scope on one question: can a test change it? If yes, test scope.
3. Use `auto: true` only for something that must apply universally — it costs every test, named or not.

WHY THE CHANGE BELONGS HERE

A fixture is set up and torn down by the framework, so it runs on failure too. Anything a test would otherwise have to remember to clean up belongs here, because "remember to" is not a mechanism.

WATCH OUT FOR

- A fixture cycle deadlocks with an unhelpful error. Split the shared part into a third fixture rather than trying to break the cycle in place.

> - Playwright reads the parameter destructuring to resolve dependencies, so a fixture with none must be written `async ({}, use)` — which is why `no-empty-pattern` is disabled for this folder.
> - Worker-scoped mutable state makes failures depend on worker count. Both of the worker fixtures here are effectively immutable.
> - The recorder attaches only on failure; a passing test's recording is noise.

Related: `src/fixtures/index.ts` · `src/mocks/mock.server.ts`

API SURFACE (1 EXPORT)

`CONST` **test**

src/fixtures/custom-matchers.ts

179 lines · 1 exports

Nine custom `expect` matchers for API responses, registered through `expect.extend` — which returns a *typed* `expect`, so no separate type declaration is needed.

MATCHER	ASSERTS
`toHaveStatus(code | codes)`	An exact status, or one of several
`toBeSuccessful()`	Any 2xx
`toHaveHeader(name, value?)`	A header exists, optionally matching a string or pattern
`toMatchSchema(schema, name?)`	A Zod schema, reporting every violation
`toMatchJsonSchema(schema)`	A JSON Schema document
`toSatisfyContract(contract)`	The published OpenAPI document
`toHaveJsonPath(path, value?)`	A value at a JSON path
`toRespondWithin(ms)`	A latency bound
`toMatchPayload(expected, options?)`	Structural equality, failing with a **diff** rather than two pretty-printed objects

Why both these and the response methods

The response wrapper already carries assertions. These exist for the cases where `expect(...)` reads better — negated assertions, `expect.soft`, and anything that is not a response, such as `toMatchPayload`.

Every matcher's failure message appends the same `context(response)` block — the request, the status, the timing and a body excerpt — so a matcher failure and a response-method failure carry the same information.

The typing detail that matters TYPESCRIPT

```typescript
// expect.extend returns a NEW expect carrying the matchers' types, which is why
// the result is captured rather than discarded. Mutating the global expect would
// register them at run time and leave TypeScript unaware of them.
export const expect = baseExpect.extend({ /* … */ });
```

WHEN YOU NEED TO CHANGE IT

- An assertion should be negatable or soft.
- An assertion applies to something that is not a response.

HOW TO CHANGE IT

1. Add a method to the object, returning `MatcherResult`. The type flows to every call site automatically.

```typescript
toHaveCursor(response: ApiResponse, expected?: string): MatcherResult {
  const cursor = response.path<string>('nextCursor');
  const pass = expected === undefined ? cursor !== undefined : cursor === exp
  return { pass, message: () => `…${context(response)}` };
}
```

2. Write the message for both directions — a negated matcher shows the `pass: true` branch.

WHY THE CHANGE BELONGS HERE

A matcher's message is read at the worst possible moment. Including the request that produced the response turns "expected 200, got 500" into something somebody can act on without re-running anything.

WATCH OUT FOR

- Import `expect` from `@fixtures`, never from `@playwright/test` — the Playwright one has none of these.
- The `message` closure is lazy, so the body excerpt is only rendered when the assertion actually fails.

Related: `src/core/api.response.ts` · `src/fixtures/index.ts` · `src/utils/diff.utils.ts`

API SURFACE (1 EXPORT)

`CONST` **expect**

`expect.extend` returns a *new* `expect` carrying the matchers' types, which is why the result is captured rather than discarded. Mutating the global `expect` would register the matchers at runtime but leave TypeScript unaware of them, and every call site would fail to compile.

src/fixtures/index.ts

13 lines · 2 exports

The single import for every test. `import { test, expect } from '@fixtures'` — and nothing else.

RULE Importing `test` or `expect` from `@playwright/test` gives you a `test` with no cleanup registry, no contract guard and none of the custom matchers — and nothing will tell you. This is the single most expensive mistake available in the repository.

What every spec starts with — TYPESCRIPT

```typescript
import { test, expect } from '../../src/fixtures';
```

WHEN YOU NEED TO CHANGE IT

- You add a fixture or a matcher that tests need.

HOW TO CHANGE IT

1. Re-export it. If it is a type, use `export type`.

WHY THE CHANGE BELONGS HERE

One import path makes the rule enforceable — a lint rule can ban the Playwright import outright, which is much easier than reviewing for it.

WATCH OUT FOR

- Adding an ESLint `no-restricted-imports` rule for `@playwright/test` inside `tests/**` is worth doing the first time somebody gets this wrong.

Related: `src/fixtures/api.fixture.ts` · `src/fixtures/custom-matchers.ts` · `tests/README.md`

API SURFACE (2 EXPORTS)

RE-EXPORT	**test** from `'./api.fixture'`
RE-EXPORT	**expect** from `'./custom-matchers'`

Hooks (src/hooks)

src/hooks/auth.setup.ts

74 lines

Captures an authenticated session before the suite runs. **The only file in the framework that knows anything about a specific API's login endpoint** — so when the login

contract changes, exactly one file changes.

It runs as a Playwright *project* rather than inside `globalSetup`, which buys three things: it appears in the report as a real step, it retries like any other test, and every other project can declare `dependencies: ['setup']` so nothing starts until credentials exist.

The part you adapt is fenced — TYPESCRIPT

```typescript
/* ---- Adapt from here down to the API under test. ------------------ */
const response = await request.post(`${config.baseUrl}${config.apiPrefix}/auth/lo
  data: { username: credentials.username, password: credentials.password },
  failOnStatusCode: false,
});
/* ---- Adapt to here. ----------------------------------------------- */
```

It skips rather than fails when no credentials are configured, so a fork's pull request or an offline contract run is not blocked by a missing secret.

NOTE Only useful for APIs that authenticate by session or long-lived token. When the API uses OAuth client credentials, delete this file and the `setup` project — the token fixture covers that case with no setup step.

WHEN YOU NEED TO CHANGE IT

- The login endpoint, its payload or its response shape changes.
- More than one role needs a captured session.

HOW TO CHANGE IT

1. Change what is inside the fence. Everything outside it — the skip, the file write, the permissions — stays.
2. For several roles, loop and write each under its own key.

```typescript
for (const role of ['admin', 'standard'] as const) {
  if (!hasUser(role)) continue;
  tokens[role] = await login(getUser(role));
}
```

WHY THE CHANGE BELONGS HERE

Every framework ends up with one file that knows the application. Making that explicit — one file, fenced, documented — is what stops application knowledge leaking into ten others.

WATCH OUT FOR

- The written file is a live credential: mode `0600`, under git-ignored `storage/`.
- Delete `storage/*.json` after a password change, or the old session keeps being replayed.
- The assertion message names this file, so somebody hitting a login failure is told where to look.

Related: `src/auth/token.store.ts` · `src/config/env.config.ts` · `playwright.config.ts` · `.gitignore`

src/hooks/global.setup.ts

97 lines · 1 exports

Runs once before any test. Its job is to fail fast and loudly when the run cannot possibly succeed.

A suite that starts against an unreachable environment produces hundreds of confusing connection errors. This turns that into one sentence naming the URL it could not reach.

The reachability check is deliberately tolerant — TYPESCRIPT

```typescript
const response = await context.get(config.baseUrl, { failOnStatusCode: false });
log.info('target reachable', { url: config.baseUrl, status: response.status() });
```

A 401 or a 404 still proves the host is up and reachable, which is the only thing this check is for. **Only a transport failure stops the run** — anything stricter would fail on APIs with no root route.

It also creates the output directories up front, so a reporter writing its first file never races another worker, and writes `reports/run-context.json` with the environment, the start time and the CI commit and branch.

> **RULE** Anything whose failure would make *every* test fail belongs here, and nothing else does. Per-test setup belongs in a fixture, which knows what that test needs and runs on failure too.

WHEN YOU NEED TO CHANGE IT

- A precondition should stop the whole run.
- The run context should record something more.

HOW TO CHANGE IT

1. Add a check that throws with an actionable message.

```
if (config.strictContracts && !fs.existsSync(OPENAPI_PATH)) {
  throw new Error('STRICT_CONTRACTS is on but src/data/openapi.json is missin
}
```

2. Resist adding data seeding here. Seeding that all tests share is shared mutable state, which is the thing fixtures exist to avoid.

WHY THE CHANGE BELONGS HERE

The difference between "one clear message" and "four hundred connection errors" is entirely about where the check lives. Failing before the first test is worth more than any diagnostic afterwards.

> **WATCH OUT FOR**
>
> - It runs once per **run**, not per worker, so nothing here can be worker-specific.
> - Anything thrown here aborts the run before a single test appears in the report.
> - The request context must be disposed, or the process may not exit.

Related: `src/hooks/global.teardown.ts` · `src/config/env.config.ts` · `playwright.config.ts`

API SURFACE (1 EXPORT)

`FUNCTION` `globalSetup(_playwrightConfig: FullConfig): Promise<void>`

src/hooks/global.teardown.ts

37 lines · 1 exports

Runs once after every test. Clears the token cache and finalises the run context with a duration.

Kept deliberately small. Per-test cleanup belongs in the `cleanup` fixture, which runs even when a test fails and knows what *that* test created; global teardown runs once and knows nothing, so anything it deletes it deletes blindly.

Clearing the token store is a security measure: it stops a live access token lingering in a cache file on a shared build agent.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A shared resource needs closing.
> - The run summary should record something more.

> **HOW TO CHANGE IT**
>
> 1. Close what global setup opened. Keep the symmetry visible.
> 2. Never delete data here based on a naming convention — one wrong pattern deletes somebody else's fixtures, and there is no test to blame it on.

> **WHY THE CHANGE BELONGS HERE**
>
> Teardown that runs once cannot know what any individual test created. Cleanup belongs where the knowledge is, which is the fixture.

> **WATCH OUT FOR**
>
> - It does not run if global setup threw — anything that must always run has to handle that.
> - Playwright does not fail a run because teardown threw, so an error here is easy to miss. Log rather than throw.

Related: `src/hooks/global.setup.ts` · `src/utils/cleanup.registry.ts` · `src/auth/token.store.ts`

API SURFACE (1 EXPORT)

`FUNCTION` **globalTeardown(): void**

Reference: utilities, stubbing & reporting

Cross-cutting helpers, the stub server, and what a run leaves behind

Utilities (src/utils)

src/utils/cleanup.registry.ts 108 lines · 1 exports

Tracks resources a test created so they can be removed afterwards. Deletions run in reverse order of registration, and a failing deletion is reported but never fails the test.

API suites leak. A test creates an order, asserts something, and the order stays forever; a thousand runs later the list endpoint is slow and somebody spends a day cleaning a database by hand. The fix has to be cheap enough that nobody skips it.

Cheap enough — one expression TYPESCRIPT

```
return this.cleanup.track(created, `order ${created.id}`, () => this.remove(creat
```

Two decisions worth knowing

- **Reverse order.** A child resource usually has to go before its parent, and creation order already encodes that relationship. `priority` is there for the rare case where it does not.
- **A failing deletion never fails the test.** A cleanup problem must not disguise itself as a product problem. It is logged, counted, and the drain continues.

`forget(description)` removes an entry — used when the test deleted the resource itself, so teardown does not try again.

> WHEN YOU NEED TO CHANGE IT
>
> - Cleanup ordering needs to be more explicit.
> - A failed cleanup should be reported somewhere.

> HOW TO CHANGE IT
>
> 1 Use `priority` when reverse-creation order is not the right order.
>
> ```
> this.cleanup.register('tenant 42', () => deleteTenant(42), -10); // last
> ```
>
> 2 To surface leaks, attach the drain result to the test in the `cleanup` fixture.
>
> ```
> const { failed } = await registry.drain();
> if (failed) await testInfo.attach('cleanup-failures.txt', { body: String(fail
> ```

> WHY THE CHANGE BELONGS HERE
>
> Cleanup registered at the point of creation is cleanup that happens. Cleanup that depends on somebody writing an `afterEach` is cleanup that happens for the first three tests.

> **WATCH OUT FOR**
>
> - Registering after the drain has started runs the deletion immediately and logs a warning — the alternative is silently dropping it.
> - The `cleanup` fixture drains after every test, including a failed one. A failed test is the most likely to have created something and not removed it.

Related: `src/core/base.service.ts` · `src/fixtures/api.fixture.ts`

API SURFACE (1 EXPORT)

CLASS **CleanupRegistry**

`register(description: string, run: () => Promise<void>, priority = 0): void`	Registers an undo action. Returns the value so it can wrap a creation.
`track(value: T, description: string, remove: () => Promise<void>, priority = 0): T`	Registers a deletion for a created resource and passes the resource through, so a service method reads as one expression: `return this.cleanup.track(created, \user \${created.id}\, () => …)`.
`get size: number`	How many undo actions are pending.
`list(): readonly CleanupEntry[]`	Everything still registered, for assertions and diagnostics.
`forget(description: string): void`	Forgets an entry — use when the test itself already deleted the resource.
`discard(): void`	Abandons everything without running it. For debugging a failing teardown.
`drain(): Promise<{ removed: number; failed: number }>`	Runs every registered deletion, highest priority first and newest first within a priority. Never throws.

src/utils/data.utils.ts

121 lines · 5 exports

Test data: seeded Faker, factories, unique identifiers, and curated collections of adversarial strings, numbers and dates.

Two rules

Generate, do not hard-code. A suite whose fixtures say `alice@example.com` fails the second time it runs against an environment with a uniqueness constraint, and passes forever against a database somebody seeded by hand.

Generate deterministically. Faker is seeded per test from the test's own title, so a failing run can be reproduced exactly while two tests still get different data. Random-but-reproducible is the combination that makes generated data safe to rely on.

Factories state only what the test cares about TYPESCRIPT

```typescript
const admin = buildUser({ role: 'admin' });
const order = buildOrder({ currency: 'USD' });
const invalid = without(buildUser(), 'email');   // for a missing-field test
```

Adversarial input

Four curated collections. Every entry has caused a production incident somewhere.

COLLECTION	CONTAINS
`EDGE_CASE_STRINGS`	Quotes, backslashes, combining marks, RTL text, zero-width characters, emoji sequences, a 5000-character string, `${…}` template syntax
`INJECTION_PAYLOADS`	SQL, NoSQL, XSS, path traversal, command injection, template injection, LDAP, CRLF, XXE

NOTE Numeric and date edge-case collections lived here too and no test read them, so they were removed. If you add one back, remember that `9007199254740993` cannot be written as a JavaScript literal without losing precision — send it as a string and check whether the API round-trips it or silently rewrites it to `…992`.

WHEN YOU NEED TO CHANGE IT

- A resource needs a factory.
- An edge case bit you in production — add it here.

HOW TO CHANGE IT

1. Add a factory beside `buildUser`, taking a seeded Faker instance so it stays deterministic.

```typescript
export function buildInvoice(source: Faker) {
  return {
    reference: `INV-${source.string.numeric(6)}`,
    dueDate: source.date.soon({ days: 30 }).toISOString(),
    totalMinor: source.number.int({ min: 100, max: 500_000 }),
  };
}
```

2. Add edge cases to the existing collections rather than creating a new one, so a table-driven test picks them up automatically.

WHY THE CHANGE BELONGS HERE

Edge-case collections are institutional memory. Every entry is a bug somebody already found; keeping them in one place means the next service gets tested against all of them for free.

WATCH OUT FOR

- The `data` fixture is seeded per test. Importing the global `faker` directly loses that determinism.

- `uniqueId` uses the clock and randomness, so it is unique but **not** reproducible — which is right for a value that must not collide across runs.

Related: `src/fixtures/api.fixture.ts` · `src/utils/security.utils.ts`

API SURFACE (5 EXPORTS)

`FUNCTION` **`seededFaker(seed: string): Faker`**

A Faker instance seeded from a string, so a title maps to stable data.

`FUNCTION` **`uniqueId(prefix = 'pw'): string`**

A run-unique identifier, safe to embed in names that must not collide.

`CONST` **`buildUser`**

A representative user payload. Adapt the shape to the API under test.

`CONST` **`EDGE_CASE_STRINGS`**

Strings that break naive input handling. Every one of these has caused a production incident somewhere: encoding bugs, truncation at a byte boundary rather than a character boundary, unescaped template interpolation, SQL built by concatenation.

`CONST` **`INJECTION_PAYLOADS`**

Payloads a security test sends where a normal value is expected.

src/utils/diff.utils.ts

149 lines · 3 exports

Structural comparison: a value diff with ignorable fields, and a *shape* comparison for detecting breaking changes.

Two jobs

- **Response against a baseline**, which catches the field a service quietly stopped returning.
- **Response against response** — old version against new, one region against another — which is how a migration is verified without writing an assertion per field.

Ignoring what legitimately changes — TYPESCRIPT

```
expect(current).toMatchPayload(baseline, { ignore: VOLATILE_FIELDS });
```

Pass `ignore` for the fields that change on every run — ids, timestamps, trace ids, etags. Without ignoring them every diff is noise, and a diff nobody reads is a check nobody has. `*.name` matches at any depth and `prefix*` matches a prefix.

Ignore what churns, compare the rest TYPESCRIPT

```typescript
expect(current).toMatchPayload(baseline, {
  ignore: ['id', '*.createdAt', 'meta.traceId'],
  nullIsAbsent: true,
});
```

A shape-comparison pair — deriving a type map and reporting only removed or retyped fields — lived here and was never called from a test. It was removed; `src/utils/jsonpath.utils.ts` still provides `leafPaths`, which is the piece such a check would be built on.

Array comparison

Unordered by default, because most list endpoints do not guarantee order. Pass `strictArrayOrder` when the order is part of the contract — a sorted result, a sequence of events.

WHEN YOU NEED TO CHANGE IT

- A field should be ignored everywhere.
- A different comparison semantic is needed.

HOW TO CHANGE IT

1. Pass it in `ignore` at the call site, or add a shared constant beside the matcher in `src/fixtures/custom-matchers.ts` if the whole suite should skip it.
2. Add a `DiffOptions` flag rather than a second function, so one walker keeps all the semantics.

WHY THE CHANGE BELONGS HERE

The failure message is the reason this exists. Finding one changed field inside a large response by eye is genuinely difficult; a diff makes it immediate.

WATCH OUT FOR

- Unordered array matching is O(n²). For very large arrays, pass `strictArrayOrder` if order is in fact stable.
- Paths collapse array indices to `[]`, so a two-item and a three-item response compare at the same paths — otherwise every list endpoint diffs against itself.

Related: `src/fixtures/custom-matchers.ts` · `src/utils/jsonpath.utils.ts`

API SURFACE (3 EXPORTS)

INTERFACE **DiffOptions**

```typescript
readonly ignore?: string[];
readonly strictArrayOrder?: boolean;
readonly nullIsAbsent?: boolean;
```

FUNCTION

diff(expected: unknown, actual: unknown, options: DiffOptions = {}): Difference[]

Every difference between two payloads. Empty means they match.

FUNCTION **formatDiff(differences: readonly Difference[]): string**

A readable multi-line rendering, for an assertion message.

src/utils/file.utils.ts

18 lines · 1 exports

Resolving a path against the repository root, so the suite behaves the same regardless of the working directory it was launched from.

The whole module is `fromRoot`. It is used by `global.setup.ts` to create the report directories and by `api.fixture.ts` to find an optional OpenAPI specification — both of which run before anything has established a working directory.

The whole module — TYPESCRIPT

```typescript
fs.mkdirSync(fromRoot('reports', 'artifacts'), { recursive: true });
const spec = fromRoot('src', 'data', 'openapi.json');
```

This module used to carry readers for JSON, CSV and NDJSON, temporary-file builders, checksums and an artefact writer, alongside a `src/data/` folder of fixtures to feed them. No test read any of it, so the helpers, the fixtures and the `csv-parse` dependency were removed together.

NOTE If a data-driven suite is wanted later, add the reader in the same change as the first test that calls it. A reader with no caller is indistinguishable from a reader nobody needs — which is exactly how the previous set accumulated.

WHEN YOU NEED TO CHANGE IT

- A test needs to read a data file, or a run should leave an artefact behind.

HOW TO CHANGE IT

1. Add the reader here: resolve the path through `fromRoot`, check existence with an error naming the path it looked in, parse, and return a typed value.
2. For an upload-limit test, generate the file at run time rather than committing a multi-megabyte fixture that slows every clone forever.

> **WHY THE CHANGE BELONGS HERE**
>
> Path resolution is the one file concern every entry point genuinely shares. Everything else was speculative.

Related: `src/hooks/global.setup.ts` · `src/fixtures/api.fixture.ts`

API SURFACE (1 EXPORT)

`FUNCTION` **`fromRoot(...segments: string[]): string`**

Resolves a path relative to the repository root.

src/utils/header.utils.ts

220 lines · 11 exports

Header parsing: case-insensitive lookup, `Set-Cookie`, `Content-Type`, `Link`, `Retry-After`, rate-limit counters, and redaction.

HTTP headers are case-insensitive, may repeat, and several of the ones that matter carry structured values inside a single string. Parsing them in one place keeps that fiddly, easy-to-get-subtly-wrong logic out of the tests.

HELPER	HANDLES
`parseSetCookie`	Name, value and every attribute. Splits on **newlines**, because Playwright joins repeated headers that way — a comma appears inside an `Expires` date.
`parseContentType`	Media type plus parameters. `isJsonContentType` also accepts `+json` suffixes such as HAL.
`parseLinkHeader` / `nextLink`	RFC 8288 pagination — GitHub-style `rel="next"`.
`parseRetryAfter`	Either form — delta-seconds or an HTTP date — normalised to milliseconds.
`parseRateLimit`	`X-RateLimit-*`, treating a small `reset` as a delta and a large one as an epoch timestamp.
`redactHeaders` / `redactValue`	Replaces a credential with a correlatable stub.

Redaction keeps enough to correlate, not enough to use — TYPESCRIPT

```ts
redactValue('eyJhbGciOiJIUzI1NiIs…')
// → 'eyJh…Is (243 chars, redacted)'
```

That form is deliberate: two log lines with the same stub are the same credential, which is often exactly what you need to know, without the value ever being usable.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A header carries structure nothing parses.

- A credential-bearing header is not being redacted.

HOW TO CHANGE IT

1. Add the header name to `SENSITIVE_HEADERS` — that is all redaction needs.
2. Add a parser returning a typed shape rather than a string, so callers do not re-parse.

WHY THE CHANGE BELONGS HERE

These parsers are individually trivial and collectively the source of a lot of subtle bugs — the `Set-Cookie` comma being the classic. One implementation, tested once, used everywhere.

WATCH OUT FOR

- `HeaderMap` keys are lower-cased by convention. `getHeader` compares case-insensitively anyway, for safety.
- Redaction happens on the way into logs, reports and recordings. Anything that writes headers elsewhere must call it too.

Related: `src/core/api.response.ts` · `src/mocks/recorder.ts` · `src/utils/pagination.utils.ts`

API SURFACE (11 EXPORTS)

FUNCTION **`getHeader(headers: HeaderMap, name: string): string | undefined`**

Case-insensitive single header lookup.

INTERFACE **`ParsedCookie`**

One `Set-Cookie` entry, split into its name, value and attributes.

```
readonly name: string;
readonly value: string;
readonly domain?: string;
readonly path?: string;
readonly expires?: string;
readonly maxAge?: number;
readonly secure: boolean;
readonly httpOnly: boolean;
readonly sameSite?: 'Strict' | 'Lax' | 'None';
```

FUNCTION **`parseSetCookie(raw: string | undefined): ParsedCookie[]`**

Parses `Set-Cookie`. Playwright joins repeated headers with a newline, so that is the separator honoured here — splitting on commas would break on the comma inside an `Expires` date.

FUNCTION **`parseContentType(raw: string | undefined): ParsedContentType`**

FUNCTION **isJsonContentType(raw: string | undefined): boolean**

True when a content type is JSON, including `+json` suffixes like HAL.

FUNCTION **parseLinkHeader(raw: string | undefined): LinkRelation[]**

Parses `Link: <…>; rel="next", <…>; rel="last"` — GitHub-style pagination.

FUNCTION **nextLink(raw: string | undefined): string | undefined**

The `next` URL from a `Link` header, when the server sent one.

FUNCTION

parseRetryAfter(raw: string | undefined, now = Date.now()): number | undefined

`Retry-After` in milliseconds. The header is either delta-seconds or an HTTP date; both forms are normalised so backoff code has one thing to read.

INTERFACE **RateLimitInfo**

Rate-limit counters, using the widely adopted `X-RateLimit-*` convention.

```
readonly limit?: number;
readonly remaining?: number;
readonly resetAt?: number;
readonly retryAfterMs?: number;
```

FUNCTION **parseRateLimit(headers: HeaderMap, now = Date.now()): RateLimitInfo**

FUNCTION **redactHeaders(headers: HeaderMap): HeaderMap**

src/utils/jsonpath.utils.ts

134 lines · 4 exports

A small, dependency-free JSON path reader supporting property access, array indices (including negative), wildcards and recursive descent.

Full JSONPath implementations bring a parser, a filter-expression evaluator and a large dependency tree. What API assertions actually need is the ability to name a nested value — so that is what this supports, with syntax any reviewer can read without consulting a specification.

SYNTAX	MATCHES
`a.b.c`	Property access
`a[0]`, `a[-1]`	Array index; negative counts from the end

SYNTAX	MATCHES
`a[*].b`	Every element of an array
`a.*`	Every value of an object
`..name`	Every `name` at any depth

The four entry points — TYPESCRIPT

```ts
readPath(payload, 'data.items[0].id');   // one value, or undefined
readAll(payload, '..email');             // every match
hasPath(payload, 'meta.cursor');         // presence
leafPaths(payload);                      // every leaf path, for shape comparison
```

`leafPaths` is what `ApiResponse.fields()` is built on: it turns a payload into the set of paths it contains, which is the thing worth comparing against a baseline.

WHEN YOU NEED TO CHANGE IT

- A path expression the framework needs is not supported.

HOW TO CHANGE IT

1. Add a token kind in `tokenize` and a case in `collect`. Both are short and the change is local.

```ts
// e.g. a slice: a[0:3]
if (/^\[\d+:\d+\]$/.test(raw)) tokens.push({ kind: 'slice', from, to });
```

2. Resist adding filter expressions. At that point a real JSONPath library is the honest answer.

WHY THE CHANGE BELONGS HERE

Paths make assertions readable. `response.path('data.items[0].id')` says what it is looking for; a chain of optional accesses says how it is looking for it.

WATCH OUT FOR

- Wildcards and `..` return arrays. `readPath` takes the first match, which is rarely what you want with those — use `readAll`.
- A path that does not resolve returns `undefined` rather than throwing, so `expectPathExists` is the way to assert presence.

Related: `src/core/api.response.ts` · `src/utils/diff.utils.ts`

API SURFACE (4 EXPORTS)

FUNCTION **`readPath(source: unknown, path: string): unknown`**

Reads a single value. Returns `undefined` when the path does not resolve.

`FUNCTION` **`readAll(source: unknown, path: string): unknown[]`**

Reads every value a path matches. Wildcards and `..` can match many.

`FUNCTION` **`hasPath(source: unknown, path: string): boolean`**

True when the path resolves to anything other than `undefined`.

`FUNCTION` **`leafPaths(source: unknown, prefix = ''): string[]`**

Every leaf path in a payload, as dotted strings. Useful for asserting that a response gained or lost fields between versions, and for writing a first draft of a schema from a real response.

src/utils/logger.ts

112 lines · 2 exports

A dependency-free structured logger. Readable lines locally, single-line JSON in CI for log aggregation.

Hand-written rather than pulled from npm, deliberately: the popular logging libraries bring transports, colour packages and native bindings the suite does not need, and at least one of them fails to load on current Node. This is eighty lines with no dependencies.

Scoped loggers, so a line says where it came from — TYPESCRIPT

```typescript
const log = logger.child('auth:oauth2');
log.debug('requesting token', { grant: 'client_credentials', url });
```

Everything goes to **stderr**, so a test's stdout — used by reporters and by `--reporter=json` — is never polluted by log lines.

> **NOTE** The ANSI escape character is built with `String.fromCharCode(27)` rather than written as a literal, so no control byte appears in the source. That keeps the file safe to open in any editor and to paste into a diff.

WHEN YOU NEED TO CHANGE IT

- Logs need a field on every line.
- A destination other than stderr is needed.

HOW TO CHANGE IT

1. Add the field in `write`, before the context spread, so a caller cannot accidentally shadow it.

   ```
   const line = JSON.stringify({ time, level, scope: this.scope, runId, message,
   ```

2. For a new destination, change `emit`. Keep stderr as the default — stdout belongs to the reporters.

> **WHY THE CHANGE BELONGS HERE**
>
> Log output is the diagnostic of last resort, so it has to work everywhere and never be the thing that breaks. No dependencies is the strongest guarantee of that available.

> **WATCH OUT FOR**
>
> - `enabled(level)` exists so an expensive context object can be skipped rather than built and discarded.
> - JSON mode is on automatically in CI, off locally. Force it with the `json` option if you need it either way.

Related: `src/config/env.config.ts` · `src/core/http.client.ts`

API SURFACE (2 EXPORTS)

CLASS Logger

`child(scope: string): Logger`	A logger for a sub-component, inheriting level and format.
`error(message: string, context?: LogContext): void`	
`warn(message: string, context?: LogContext): void`	
`info(message: string, context?: LogContext): void`	
`debug(message: string, context?: LogContext): void`	
`enabled(level: LogLevel): boolean`	True when a level would be emitted — guard expensive context building.

CONST logger

The shared logger. Call `.child()` rather than constructing new roots.

src/utils/pagination.utils.ts

92 lines · 3 exports

Walkers for the four pagination conventions — `Link` header, cursor, offset — plus a normaliser and a defect detector.

Every API paginates differently, and a test that only ever reads page one will pass while page two is broken. These turn any convention into one list, with a hard page ceiling so a server bug cannot become an infinite loop.

WALKER	CONVENTION
`followLinkHeader`	RFC 8288 `rel="next"` — the only style needing no knowledge of the API's parameters

WALKER	CONVENTION
`followOffset`	Offset/limit, stopping when a short page comes back

The defect detector

The two bugs pagination tests exist to catch — TYPESCRIPT

```ts
const defects = findPaginationDefects(pages, (item) => item.id);
expect(defects.duplicates).toEqual([]);
expect(defects.uniqueItems).toBe(defects.totalItems);
```

An item on two pages, and an item on none. Both happen when the underlying data changes mid-walk, which is exactly why cursor pagination exists — and exactly what makes it worth testing.

NOTE `followCursor` stops if a cursor repeats. A server that loops would otherwise hang the test until the whole-test timeout; this turns it into a clear, reportable failure.

WHEN YOU NEED TO CHANGE IT

- The API uses a convention none of these cover.
- The page ceiling is too low for a legitimate dataset.

HOW TO CHANGE IT

1. Add a walker following the same shape: a reader callback, a ceiling, and an options object.

```ts
export async function followKeyset<T>(
  readPage: (after: string | undefined) => Promise<{ items: T[]; last?: strin
  options: PaginationOptions = {},
): Promise<T[]> { /* … */ }
```

2. Raise `MAX_PAGES` only with a reason; the ceiling exists to turn a hang into a failure.

WHY THE CHANGE BELONGS HERE

Pagination is where the difference between "the endpoint works" and "the endpoint works at scale" shows up, and it is almost never covered because walking pages by hand in a test is tedious. Making it one line removes the excuse.

WATCH OUT FOR

- The ceiling logs a warning when hit rather than throwing, so a legitimate large dataset produces partial results with a visible reason.

Related: `src/utils/header.utils.ts` · `src/services/template.service.ts`

API SURFACE (3 EXPORTS)

FUNCTION

```
followLinkHeader(first: ApiResponse, fetchNext: (url: string) =>
Promise<ApiResponse>, extract: (response: ApiResponse) => T[], options:
PaginationOptions = {}): Promise<T[]>
```

Follows `Link: rel="next"` until the server stops sending one. The GitHub convention, and the only style where the client needs no knowledge of the API's parameter names at all.

FUNCTION

```
followOffset(readPage: (offset: number, limit: number) => Promise<T[]>,
pageSize = 50, options: PaginationOptions = {}): Promise<T[]>
```

Follows offset/limit pagination until fewer than `limit` items come back.

FUNCTION

```
findPaginationDefects(pages: T[][], identify: (item: T) => string): {
duplicates: string[]; totalItems: number; uniqueItems: number }
```

Checks a paginated endpoint for the two defects pagination tests exist to catch: an item that appears on two pages, and one that appears on none.

src/utils/performance.utils.ts

171 lines · 5 exports

Latency measurement: percentiles, a collector that samples every request, and a sampler for repeated calls.

An API suite is the cheapest place a team ever gets performance signal — the requests are already being made, so the timings are already there. What is usually missing is the discipline to look at a *distribution* rather than a single number: a mean hides the tail, and the tail is what users feel.

Attached automatically by the fixture — TYPESCRIPT

```typescript
// Every request in the test is sampled:
client.onExchange(latency.record);

// And the report is attached as latency.json when the test ends.
latency.report();   // [{ route: 'GET /users/{id}', summary: { p95: 412, … } }]
```

Nearest-rank percentiles

Rather than interpolation, because nearest-rank always returns a value that was **actually observed** — which is what makes a reported p95 defensible in a conversation with the team that owns the service.

Route collapsing

Otherwise every request is its own bucket — TYPESCRIPT

```
routeOf('https://api/v1/users/42')                    // '/v1/users/{id}'
routeOf('https://api/v1/orders/3f2b…-a1c9/items')      // '/v1/orders/{uuid
```

CAREFUL Latency percentiles from a functional suite are useful signal. They are not a load test, and this module is not a load-testing tool — measuring under load needs a tool built for it.

WHEN YOU NEED TO CHANGE IT

- A route shape is not being collapsed.
- Another statistic is needed.

HOW TO CHANGE IT

1. Add a pattern to `routeOf` for your identifier shape.

```
if (/^[A-Z]{3}-\d{6}$/.test(segment)) return '{reference}';
```

2. Add the statistic to `summarize` and to `LatencySummary` together.

WHY THE CHANGE BELONGS HERE

Free signal that nobody has to set up is signal that actually gets looked at. Attaching the report to every test means the data is there when somebody asks "was it always this slow?".

WATCH OUT FOR

- Measure in the `performance` project, which runs one worker. A p95 measured under eight parallel workers describes the load the suite generates.
- `sample()` discards warm-up runs by default: the first call to a cold endpoint measures connection setup and JIT.
- `record` is a bound property, so it can be passed to `onExchange` directly.

Related: `src/fixtures/api.fixture.ts` · `src/core/http.client.ts` · `playwright.config.ts`

API SURFACE (5 EXPORTS)

INTERFACE **LatencySummary**

A summary of many samples.

```
readonly count: number;
readonly min: number;
readonly max: number;
readonly mean: number;
readonly p50: number;
readonly p90: number;
readonly p95: number;
readonly p99: number;
readonly stdDev: number;
```

FUNCTION **percentile(samples: readonly number[], p: number): number**

A percentile using nearest-rank on a sorted sample. Nearest-rank rather than interpolation because it always returns a value that was actually observed, which is what makes a reported p95 defensible in a conversation with the team that owns the service.

FUNCTION **summarize(samples: readonly number[]): LatencySummary**

Summarises a set of durations in milliseconds.

CLASS **LatencyCollector**

Collects timings across a test or a whole run. Attach it to a client with `client.onExchange(collector.record)` and every request is sampled with no further ceremony.

`record`	Records one exchange. Bind before passing to `onExchange`.
`add(key: string, durationMs: number): void`	Adds a sample by hand — for work that is not a single HTTP request.
`summaryFor(key: string): LatencySummary`	The summary for one route.
`report(): { route: string; summary: LatencySummary }[]`	Every route sampled, slowest p95 first.
`breaches(budgetMs: number): { route: string; p95: number }[]`	Routes whose p95 exceeds a budget — the list worth failing a build on.
`clear(): void`	

FUNCTION

sample(operation: () => Promise<unknown>, options: { runs?: number; warmup?: number; concurrency?: number } = {}): Promise<LatencySummary>

Runs an operation repeatedly and summarises the result. `warmup` runs are discarded, because the first call to a cold endpoint measures connection setup and JIT rather than the endpoint itself.

src/utils/security.utils.ts

223 lines · 8 exports

Security checks that belong in a functional API suite: information disclosure, security headers, CORS, and authorisation matrices.

Not a penetration test — that is a different activity with different authorisation. These are the checks that catch the mistakes teams actually ship: an endpoint that forgot its authorisation check, a header that leaks the framework version, CORS opened to `*` with credentials, an error message that returns a stack trace to the caller.

Each helper **reports findings rather than throwing**, so a test can assert on the whole set and the report shows every problem at once.

Disclosure

RULE	CATCHES
`version-disclosure`	A `Server` or `X-Powered-By` header containing a version number
`stack-trace`	A stack frame in a response body
`sql-fragment`	A SQL statement echoed back
`file-path`	A server-side filesystem path
`connection-string`	A database URL
`private-key`, `aws-key`, `bearer-token`	Credentials in a response — the high-severity findings

A bare product name in `Server` is acceptable; a version number is the disclosure. That distinction is in the rule, so it does not produce noise.

CORS

The dangerous combination is `Access-Control-Allow-Origin: *` together with `Access-Control-Allow-Credentials: true`. Browsers reject it, so in practice it means the server is *reflecting* whatever origin it is sent — which is worse, and is checked separately.

The authorisation matrix

Generate the grid, do not write the cases you thought of — TYPESCRIPT

```typescript
const matrix = buildAccessMatrix(
  ['admin', 'standard', 'readonly'],
  [{ method: 'DELETE', path: '/users/{id}', allowed: ['admin'] }],
);
// → 3 expectations: admin permitted, the other two refused.
```

`judgeAccess` knows that 401, 403 **and 404** all count as refusal — returning 404 to hide a resource's existence is a legitimate design. Returning 200 to a role that should be refused is always a finding.

> **RULE** Authorisation defects are defects of *omission* — the endpoint nobody remembered to protect. Generating the whole grid is the only way to catch that class reliably.

WHEN YOU NEED TO CHANGE IT

- A disclosure pattern is missing.
- The security-header baseline should change.
- Findings need different severities.

HOW TO CHANGE IT

1. Add a pattern to `LEAK_PATTERNS` with an appropriate severity.

```
{ rule: 'internal-hostname', pattern: /\b\w+\.internal\.example\.com\b/ },
```

2. Adjust `auditSecurityHeaders` to match your organisation's baseline — the defaults are defensible, not universal.

WHY THE CHANGE BELONGS HERE

These checks cost almost nothing once the requests are already being made, and they catch a class of defect that functional tests never look at. Reporting rather than throwing is what lets one test cover a whole surface.

WATCH OUT FOR

- Never point the `security` project at production. The payloads are deliberately hostile.
- The header baseline is opinionated. Loosen it deliberately rather than deleting the test.
- `buildAccessMatrix` assumes a permitted call returns 200. Pass `allowedStatus` for a resource that returns 201 or 204.

Related: `src/utils/data.utils.ts` · `src/core/api.response.ts` · `playwright.config.ts`

API SURFACE (8 EXPORTS)

INTERFACE **SecurityFinding**

One thing worth fixing, with enough context to act on it.

```
readonly severity: 'high' | 'medium' | 'low';
readonly rule: string;
readonly detail: string;
```

FUNCTION **auditDisclosure(response: ApiResponse): SecurityFinding[]**

Checks a response for information disclosure.

FUNCTION **auditSecurityHeaders(headers: HeaderMap): SecurityFinding[]**

Checks the baseline security headers a public API should send.

FUNCTION

auditCors(headers: HeaderMap, requestOrigin: string): SecurityFinding[]

Checks a CORS preflight. The dangerous combination is `Access-Control-Allow-Origin: *` together with `Access-Control-Allow-Credentials: true` — browsers reject it, so it usually means the server is reflecting whatever origin it is sent, which is worse.

INTERFACE **AccessExpectation**

One cell of an authorisation matrix: who, doing what, and what should happen.

```
readonly role: string;
readonly method: HttpMethod;
readonly path: string;
readonly expected: number;
readonly note?: string;
```

FUNCTION

buildAccessMatrix(roles: readonly string[], operations: readonly { method: HttpMethod; path: string; allowed: readonly string[] }[], options: { deniedStatus?: number; allowedStatus?: number } = {}): AccessExpectation[]

Builds the full matrix from a list of roles and operations. Authorisation defects are defects of *omission* — the endpoint nobody remembered to protect. Generating the whole grid, rather than writing the cases somebody thought of, is the only way to catch that class reliably.

FUNCTION

judgeAccess(expectation: AccessExpectation, actualStatus: number): SecurityFinding | undefined

Judges an authorisation response. The distinction that matters: 401 means "we do not know who you are", 403 means "we know, and no". Returning 404 to hide a resource's existence is a legitimate design and is accepted here too, but returning 200 to a role that should be refused is always a finding.

FUNCTION **formatFindings(findings: readonly SecurityFinding[]): string**

Formats findings for an assertion message or a report attachment.

src/utils/xml.utils.ts

125 lines · 4 exports

XML and SOAP support: parsing, hand-rolled serialisation, path reading, envelope construction and fault extraction.

Plenty of production APIs still speak XML — SOAP services, RSS and Atom feeds, sitemaps, legacy partner integrations. The response wrapper delegates here, so a project that never touches XML never imports this module.

Parser conventions

Attributes are prefixed `@` and text content is exposed as `#text`, so a parsed document can be walked with the **same dotted paths used for JSON**. Namespace prefixes are stripped, so `soap:Envelope` reads as `Envelope` — assertions should not have to know which prefix the server chose today.

One path syntax for both formats — TYPESCRIPT

```ts
const body = parseXml(response.text());
readPath(body, 'Envelope.Body.GetPriceResponse.Price');
readPath(body, 'catalog.item[0].@id');
```

SOAP faults

SOAP reports application errors with HTTP 500 and a fault in the body, so a status assertion alone will not tell you what went wrong. `soapFault` reads both conventions — 1.1's `faultcode` / `faultstring` and 1.2's `Code` / `Reason` — so one helper works against either version.

> **NOTE** This module parses; it does not serialise. An envelope builder lived here and no test ever sent SOAP, so it was removed. Add one back alongside `parseXml` when a test needs to send a request rather than read a response.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The service needs SOAP 1.2 envelope construction.
> - Namespace prefixes must be preserved.

> **HOW TO CHANGE IT**
>
> 1. For SOAP 1.2, add an envelope builder beside `parseXml`, escaping all five reserved characters. Write it by hand rather than pulling in the parser package's builder, which is deprecated in favour of a separate dependency.
> 2. To keep prefixes, set `removeNSPrefix: false` — and expect every path in the suite to need updating.

> **WHY THE CHANGE BELONGS HERE**
>
> Making XML read through the same path syntax as JSON is what lets one set of assertion habits cover both. A test author should not need a different mental model because a service is older.

> **WATCH OUT FOR**
>
> - Namespace stripping means two differently-namespaced elements with the same local name collide. That has not happened in practice, but it is the trade-off.
> - The parser coerces attribute values, so `@id="1"` becomes the number `1`.

Related: `src/core/api.response.ts` · `src/utils/jsonpath.utils.ts`

API SURFACE (4 EXPORTS)

FUNCTION **`parseXml(xml: string): UnknownRecord`**

Parses an XML document into a plain object.

FUNCTION **`looksLikeXml(body: string): boolean`**

True when a body looks like XML, so the response wrapper can pick a parser.

INTERFACE **SoapFault**

A SOAP `<Fault>` when the response carries one. SOAP reports application errors with HTTP 500 and a fault in the body, so a status assertion alone will not tell you what went wrong — this will.

```
readonly code: string;
readonly message: string;
readonly detail?: unknown;
```

FUNCTION **`soapFault(xml: string): SoapFault | undefined`**

Stubbing and recording (src/mocks)

`src/mocks/mock.server.ts` 303 lines · 5 exports

A programmable stub server: match a request, return a response — with delays, failures and controlled flakiness on demand.

Three things are impossible to test against a real dependency: the failure you cannot trigger on demand (a 503 from a payment provider), the response you cannot produce (a malformed payload from a partner), and the timing you cannot control (a gateway timeout). A stub you control makes all three routine, and lets the contract suite run with no network at all.

METHOD	PRODUCES
`get(path, json)` / `post(path, json)`	A fixed JSON response
`fail(path, status)`	An error status — the case a real dependency will not give you
`slow(path, delayMs)`	A delayed response, for exercising client timeouts
`flaky(path, failures, success)`	**Fails n times, then succeeds** — how retry behaviour is tested honestly
`stub({ method, path, respond, times })`	The general form; `respond` can be a function of the request

Why flaky() matters — TYPESCRIPT

```typescript
mockServer.flaky('/payments', 2, { status: 'settled' });

const response = await client.post('/payments').retries(3).send();
response.expectOk();
expect(response.timing.attempts).toBe(3);
```

The endpoint really does recover, so a client that gives up too early fails and one that retries forever never finishes. A mocked-out client that always succeeds proves neither.

Asserting on what the client sent

TYPESCRIPT

```typescript
const [request] = mockServer.requestsFor('POST', '/orders');
expect(request.headers['idempotency-key']).toBeDefined();
expect(bodyOf(request).currency).toBe('GBP');
expect(mockServer.callCount('/orders')).toBe(1);   // caching, or a stray retry
```

The unmatched-request response

An unmatched request returns **501 with a body listing what arrived and what was registered**. That is almost always a test-authoring mistake, and a silent 404 would make it a ten-minute puzzle.

Built on Node's `http` module rather than a framework: the whole surface is "match a request, return a response", and every dependency added to a test framework is one more thing that can break a release.

> **WHEN YOU NEED TO CHANGE IT**
>
> - Stubs need behaviour that is not expressible.
> - Path matching needs another form.

> **HOW TO CHANGE IT**
>
> 1 Most behaviour is already a function of the request — no code change needed.

```
  mockServer.stub({
    method: 'GET',
    path: '/orders/*',
    respond: (request) => ({
      status: request.query.tenant === 'known' ? 200 : 404,
      json: { id: request.path.split('/').pop() },
    }),
  });
```

2 For streaming or another protocol, add a branch in `handle` rather than a second server.

WHY THE CHANGE BELONGS HERE

The stub server is what makes the contract suite runnable on a fork's pull request, with no environment and no secrets. That property is worth more than any individual stub.

WATCH OUT FOR

- `start(0)` asks the OS for a free port, which is what the fixture does — parallel workers would otherwise collide on a fixed port.
- Later registrations win, so a test can override what setup registered. `reset()` clears everything, and is worth calling at the top of a test that registers its own.
- Path globs escape everything that is not a wildcard, so a path containing a dot or a bracket cannot accidentally behave as a pattern.

Related: `src/mocks/recorder.ts` · `src/fixtures/api.fixture.ts` · `docker-compose.yml`

API SURFACE (5 EXPORTS)

INTERFACE **StubResponse**

What a stub sends back.

```
readonly status?: number;
readonly headers?: HeaderMap;
readonly json?: unknown;
readonly body?: string | Buffer;
readonly delayMs?: number;
```

INTERFACE **Stub**

One registered stub.

```
readonly method: HttpMethod | 'ANY';
readonly path: string;
readonly respond: StubResponse | ((request: RecordedRequest) => StubResponse |
readonly times?: number;
readonly name?: string;
```

INTERFACE RecordedRequest

A request the server received, kept so tests can assert on it.

```
readonly method: string;
readonly url: string;
readonly path: string;
readonly query: Record<string, string>;
readonly headers: HeaderMap;
readonly body: string;
readonly json?: unknown;
readonly receivedAt: number;
```

CLASS MockServer

`get requests: readonly RecordedRequest[]`	Every request the server received, in order.
`get url: string`	Base URL to point a client at, once started.
`start(port = config.mockServerPort): Promise<string>`	Starts listening. Port 0 asks the OS for a free port, avoiding clashes.
`stop(): Promise<void>`	Stops listening and releases the port.
`stub(stub: Stub): this`	Registers a stub. Later registrations win, so a test can override setup.
`get(path: string, json: unknown, status = 200): this`	Shorthand for a JSON response to a GET.
`post(path: string, json: unknown, status = 201): this`	Shorthand for a JSON response to a POST.
`fail(path: string, status = 500, json: unknown = { error: 'stubbed failure' }): this`	Replies with an error status — the case a real dependency will not give you.
`slow(path: string, delayMs: number, json: unknown = {}): this`	Replies after a delay, to exercise client timeouts.
`flaky(path: string, failures: number, success: unknown, status = 503): this`	Fails a given number of times, then succeeds. This is how retry behaviour is tested honestly: the endpoint really does recover, so a client that gives up too early fails and one that retries forever never finishes.
`reset(): void`	Forgets every stub and every recorded request.
`requestsFor(method: string, path: string): RecordedRequest[]`	Requests matching a method and path, for asserting what the client sent.
`callCount(path: string): number`	How many times a path was called — the assertion for caching and retries.

FUNCTION

```
stubFromRecording(entry: { method: HttpMethod; path: string; status: number; body: string; headers?: HeaderMap; }): Stub
```

A stub built from a recorded exchange — see `recorder.ts`.

src/mocks/objects.stub.ts

260 lines · 2 exports

An **executable model** of the `api.restful-api.dev/objects` contract: the whole CRUD resource, in memory, reproducing every observed quirk.

Why it exists

The live API is genuinely persistent, which is what makes it worth testing against — but it allows only 50 anonymous requests per 24 hours. A suite that burns the day's quota on its third run is a suite nobody can use.

1. The same service object and the same lifecycle can be exercised **exhaustively, offline, with no quota** — in CI, on a fork's pull request, on a plane.
2. The model is a **check on our understanding**. When the live suite runs within quota and agrees with it, the contract is right. When they disagree, either the API changed or we misread it.
3. Failure modes the real API will not produce on demand — a 500, a timeout, a malformed body — become ordinary test setup.

The quirks it reproduces deliberately

> RULE A stub that behaves the way you *wish* the API behaved tests nothing. Every quirk below was observed against the live service and is reproduced exactly, including the wording of the error messages.

- POST answers **200**, not 201.
- POST carries `createdAt`; PUT and PATCH carry `updatedAt`; **GET carries neither**.
- DELETE answers **200 with a message body**; a second DELETE answers 404.
- Created objects are retrievable by id but **never** listed.
- Errors are `{ error: string }`, not problem details.
- A malformed body answers 400; an empty body is **accepted** and stored with `name: null`.

Using it

Register, then point a derived client at the stub — TYPESCRIPT

```typescript
test.beforeEach(({ mockServer }) => {
  mockServer.reset();
  stubObjectsApi(mockServer);
});

const objects = new ObjectService(http.withBaseUrl(mockServer.url), { cleanup });
```

The reset-and-re-register in `beforeEach` is what keeps the tests independent: the stub server is worker-scoped, so without it one test's writes would be visible to the next.

WHEN YOU NEED TO CHANGE IT

- The live API changes.
- A behaviour is not modelled and a test needs it.

HOW TO CHANGE IT

1. Change the handler, then **run the live suite within quota** to confirm the model and the API still agree. A model nobody checks against reality is a fixture that encodes an assumption and then defends it forever.
2. For a one-off failure mode, prefer the stub server's own helpers — `fail`, `slow`, `flaky` — over adding a branch here.

WHY THE CHANGE BELONGS HERE

Running the same service object against both a real API and a model of it is what makes the model trustworthy, and what lets the exhaustive coverage run on every pull request rather than three times a day.

WATCH OUT FOR

- The stub server collapses repeated query parameters, so the list handler parses `request.url` directly to see the repeated `id` form the live API requires.
- Ids are generated in the same hex shape the live API uses, so a test asserting the format passes against both.

Related: `tests/contract/objects.stubbed.spec.ts` · `src/services/object.service.ts` · `src/mocks/mock.server.ts`

API SURFACE (2 EXPORTS)

CONST **SEEDED_COUNT**

How many objects the live list endpoint returns. Asserted by the tests.

FUNCTION

```
stubObjectsApi(server: MockServer): { /** Everything currently stored, seeded objects included. */ readonly store: Map<string, StoredObject>; /** Resets to just the seeded objects. Call between tests that share a worker. */ reset: () => void; }
```

Registers the whole `/objects` contract on a stub server. Returns a handle exposing the backing store, so a test can seed a specific state or assert on what was written without going through HTTP.

src/mocks/recorder.ts

120 lines · 1 exports

Records exchanges and replays them as stubs. Evidence for a bug report, and a fixture source for running offline.

Record, then replay with no network — TYPESCRIPT

```ts
// Recording — the fixture attaches this on failure automatically.
client.onExchange(recorder.record);
recorder.save('src/data/recordings/orders.json');

// Replaying.
const recording = ExchangeRecorder.load('src/data/recordings/orders.json');
for (const stub of ExchangeRecorder.toStubs(recording)) mockServer.stub(stub);
```

RULE Recordings are **redacted on the way out**. A recording containing a live bearer token is a credential sitting in the repository, and the only reliable defence is to make redaction the default rather than a step somebody has to remember.

`toStubs` replays only successful exchanges by default. A recording made while the target was briefly broken would otherwise bake that breakage into every future offline run.

WHEN YOU NEED TO CHANGE IT

- Recordings should capture more.
- Replay should include error responses.

HOW TO CHANGE IT

1. Include errors deliberately, when the point of the replay is the error path.

   ```ts
   ExchangeRecorder.toStubs(recording, { includeErrors: true });
   ```

2. To capture more, extend `RecordedExchange` — and check the redaction still covers everything added.

> **WHY THE CHANGE BELONGS HERE**
>
> A recorded exchange is the artefact that ends a "works for me" conversation. It shows exactly what was sent and exactly what came back, at the moment it failed.

> **WATCH OUT FOR**
>
> - Response bodies are only recorded when `LOG_BODIES` is on, because a full body on every request is a lot of memory in a large suite.
> - Check a recording before committing it. Redaction covers the known credential headers; a token in a *body* is on you.
> - The fixture saves only on failure — a passing test's recording is noise.

Related: `src/mocks/mock.server.ts` · `src/utils/header.utils.ts` · `src/fixtures/api.fixture.ts`

API SURFACE (1 EXPORT)

CLASS **ExchangeRecorder**

`record`	Records one exchange. Bind before passing to `client.onExchange`.
`get size: number`	
`save(filePath: string): void`	Writes the recording to disk.
`load(filePath: string): Recording`	Loads a recording written earlier.
`toStubs(recording: Recording, options: { includeErrors?: boolean } = {}): Stub[]`	Turns a recording into stubs for the mock server. Only successful exchanges are replayed by default: a recording made while the target was briefly broken would otherwise bake that breakage into every future offline run.

Reporters (src/reporters)

src/reporters/summary.reporter.ts 176 lines · 1 exports

Writes `reports/summary.json`: one machine-readable file saying whether the run passed, which tests failed and why, which were slowest, and how long it all took.

Playwright's HTML report is excellent for a person with a browser. What a pipeline needs is different: one file small enough to post into a chat message or gate a deployment on. Every field is chosen to be diffable between two runs of the same suite.

What it produces JSON

```
{
  "status": "failed",
  "environment": "staging",
  "durationMs": 184320,
  "totals": { "total": 412, "passed": 405, "failed": 2, "flaky": 3, "skipped": 2,
  "passRate": 98,
  "slowestTests": [{ "title": "orders › bulk import", "durationMs": 24118 }],
  "failures": [{ "title": "…", "file": "tests/api/orders.spec.ts", "project": "ap
}
```

The counting decision that matters

A test that passed on retry is flaky, not passed TYPESCRIPT

```
case 'passed':
  if (result.retry > 0) this.counts.flaky += 1;
  else this.counts.passed += 1;
  break;
```

Playwright reports a test that passed on retry as `passed`. Counting those separately is what keeps a suite from quietly rotting behind a green tick — the flaky count is the number that tells you whether the suite is getting worse.

Failures carry their tags, so a `@smoke` failure can be spotted immediately, and their error is truncated to twelve lines — enough to triage, short enough to read.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A pipeline needs a field that is not here.
> - The console summary should say something else.

> **HOW TO CHANGE IT**
>
> 1. Add the field to the `Summary` interface and populate it in `onEnd`. Keep it diffable — a value that changes on every run is noise.
> 2. Add a per-test field by capturing it in `onTestEnd`, where the `TestCase` and `TestResult` are both available.

> **WHY THE CHANGE BELONGS HERE**
>
> A pipeline should not have to parse an HTML report or a JUnit XML to answer "did it pass, and what broke". One small JSON file answers it, and can gate a deploy.

> **WATCH OUT FOR**
>
> - **A reporter must never crash the run.** `onError` is deliberately empty — Playwright already reports the error itself.

- The output path is derived from `configFile`, not `config.rootDir`: `rootDir` resolves to the common ancestor of the projects' test directories, which is `tests/`, and would put the summary somewhere nobody looks.
- `--reporter=` on the command line replaces the whole config list, so a run with it produces no summary.

Related: `playwright.config.ts` · `.github/workflows/api-tests.yml`

API SURFACE (1 EXPORT)

CLASS **SummaryReporter** implements Reporter

```
onBegin(config: FullConfig, _suite: Suite): void
```

```
onTestEnd(test: TestCase, result: TestResult): void
```

```
onEnd(result: FullResult): void
```

```
onError(): void
```

Reporters must never crash the run, so failures here are swallowed.

Test data (src/data)

src/data/openapi.json

225 lines

An OpenAPI 3.0.3 document describing the `/objects` resource. Loaded automatically by the `contract` fixture — dropping a file at this path is all the wiring required.

CAREFUL restful-api.dev publishes **no** OpenAPI document, so this one was written by hand from observed behaviour. Against a team's published document, conformance testing detects drift between the API and its contract — the highest-value contract test there is. Against a document you wrote yourself, it detects drift between the API and *your understanding of it*, which is what a consumer-driven contract test does and is still worth having. Replace this file with your API's real document.

What it models

Six operations across two paths, with `CreatedObject` and `UpdatedObject` composed from a shared `Object` via `allOf` — because `createdAt` appears only on create and `updatedAt` only on updates, and a single schema requiring either would fail on every read.

The `Error` schema is documented as `{ error: string }` with a note that it is **not** RFC 9457 problem details. Recording a non-standard shape faithfully is more useful than documenting the shape you wish the API had.

WHEN YOU NEED TO CHANGE IT

- Your API publishes a document — replace this one.
- The API gains an operation.

HOW TO CHANGE IT

1. Bundle first; external `$ref`s are deliberately not resolved.

   ```
   npx @redocly/cli bundle openapi.yaml -o src/data/openapi.json
   ```

2. Update the operation count asserted in `tests/contract/objects.openapi.spec.ts` — it is asserted so a truncated document cannot make every other check pass by matching nothing.

WHY THE CHANGE BELONGS HERE

A document at a known path, picked up automatically, is what makes conformance testing cheap enough to actually adopt.

WATCH OUT FOR

- Only in-document `$ref`s are followed, by design — a second resolver behaving subtly differently from the team's own tooling would be worse than none.
- Deleting this file cleanly disables the OpenAPI suite, which skips rather than fails.

Related: `src/contracts/openapi.ts` · `tests/contract/objects.openapi.spec.ts` · `src/fixtures/api.fixture.ts`

src/data/README.md

20 lines

Documents what lives in `src/data/`, what belongs there and what does not. Everything in the folder is committed, so everything in it must be safe to make public.

BELONGS HERE	DOES NOT
A file some test actually reads	Anything generated per run — use `data.utils.ts`
A small real file, when a test reads it	Anything large — generate it at run time
The expected shape of an error	Anything secret — credentials come from the environment
An OpenAPI document, as `openapi.json`	

WHEN YOU NEED TO CHANGE IT

- You add a data file.

HOW TO CHANGE IT

1. Add the file in the same change as the test that reads it, in the smallest format that works — CSV beats JSON for a table a non-engineer will edit.
2. Add a row to the table in the README, naming what reads it.

> 3 Resolve it through `fromRoot` rather than a relative path, so the suite works from any working directory.

> **WHY THE CHANGE BELONGS HERE**
>
> A folder of unexplained data files becomes a folder nobody dares delete from. One table saying what each file is for keeps it maintainable.

Related: `src/utils/file.utils.ts` · `src/contracts/openapi.ts`

Reference: tests & repository docs

Where specs live and the conventions they follow

Tests folder

tests/api/client.behaviour.spec.ts 312 lines

Tests the framework itself against **https://httpbin.org**, a request-inspection service that echoes back exactly what it received. Fifteen tests proving the client puts on the wire what it claims to.

Why an echo server rather than a stub

> **RULE** A stub is written by the same person who wrote the client, so the two can share a misunderstanding and still agree. An echo server is neutral: it reports what genuinely arrived over a real connection, through a real TLS handshake, with real header normalisation applied.

This is not hypothetical. The `.text()` body encoding bug that this suite now guards against was invisible to the stub — the stub received exactly what the transport chose to send, so it agreed with the client's mistake. Only a neutral echo could show that a body meant to be malformed was arriving as valid JSON.

The regression test for that bug — TYPESCRIPT

```typescript
const raw = '{ deliberately not json';

const response = await http
  .withBaseUrl(PUBLIC_APIS.httpBin)
  .post('/post')
  .text(raw, 'application/json')
  .send();

expect(response.path<string>('data'), 'the body must arrive byte-for-byte').toBe(
expect(response.path('json')).toBeNull();   // httpbin could not parse it either
```

What the fifteen tests cover

AREA	PROVEN
Bodies	JSON, url-encoded forms, multipart with fields and files, raw text sent verbatim, binary

AREA	PROVEN
Query	Repeated parameters, comma-joined arrays, and `undefined` values dropped rather than sent as the string "undefined"
Encoding	Unicode, emoji, combining marks, right-to-left text and newlines survive a round trip intact
Credentials	Basic, bearer, and `anonymous()` proving no `Authorization` header is sent at all
Status	An unusual status (418) is reported rather than swallowed
Redirects	Followed by default; `noRedirect()` returns the 3xx so its `Location` can be asserted
Headers	Client defaults and per-request headers both arrive, with the request winning
Responses	gzip decoded transparently, binary available as bytes with an intact length

Several of these assert things the framework's own documentation claims. A claim in a comment is a hope; a claim with a test against a neutral server is a fact.

> **WHEN YOU NEED TO CHANGE IT**
>
> - You add a body encoding, a query format or a credential scheme.
> - A framework claim is worth pinning down.

> **HOW TO CHANGE IT**
>
> 1. Add a test that sends the thing and asserts on what httpbin echoed back. `/post` and `/get` echo the whole request; `/headers` echoes just the headers.
> 2. For a new status behaviour, `/status/{code}` answers with any code on demand.

> **WHY THE CHANGE BELONGS HERE**
>
> Every other suite tests an API *through* the framework. This one tests the framework, which is the only way to be sure a failure elsewhere is the API's fault and not the tooling's.

> **WATCH OUT FOR**
>
> - httpbin capitalises header names in its echo (`headers.Content-Type`), which is why the paths read that way.
> - It is a shared public service and occasionally slow; the assertions are about content, not timing.

Related: `src/core/request.builder.ts` · `src/core/http.client.ts` · `src/auth/static.auth.ts`

tests/api/objects.crud.spec.ts

306 lines

The headline suite: a full create-read-update-delete lifecycle against **https://api.restful-api.dev**, a live public API that genuinely persists writes. No stubs, no credentials.

Why this file is small

The API allows **50 anonymous requests per 24 hours**, then answers `405` with an explanatory body until the window resets. That single fact shapes the whole file: one full lifecycle plus four focused checks, about sixteen requests, and it **skips rather than fails** when the quota is gone.

> **RULE** A red build caused by somebody else having used the day's quota teaches nobody anything. The exhaustive coverage lives in `tests/contract/objects.stubbed.spec.ts`, which runs the same service against an executable model of the same contract — unlimited, offline, every run.

The division of labour

SUITE	PROVES	HOW OFTEN
This file	The API really behaves as we believe	A few times a day, within quota
`objects.stubbed.spec.ts`	Our client handles that behaviour correctly	Every single run

If the two disagree, either the API changed or we misread it — and either way that is exactly what a suite is for.

The quota probe

One probe per worker, not one per test — TYPESCRIPT

```typescript
let quotaProbe: Promise<boolean> | undefined;

test.beforeEach(async ({ api }) => {
  quotaProbe ??= api.objects.quotaAvailable();

  test.skip(
    !(await quotaProbe),
    'The restful-api.dev anonymous quota (50 requests / 24h) is exhausted. ' +
      'The same lifecycle is covered offline by tests/contract/objects.stubbed.sp
  );
});
```

The probe costs a request, so the **promise** is memoised at module scope rather than the result. Caching the promise means concurrent tests in the same worker share one in-flight request instead of racing to make several.

The five tests

TEST	WHAT IT PROVES
the full lifecycle `@smoke`	Create → read → replace → patch → delete, in order, on one object. The only test whose steps depend on each other, and the only one that proves the operations compose.
CREATE answers 200	The real status is 200, not the 201 a REST purist expects — and the response carries `createdAt`.
LIST filters by repeated ids	The `?id=1&id=2` encoding really filters. Encoded as `?id=1,2,3` the API ignores it and returns everything — a request that succeeds and filters nothing.
DELETE is not repeatable	200 with a message body, then 404. Which is why `remove()` tolerates 404 during cleanup.
a read matches its schema	Reads carry no timestamps at all, asserted through `fields()`.

Why the read after every write

Each mutation is followed by a read. Without it, a create test passes just as happily against an API that accepts the request and discards it — and a PATCH test never notices that the API silently cleared the fields it was not sent.

What makes it safe to run in parallel

- Every object is created by the test that uses it. Nothing depends on data somebody else left behind.
- Names come from `uniqueId`, so two workers cannot collide.
- `create` registers its own deletion, so the framework tidies up even when a test fails.
- The API never returns created objects from its list endpoint, so a test cannot be disturbed by what the rest of the internet is creating right now.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The API changes — this suite is what tells you.
> - You point the framework at your own API: replace the service and the schemas, and this structure carries over unchanged.
> - The quota changes, or you sign up for an account with a larger one.

> **HOW TO CHANGE IT**
>
> 1. To run everything that spends no quota:
>
>    ```
>    npx playwright test --grep-invert @live
>    ```
>
> 2. To spend quota deliberately, on just the lifecycle:
>
>    ```
>    npx playwright test --grep "@objects.*@smoke"
>    ```
>
> 3. When adapting to your own API, keep the shape: one composing lifecycle test, several focused ones. The lifecycle tells you *that* something broke; the focused tests tell you *what*.

> **WHY THE CHANGE BELONGS HERE**
>
> A CRUD suite is the first thing anyone writes against a new API and the thing most often written badly — a create that never reads back, a PATCH that never checks what survived, a delete that never confirms. This file is the reference for doing it properly against something real.

> **WATCH OUT FOR**
>
> - The quota is shared with everyone using the public API, so it can be gone before your first run of the day. That is what the skip is for.
> - `test.skip` inside `beforeEach` skips the individual test, not the file — which is what allows the probe to be awaited first.
> - The focused CREATE test creates outside the service, so it removes the object by hand. That is precisely the chore `create()` exists to remove.

Related: `src/services/object.service.ts` · `tests/contract/objects.stubbed.spec.ts` · `src/mocks/objects.stub.ts`

tests/api/posts.pagination.spec.ts

144 lines

Reads and pagination against **https://jsonplaceholder.typicode.com** — the API chosen because it has genuine RFC 8288 `Link` header pagination, a completely stable 100-record dataset, and no quota.

Why a second API

The CRUD target proves writes persist but has no pagination at all and a tight quota. This one has the opposite properties, so between them both halves get real coverage.

The assertion worth copying

A pagination walk must produce the dataset exactly once — TYPESCRIPT

```typescript
const everything = await api.posts.walkAllPages(10);

expect(everything).toHaveLength(PostService.TOTAL_POSTS);

const defects = findPaginationDefects([everything], (post) => String(post.id));
expect(defects.duplicates, 'no post may appear on two pages').toEqual([]);
expect(defects.uniqueItems).toBe(PostService.TOTAL_POSTS);
```

The two defects pagination tests exist to catch are an item that appears on two pages and one that appears on none. Both happen when the underlying data shifts mid-walk, and neither is visible from a single page — which is why a test that only reads page one passes while page two is broken.

What else it covers

- The `Link` header advertises `first`, `next` and `last`, and `response.nextPageUrl()` agrees with `parseLinkHeader`.
- `X-Total-Count` matches the length of the walk.
- Page size is honoured, the final page is short, and it carries no `next` — which is how offset-style walkers know to stop.
- Every item in a page validates against the schema, reported in one assertion rather than stopping at the first bad item.
- Reads declare a `Cache-Control` header — an omission only an API test ever notices.

> **NOTE** This API *simulates* writes: a POST answers 201 with a plausible body and stores nothing. That is why the service's method is called `simulateCreate`, and why the lifecycle suite runs elsewhere. A method named `create` that does not create is a trap for the next person.

> **WHEN YOU NEED TO CHANGE IT**
>
> - Your API paginates differently — swap the walker.
> - The dataset size changes.

> **HOW TO CHANGE IT**
>
> 1. For `Link`-header pagination use `followLinkHeader`; for offset/limit use `followOffset`. Both walkers take the same shape of callback.
> 2. Assert the total from a constant, as `PostService.TOTAL_POSTS` does, so a change is one edit.

> **WHY THE CHANGE BELONGS HERE**
>
> Pagination is where "the endpoint works" and "the endpoint works at scale" diverge, and it is almost never covered because walking pages by hand in a test is tedious. Making it one line removes the excuse.

> **WATCH OUT FOR**
>
> - The dataset is stable, which is what allows exact counts. Against changing data, assert invariants — no duplicates, no gaps — rather than totals.
> - The walkers have a hard page ceiling, so a server that always advertises a next page produces a clear failure rather than a hang.

Related: `src/services/post.service.ts` · `src/utils/pagination.utils.ts` · `src/utils/header.utils.ts`

`tests/contract/objects.openapi.spec.ts` 274 lines

Validates every response in a lifecycle against the OpenAPI document in `src/data/openapi.json`, proves the check has teeth, and reports the contract-coverage gap.

What this adds over the Zod schemas

The stubbed suite validates against schemas the team wrote — what the test author *believed*. An OpenAPI document says what the API *promises*, and checking against it catches the one thing hand-written schemas never do: the API and its published contract have drifted, and every consumer who trusted the document is broken.

> **CAREFUL** restful-api.dev publishes no OpenAPI document, so the one in `src/data/` was written by hand from observed behaviour. Against a team's **published** document this suite detects drift between the API and its contract — the highest-value contract test there is. Against a document you wrote yourself it detects drift between the API and *your understanding of it*, which is what a consumer-driven contract test does and is still worth having. Know which you have.

The test that matters most

Proving the check can fail — TYPESCRIPT

```ts
mockServer.stub({
  method: 'GET',
  path: '/objects/*',
  /* `id` must be a string and is required; here it is a number. */
  respond: { status: 200, json: { id: 42, name: 'wrong types' } },
});

const result = documented(contract).validate('GET', '…/objects/1', 200, response.

expect(result.valid).toBe(false);
expect(result.errors.join(' ')).toContain('id');
```

A conformance suite that has never failed is a conformance suite nobody should trust. This is the test that shows the machinery actually rejects something.

Coverage

Which documented operations did the run never touch? — TYPESCRIPT

```ts
const gaps = documented(contract).uncovered(exercised);
expect(gaps.map((o) => o.operationId)).toEqual(
  expect.arrayContaining(['replaceObject', 'updateObject', 'deleteObject']),
);
```

That number is worth tracking over time. An endpoint nobody tests is an endpoint nobody notices breaking.

The narrowing helper

Why not just use `contract!` TYPESCRIPT

```ts
function documented(contract: OpenApiContract | undefined): OpenApiContract {
  if (!contract) throw new Error('No OpenAPI document loaded from src/data/openap
  return contract;
}
```

The fixture is optional because a project may ship no document, and `beforeEach` skips the suite when it is absent — but the compiler cannot see that a skip happened. Making the requirement explicit means a missing document fails with a sentence rather than a null-property error.

> **WHEN YOU NEED TO CHANGE IT**
>
> - Your API publishes an OpenAPI document — replace `src/data/openapi.json` with it.
> - The document gains an operation.

> **HOW TO CHANGE IT**
>
> 1. Bundle the specification first; external `$ref`s are deliberately not resolved.
>
>    ```
>    npx @redocly/cli bundle openapi.yaml -o src/data/openapi.json
>    ```
>
> 2. Nothing else needs wiring — the `contract` fixture picks the file up automatically.
> 3. Update the operation count in the first test; it is asserted so that a truncated document cannot make every other check pass by matching nothing.

> **WHY THE CHANGE BELONGS HERE**
>
> Contract conformance is the one check that catches a breaking change *before* a consumer does. Making it cheap — drop a document in `src/data/` — is what makes it get used.

> **WATCH OUT FOR**
>
> - All but one test runs against the stub, so the suite stays offline. The single `@live` test is the only one that spends quota, and it skips when the quota is gone.
> - The document models `CreatedObject` and `Object` separately, because `createdAt` appears only on create. A single schema would fail on every read.

Related: `src/data/openapi.json` · `src/contracts/openapi.ts` · `src/fixtures/api.fixture.ts`

tests/contract/objects.stubbed.spec.ts

319 lines

The same `ObjectService`, the same lifecycle, run against an in-memory model of the same contract. Unlimited, offline, deterministic — and able to produce failure modes no real API will give you on demand.

What it proves, and what it cannot

It proves the **client half** of the contract: that the service builds the right requests, sends the right bodies, and correctly interprets every response including the quirks. It cannot prove the API still behaves that way — only the live suite can do that. Together they cover both halves.

Twelve tests, in four groups

GROUP	COVERS
Lifecycle	Create → read → replace → patch → delete, with the PUT/PATCH distinction asserted explicitly.
Request shape	What actually went on the wire: content type, idempotency key, body, and the repeated-id query encoding.
Failure modes	404 and its error shape, a repeated delete, a malformed body, and the API's complete absence of validation.
Client resilience	Retry on a transient failure, **no** retry on a verdict, and a timeout — none of which a real API will produce to order.

The resilience test that a stub makes possible — TYPESCRIPT

```typescript
mockServer.flaky('/objects/*', 2, { id: '1', name: 'recovered', data: null });

const response = await http.withBaseUrl(mockServer.url).get('/objects/1').retries

response.expectOk().expectPath('name', 'recovered');
expect(response.timing.attempts).toBe(3);
```

The endpoint really does recover, so a client that gives up too early fails and one that retries forever never finishes. A mock that always succeeds would prove neither.

And its opposite — the assertion that verdicts are never retried — TYPESCRIPT

```typescript
mockServer.fail('/objects/*', 404, { error: 'Object with id=x was not found.' });

const response = await http.withBaseUrl(mockServer.url).get('/objects/x').retries

expect(response).toHaveStatus(404);
expect(response.timing.attempts).toBe(1);
```

NOTE That second assertion guards the structural decision in `http.client.ts`: only the transport call sits inside the retry `try`. A 404 is an answer, not a fault, and retrying it would be three identical failures and a slower suite.

Test isolation

The stub server is worker-scoped, so `beforeEach` calls `mockServer.reset()` and re-registers the contract. Without that, one test's writes would be visible to the next and the suite would become order-dependent — the failure mode that makes a suite pass locally at one worker and fail in CI at eight.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The live API changes and the model must follow.
> - A new failure mode is worth covering.
> - The service gains a method.

> **HOW TO CHANGE IT**
>
> 1. Change the model in `src/mocks/objects.stub.ts`, then run the live suite within quota to confirm the two still agree.
> 2. Add a failure-mode test using the stub server's own helpers — `fail`, `slow`, `flaky` — rather than a new stub definition.

> **WHY THE CHANGE BELONGS HERE**
>
> Running the same service object against both a real API and a model of it is what makes the model trustworthy. A stub nobody checks against reality is a fixture that encodes somebody's assumptions and then defends them forever.

> **WATCH OUT FOR**
>
> - The stub server collapses repeated query parameters, so the model parses `request.url` directly to see the repeated `id` form.
> - Every test builds its service with `http.withBaseUrl(...)` — a *derived* client, which keeps latency collection, recording and the contract guard attached. A freshly constructed `HttpClient` would silently lose all three.

Related: `src/mocks/objects.stub.ts` · `tests/api/objects.crud.spec.ts` · `src/core/http.client.ts`

`tests/performance/posts.latency.spec.ts` 113 lines

Latency budgets and percentiles, measured against a real endpoint under the `performance` project's single worker.

Why the project settings matter

- **One worker**, because a p95 measured while eight workers hammer the same endpoint describes the load the suite is generating, not the endpoint.
- **No retries**, because a retried timing measurement is not a measurement.

Warm-up runs are discarded TYPESCRIPT

```typescript
const summary = await sample(() => api.posts.find(1), { runs: 20, warmup: 3 });

expect(summary.p95).toBeLessThan(5_000);
expect(summary.p99).toBeLessThan(Math.max(summary.p50 * 20, 8_000));
```

The first call to a cold endpoint measures TLS setup and JIT rather than the endpoint, and including it drags every percentile upward for no reason. The p99-versus-p50 check is the tail test: a p99 many times the median means an unstable endpoint even when the average looks fine, and the tail is what users actually feel.

The whole distribution is printed on every run, so a failure shows the shape rather than only the number that broke — a p95 with no p50 beside it is hard to act on.

A pure unit check, deliberately

One test exercises `percentile` and `summarize` on a fixed array. It costs nothing, and it means a failing budget elsewhere can never be blamed on the arithmetic.

CAREFUL This is correctness-adjacent performance signal, free because the requests are being made anyway. It is **not** a load test — nothing here generates concurrency. Treating it as one would be worse than having no load test, because it would feel like coverage.

WHEN YOU NEED TO CHANGE IT

- A budget is systematically wrong.
- A route needs its own percentile check.

HOW TO CHANGE IT

1. Set the environment-wide default in `latencyBudgetMs` or `LATENCY_BUDGET_MS`, and use `expectWithinLatencyBudget()` rather than a literal.
2. For a specific route, use `sample()` with enough runs that the percentile means something — twenty is a reasonable floor.

WHY THE CHANGE BELONGS HERE

An API suite is the cheapest place a team ever gets performance signal, because the timings are already there. What is usually missing is the discipline to look at a distribution rather than a single number.

WATCH OUT FOR

- Percentiles are nearest-rank, so a reported p95 is always a value that was actually observed — which is what makes it defensible in a conversation with the team that owns the service.
- The latency fixture attaches a per-route report to every test in every project, not just this one.

Related: `src/utils/performance.utils.ts` · `playwright.config.ts` · `src/config/timeouts.ts`

tests/README.md

103 lines

Where each kind of test belongs, and the conventions the tooling already enforces. The folder ships without specs: the framework was delivered as architecture, and the tests belong to the team that owns the API.

FOLDER	PROJECT	CONTAINS
`tests/api/`	`api`	Functional REST, GraphQL, streaming and file transfer. The bulk of the suite.
`tests/contract/`	`contract`	Schema and OpenAPI conformance. Runs offline against the stub server.
`tests/performance/`	`performance`	Latency budgets. One worker, so measurements are not self-inflicted.
`tests/security/`	`security`	Authorisation matrices, input handling, response hygiene. **Never point this at production.**

File naming is `<resource>.<behaviour>.spec.ts` — `users.create.spec.ts`, `orders.pagination.spec.ts`, `auth.token-refresh.spec.ts` — so the name makes obvious which service object the test exercises.

RULE Import `test` and `expect` from `../../src/fixtures`, never from `@playwright/test`. The Playwright ones have no cleanup registry, no contract guard and none of the custom matchers, and nothing will tell you.

WHEN YOU NEED TO CHANGE IT

- A new category of test needs a home.
- A convention changes.

HOW TO CHANGE IT

1. Add the folder, add a project in `playwright.config.ts`, add a script in `package.json`, and add a row to the table here.

WHY THE CHANGE BELONGS HERE

A tests folder with no stated conventions grows four different styles within a month. Writing them down where somebody will look — beside the tests — is what keeps the suite reviewable.

WATCH OUT FOR

- `playwright test` reports that it found no tests until you write one. That is expected.
- A test in the wrong folder gets the wrong project settings — a performance test under `tests/api/` runs under eight parallel workers and measures the load, not the endpoint.

Related: `src/fixtures/index.ts` · `playwright.config.ts`

`tests/security/response.hygiene.spec.ts` 250 lines

Response hygiene, injection transport, CORS and authorisation, against real services. Six tests that cost almost nothing because the requests are being made anyway.

Three kinds of assertion, and why the difference is stated per test

The public services used here are demonstration APIs and do not meet a production security baseline. Pretending otherwise would produce failing tests that people learn to ignore — and an ignored security test is worse than none. So each test does one of three things, and says which:

KIND	USED FOR	EXAMPLE
Assert	What must be true of any correct API	No stack trace, connection string or credential in an error body
Report	A baseline these demo services legitimately do not meet	Missing `Strict-Transport-Security`, printed rather than failed
Assert the detector fires	Where a real service genuinely exhibits the problem	httpbin's CORS origin reflection

A real finding, from a real service

`httpbin really does reflect any origin AND allow credentials` TYPESCRIPT

```typescript
expect(response.header('access-control-allow-origin')).toBe('https://evil.example
expect(response.header('access-control-allow-credentials')).toBe('true');

const rules = findings.map((finding) => finding.rule);
expect(rules).toContain('cors-origin-reflection');
```

On a production API that combination is serious: any site a victim visits could read authenticated responses. httpbin does it deliberately, because being callable from anywhere is the point of an echo service. So rather than pretending the finding is absent, the test asserts the auditor **correctly detects it against a real server** — a true positive is better evidence that a detector works than a clean run against a fixture. Against your own API, invert the assertion.

The injection test is about transport, not vulnerability

httpbin is an echo service; nobody is claiming it is vulnerable. The point is that the *framework* carries hostile input unchanged, so when the same pattern is pointed at a real API the payload that arrives is the payload that was written. A framework that escaped or mangled these would make every injection test meaningless.

NOTE An earlier version of that test searched the whole response body for `49` — the result of `{{7*7}}` had the template been evaluated. It failed intermittently, because httpbin echoes numeric headers such as `Content-Length` and the substring appeared for unrelated reasons. Asserting on the specific field instead is both stricter and stable. It is a good example of why a flaky test deserves a diagnosis rather than a retry.

The authorisation matrix

Generated, not hand-written TYPESCRIPT

```typescript
const matrix = buildAccessMatrix(
  ['valid', 'invalid'],
  [{ method: 'GET', path: '/basic-auth/framework/secret', allowed: ['valid'] }],
  { deniedStatus: 401 },
);

for (const cell of matrix) {
  const response = await http.withBaseUrl(PUBLIC_APIS.httpBin)
    .get('/basic-auth/framework/secret')
    .basic('framework', passwordFor[cell.role] ?? '')
    .expectStatus(200, 401)
    .send();

  expect(judgeAccess(cell, response.status)).toBeUndefined();
}
```

Authorisation defects are defects of *omission* — the endpoint nobody remembered to protect. Writing the cases somebody thought of catches exactly the cases somebody thought of; generating the grid is the only way to catch the class. The credential is looked up from a table rather than chosen in an `if`, so the loop body is a straight line through every cell.

Redaction

One test sends a recognisable token and asserts it does not survive into `toRecord()` — which is what the logger, the reporters and the exchange recorder all consume. If a credential got through, every recording committed to a repository would contain a live token.

> **WHEN YOU NEED TO CHANGE IT**
>
> - A disclosure pattern is missing.
> - You point this at your own API — then invert the "report" assertions.

> **HOW TO CHANGE IT**
>
> 1 Against your own API, change the reporting assertions to failures.
>
> ```
> expect(findings, formatFindings(findings)).toEqual([]);
> ```

2 Add a leak pattern in `src/utils/security.utils.ts`, not here, so every suite gets it.

WHY THE CHANGE BELONGS HERE

These checks catch a class of defect that functional tests never look at, and they are nearly free. Reporting rather than throwing is what lets one test cover a whole surface without becoming noise.

WATCH OUT FOR

- **Never point the `security` project at production.** The payloads are deliberately hostile.
- The header baseline is opinionated. Loosen it deliberately rather than deleting the test.

Related: `src/utils/security.utils.ts` · `src/utils/data.utils.ts` · `playwright.config.ts`

Repository documentation

docs/README.md

42 lines

Explains what is in `docs/`: which files are generated, how to read them, how to rebuild them, and where to edit each part.

Only the PDF is committed. The HTML site and the extracted API surface are rebuilt on demand, because a generated site in version control produces a diff on every build that tells nobody anything.

WHEN YOU NEED TO CHANGE IT

- The generator gains an output.
- The editing table stops matching the content modules.

HOW TO CHANGE IT

1 Update the tables. Keep it short — it is a signpost, not documentation of the documentation.

WHY THE CHANGE BELONGS HERE

Somebody who opens `docs/` and sees generated HTML needs to be told, immediately, not to edit it. This is that notice.

Related: `scripts/docs/build-docs.mjs` · `scripts/docs/content/`

README.md

127 lines

The repository's front door: what the framework is, how to start it, what is in it, and where the full documentation lives. Written for somebody who has just cloned it and has five minutes.

> **WHEN YOU NEED TO CHANGE IT**
>
> - The quick-start steps change.
> - A capability is added that belongs in the summary table.
> - The security rules change.

> **HOW TO CHANGE IT**
>
> 1. Keep it short. Depth belongs in this generated documentation; the README's job is to get somebody running and then point them here.
> 2. Update the capability table if you added a layer.

> **WHY THE CHANGE BELONGS HERE**
>
> It is the only documentation some people will read, and the first documentation everybody reads. It should answer "what is this and how do I run it" before anything else.

Related: `docs/` · `tests/README.md`

Playbooks

I need to change something — where do I go, what do I do, and why there?

Each entry below is a task somebody will need to do, with **where to go, what to do, and why it belongs there**. They are ordered roughly by how often they come up.

Run the suite without spending API quota

Where: the command line. The CRUD target allows 50 anonymous requests per 24 hours, and every test that spends them is tagged `@live`.

Three levels of thrift — SHELL

```shell
npx playwright test --grep-invert @live   # everything else — free and unlimited
npx playwright test --project=contract    # offline: no network at all
npx playwright test --grep @live          # deliberately spend quota
```

Why there: the tag rather than a config flag, because the distinction is a property of the *test*, not of the run. A pipeline can then choose per job — the pull-request job runs `--grep-invert @live`, and a nightly job spends the quota once.

> **NOTE** When the quota is gone the live tests skip with a reason rather than failing. That is deliberate: a red build caused by somebody else's usage teaches nobody anything, and a suite people learn to ignore is worse than no suite.

Point the CRUD suite at your own API

1. Add your environment to `src/config/environments.ts`. URLs, prefix, latency budget and the read-only flag — no secrets.

```ts
sandbox: {
  name: 'sandbox',
  apiBaseUrl: 'https://api.sandbox.example.com',
  apiPrefix: '/v2',
  verifyTls: true,
  latencyBudgetMs: 3000,
  readOnly: false,
},
```

2. Copy `src/services/object.service.ts`. Declare the schema first and infer the type from it, so the runtime check and the compile-time type cannot disagree.

```ts
export const OrderSchema = z.object({
  id: identifier,
  status: z.enum(['draft', 'paid', 'shipped']),
  createdAt: isoDateTime,
}).passthrough();
```

```
export type Order = z.infer<typeof OrderSchema>;
```

3. **Establish what the API really does by calling it**, not by reading its documentation. Every quirk in the reference service — a 200 where 201 was expected, timestamps on some responses and not others, a non-standard error shape — was found that way, and each would have broken a schema copied from a specification.

4. Register the schemas so the contract guard can find them from a real request.

```
registerSchemas([
  { name: 'order', method: 'GET', pathPattern: '/orders/{id}', status: '2xx', schem
]);
```

5. Add the service to the `api` fixture in `src/fixtures/api.fixture.ts` — one line, and every test can reach it.

6. Copy the shape of `tests/api/objects.crud.spec.ts`: one composing lifecycle test, several focused ones.

Why that shape: the lifecycle test proves the operations compose and tells you *that* something broke; the focused tests each assert one behaviour and tell you *what*. Either alone leaves a real gap.

Model an API you cannot call freely

Where: a new module in `src/mocks/`, following `objects.stub.ts`. Worth doing whenever the real dependency is rate-limited, shared, flaky, or simply down half the time.

Register the contract on the stub server, then point a derived client at it — TYPESCRIPT

```
export function stubOrdersApi(server: MockServer) {
  const store = new Map<string, StoredOrder>();

  server.stub({
    method: 'POST',
    path: '/orders',
    respond: (request) => { /* … model what the API really does … */ },
  });

  return { store, reset: () => store.clear() };
}
```

And drive it with the same service object TYPESCRIPT

```typescript
test.beforeEach(({ mockServer }) => {
  mockServer.reset();
  stubOrdersApi(mockServer);
});

const orders = new OrderService(http.withBaseUrl(mockServer.url), { cleanup });
```

- **Use `withBaseUrl`, not a new client.** Deriving keeps the run's observers — latency, recording, contract guard — attached.
- **Reset in `beforeEach`.** The stub server is worker-scoped, so without a reset one test's writes are visible to the next and the suite becomes order-dependent.
- **Model the quirks, not the ideal.** A stub that behaves the way you *wish* the API behaved tests nothing.
- **Keep a live suite.** A model nobody checks against reality is a fixture that encodes an assumption and then defends it forever.

RULE The two suites must drive **the same service object**. That is what makes the arrangement work: the stub is not a parallel implementation to keep in step, it is a server the real client talks to.

Point the suite at a different environment

Where: `.env`, or `src/config/environments.ts` for a new one.

Switch for one run SHELL

```shell
TEST_ENV=dev npx playwright test
```

To add an environment that does not exist yet, add an entry to `ENVIRONMENTS`. It is `as const satisfies Record<string, EnvironmentDefinition>`, so a missing field is a compile error rather than an `undefined` at run time, and the new key becomes a valid `TEST_ENV` automatically because the Zod enum is built from the same object.

Why there: the environment table holds only non-secret values — URLs, prefixes, budgets, the read-only flag — so it is safe in a pull request. Anything credential-shaped is read from the process environment instead.

Add an environment variable

1. Add it to `.env.example` **with a comment explaining what it does**. This file is the only documentation most people will read.
2. Add it to the schema in `src/config/env.config.ts`, using one of the coercion helpers so a blank value falls back rather than failing.

```
  MAX_UPLOAD_MB: integerish(25),
  FEATURE_FLAGS: optionalString(),
```

3 Expose it on the frozen `config` object with a name that reads well at the call site.

```
maxUploadMb: raw.MAX_UPLOAD_MB,
```

4 If CI needs it, add it to the `env:` block in `.github/workflows/api-tests.yml`.

Why there: `env.config.ts` is the only file that touches `process.env`. That makes secret handling reviewable in one place, and turns a misconfiguration into one clear message at start-up instead of a confusing `undefined` deep inside a request.

Add a service object for a new resource

1 Copy `src/services/template.service.ts`. It is written to be copied — every question that comes up on the second service object is answered in it.

2 Declare the schema first and infer the type from it, so the runtime check and the compile-time type cannot disagree.

```
export const OrderSchema = z.object({
  id: identifier,
  reference: z.string(),
  status: z.enum(['draft', 'paid', 'shipped']),
  createdAt: isoDateTime,
});

export type Order = z.infer<typeof OrderSchema>;
```

3 Register the schema so the contract guard and coverage reporting can find it.

```
registerSchemas([
  { name: 'order', method: 'GET', pathPattern: '/orders/{id}', status: '2xx', schema
]);
```

4 Set `basePath` and write intention-revealing methods. A creation registers its own cleanup in the same statement.

```
export class OrderService extends BaseService {
  protected readonly basePath = '/orders';

  async create(overrides: Partial<Order> = {}): Promise<Order> {
    const response = await this.post().json(buildInvoice(faker, overrides)).expectS
    const order = response.parse(OrderSchema, 'order');
    return this.track(order, `order ${order.id}`, () => this.remove(order.id));
  }
}
```

5 Add it to the `api` fixture in `src/fixtures/api.fixture.ts`, which makes it available to every test.

```
api: async ({ http, cleanup, log }, use) => {
  await use({
```

```
      users: new UserService(http, { cleanup, logger: log.child('users') }),
      orders: new OrderService(http, { cleanup, logger: log.child('orders') }),
    });
  },
```

Why there: the service is the only thing that knows the endpoint's path and payload shape. Spread that knowledge across tests and a rename becomes a repository-wide search instead of a one-line edit.

Add an authentication scheme

Where: a new file in `src/auth/`, implementing `AuthProvider`.

The whole interface — TYPESCRIPT

```typescript
export class MutualTlsAuth implements AuthProvider {
  readonly name = 'mtls';

  async headers(): Promise<HeaderMap> {
    return { 'x-client-cert-fingerprint': await this.fingerprint() };
  }

  invalidate(): void {
    this.cached = undefined;
  }
}
```

If it should be chosen automatically, add it to `defaultAuthProvider` in `src/fixtures/api.fixture.ts` — the order in that function is the precedence.

Why there: `headers()` is called per request rather than once, which is what allows a provider to refresh a token transparently. Anything that needs the request itself — request signing — is a signer rather than a provider; see `HmacSigner`.

Add a custom assertion

Two homes, and the choice matters.

PUT IT IN…	WHEN
`ApiResponse` (`src/core/api.response.ts`)	It is about a response, and reads better chained: `response.expectOk().expectSecurityHeaders()`
`custom-matchers.ts`	It needs to be negatable, soft, or applied to something that is not a response: `expect(payload).toMatchPayload(baseline)`

A matcher — the return type is what gives it its types at every call site TYPESCRIPT

```typescript
toHaveCursor(response: ApiResponse, expected?: string): MatcherResult {
  const cursor = response.path<string>('nextCursor');
  const pass = expected === undefined ? cursor !== undefined : cursor === expected;
  return {
    pass,
    message: () =>
      pass
        ? `Expected no cursor, found "${cursor}".`
        : `Expected cursor ${expected ?? '(any)'}, found ${cursor ?? '(none)'}.${cont
  };
}
```

Why there: matchers are exported through `expect.extend`, which returns a *typed* `expect`. Adding one to that object is all that is needed — there is no separate type declaration to keep in step.

Stub a dependency you cannot control

Where: the `mockServer` fixture, in the test itself.

Failure, latency and recovery, all on demand TYPESCRIPT

```typescript
test('retries a flaky dependency', async ({ http, mockServer }) => {
  mockServer.reset();
  mockServer.flaky('/payments', 2, { status: 'settled' });

  const response = await http.withBaseUrl(mockServer.url).post('/payments').retries(3

  response.expectOk();
  expect(response.timing.attempts).toBe(3);
  expect(mockServer.callCount('/payments')).toBe(3);
});
```

Why there: the stub proves the client really recovers, because the endpoint really does. A mocked-out client that always succeeds proves nothing at all.

Make a test wait for eventual consistency

Where: Playwright's `expect.poll()`. Never a sleep.

Poll for the thing you are actually waiting for TYPESCRIPT

```typescript
await expect
  .poll(() => api.orders.find(id), {
    message: `order ${id} never appeared in the index`,
    timeout: TIMEOUTS.POLL_TIMEOUT,
  })
  .toBeDefined();
```

RULE A fixed sleep is too short on a slow day and too long on every other day, and it hides the fact that the wait exists at all. `expect.poll()` states the condition, retries it until the assertion passes, and fails with a message naming what never happened.

Change a timeout

Where: `src/config/timeouts.ts` for a budget the whole suite shares; `.timeout(ms)` on the builder for one request.

Why there: named budgets mean a slow environment is retuned in one edit, and every number in the suite carries an explanation of what it is waiting for. A raw `30000` in a test tells the next reader nothing.

CAREFUL Raising a timeout to make a flaky test pass converts a fast failure into a slow one. Find out what is actually slow first — the latency collector attaches a per-route report to every test.

Validate against a published OpenAPI document

1. Bundle the specification into a single file — external `$ref`s are deliberately not resolved — and save it as `src/data/openapi.json`.

   ```
   npx @redocly/cli bundle openapi.yaml -o src/data/openapi.json
   ```

2. The `contract` fixture picks it up automatically; no wiring is needed.

3. Assert conformance with the matcher.

   ```typescript
   test('matches the published contract', async ({ http, contract }) => {
     test.skip(!contract, 'No OpenAPI document in src/data/');
     const response = await http.get('/users/1').send();
     expect(response).toSatisfyContract(contract);
   });
   ```

4. For coverage, ask which documented operations the suite never exercised.

   ```typescript
   const gaps = contract.uncovered(exercisedCalls);
   ```

Why there: a hand-written schema records what the test author believed. The document records what the API *promised*. Only the second catches the case where the API and its published contract have drifted and every consumer who trusted the document is broken.

Add a latency budget

Two levels TYPESCRIPT

```typescript
// Per request, in any test:
response.expectFasterThan(400);

// Per route, across many samples, in tests/performance/:
const summary = await sample(() => api.users.list(), { runs: 30, warmup: 3 });
expect(summary.p95).toBeLessThan(800);
```

The environment-wide default lives in `latencyBudgetMs` on the environment, or `LATENCY_BUDGET_MS`; `expectWithinLatencyBudget()` uses it.

Why there: percentile checks belong in the `performance` project because it runs with a single worker. Measured under eight parallel workers, a p95 describes the load the suite generates, not the endpoint.

Test authorisation properly

Generate the grid rather than the cases you thought of TYPESCRIPT

```typescript
const matrix = buildAccessMatrix(
  ['admin', 'standard', 'readonly'],
  [
    { method: 'GET', path: '/users/{id}', allowed: ['admin', 'standard', 'readonly']
    { method: 'DELETE', path: '/users/{id}', allowed: ['admin'] },
  ],
);

for (const cell of matrix) {
  test(`${cell.role} ${cell.method} ${cell.path} @security`, async ({ httpAs }) => {
    const response = await httpAs(cell.role).request(cell.method, cell.path).param('i
    expect(judgeAccess(cell, response.status), formatFindings([...])).toBeUndefined()
  });
}
```

Why there: authorisation defects are defects of *omission* — the endpoint nobody remembered to protect. Generating the whole grid is the only way to catch that class reliably; writing the cases somebody thought of catches exactly the cases somebody thought of.

Record a run and replay it offline

Record once, then run with no network — TYPESCRIPT

```
# Record against a real environment
TEST_ENV=staging npx playwright test --project=api

# The recorder saves failures automatically; save everything by hand:
recorder.save('src/data/recordings/users.json');
```

Replay through the stub server — TYPESCRIPT

```
const recording = ExchangeRecorder.load('src/data/recordings/users.json');
for (const stub of ExchangeRecorder.toStubs(recording)) mockServer.stub(stub);
```

CAREFUL Recordings are redacted on the way out, but check one before committing it. A recording containing a live token is a credential in the repository, and no amount of later deletion fixes that.

Add a project to the run

Where: the `projects` array in `playwright.config.ts`, plus a folder under `tests/` and a script in `package.json`.

A smoke project that runs before a deploy — TYPESCRIPT

```
{
  name: 'smoke',
  testDir: './tests/api',
  grep: /@smoke/,
  dependencies: ['setup'],
  retries: 0,
  workers: 4,
},
```

Why there: a project is the unit that carries its own settings and its place in the dependency order. Expressing "log in first" as a dependency rather than a global variable is what keeps it visible in the report and retryable like anything else.

Add a reporter

Where: the `reporter` array in `playwright.config.ts`; a custom one goes in `src/reporters/`.

Make anything heavy conditional, the way Allure and the blob reporter already are — a reporter that runs on every local run is a tax on every local run.

> **NOTE** A reporter passed on the command line **replaces** the list in the config. `--reporter=list` is useful for a quick loop, but it also means no `summary.json`, no JUnit and no HTML report for that run.

Change how a failure reports

Where: the private `assert` method in `src/core/api.response.ts`. Every built-in assertion funnels through it, so one edit changes the anatomy of every failure message in the suite.

Why there: consistency is the point. A reader who has seen one failure should be able to read every failure without re-orienting.

Support a new body encoding

1. Add the kind to `BodyKind` in `src/types/index.ts`. The compiler will now list every place that has to handle it — three exhaustive switches.
2. Add the builder method in `src/core/request.builder.ts`, calling `assertBodyUnset` so two bodies cannot be set at once.
3. Add the transport case in `HttpClient.transport`.
4. Add the log rendering in `describeRequestBody` in `src/core/api.response.ts`.

Why there: `BodyKind` is a discriminated union precisely so this is a compiler-guided change rather than a search. Nothing is missed because nothing *can* be missed.

Add a fixture

Where: `src/fixtures/api.fixture.ts` — declare the type in `ApiFixtures` or `WorkerFixtures`, then implement it.

Set up before, clean up after; `use` is the boundary — TYPESCRIPT

```typescript
tenant: async ({ http }, use) => {
  const created = await http.post('/tenants').json({ name: uniqueId('tenant') }).send
  const id = created.path<string>('id');

  await use(id);                                  // the test runs here

  await http.delete(`/tenants/${id}`).send();     // runs even if the test failed
},
```

CAREFUL A fixture that depends on another fixture that depends back on it deadlocks with an unhelpful error. If two need each other, split the shared part into a third — `createConsoleCapture`-style — rather than trying to break the cycle in place.

Debug one failing test

Escalating detail — TYPESCRIPT

```typescript
# Everything the framework knows
LOG_LEVEL=debug LOG_BODIES=true npx playwright test path/to/spec.ts --workers=1

# Replay the run in the trace viewer
npx playwright show-trace test-results/<test-folder>/trace.zip

# Step through in a debugger — the VS Code launch config does this too
PWDEBUG=1 npx playwright test path/to/spec.ts --debug
```

A failed test also leaves `reports/recordings/<test>.json`, containing every exchange with credentials redacted. That is usually enough to diagnose without re-running anything.

Run against a local service

SHELL

```shell
TEST_ENV=local npx playwright test
# or override just the URL
API_BASE_URL=http://localhost:8080 npx playwright test
```

The `local` environment sets `verifyTls: false`, which is the one place a self-signed certificate is acceptable.

Add a page to this documentation

1. Add the prose to `scripts/docs/content/guides.mjs` as an exported array of blocks.
2. Add an entry to `PAGES` in `scripts/docs/build-docs.mjs`.
3. Rebuild.

   ```
   npm run docs
   ```

Documenting a **new source file** is not optional: the build fails with the path and refuses to produce a site until an entry exists in `scripts/docs/content/`. That gate is the reason this document can be trusted.

Rotate a credential

1. Update the repository secret in GitHub — never a file in the repository.
2. Update your own `.env`, which is git-ignored.
3. Delete `storage/*.json`. Those hold captured sessions and tokens for the *old* credential and will keep being replayed until they are removed.
4. Re-run. `auth.setup.ts` captures a new session on the next run.

> **CAREFUL** A `storage/` file is a live credential. Committing one is equivalent to committing a password, which is why the whole directory is git-ignored and the files are written with mode `0600`.

Conventions

Rules, naming, tags, anti-patterns and the review checklist

Rules the codebase already follows. They are worth stating because most of them are enforced by tooling, and a reviewer should know which is which.

Non-negotiables

RULE **Import `test` and `expect` from `@fixtures`, never from `@playwright/test`.** Importing Playwright directly gives you a `test` with no cleanup registry, no contract guard and none of the custom matchers — and nothing will tell you. This is the single most expensive mistake available in the repository.

RULE **Never read `process.env` outside `src/config/env.config.ts`.** One reader is what makes secret handling reviewable and turns a misconfiguration into one clear message.

RULE **Never sleep.** Use `waitFor`. A fixed delay is too short on a slow day, too long on every other, and silent about what it is waiting for.

RULE **Never retry an assertion.** `retry()` is for transient work — a flaky sandbox, a service restarting. Retrying until a test passes is how a real defect reaches production.

RULE **Never commit a credential.** `.env.example` is a template with placeholders. `.env*` and `storage/*.json` are git-ignored. CI reads secrets from the repository's secret store.

RULE **Register cleanup at the point of creation.** In the same statement, in the service method. A resource created without a registered deletion is a leak that somebody else finds months later.

Naming

THING	CONVENTION	EXAMPLE
Spec file	`<resource>.<behaviour>.spec.ts`	`users.pagination.spec.ts`
Service object	`<Resource>Service`, in `<resource>.service.ts`	`OrderService` in `order.service.ts`
Schema	PascalCase, suffixed `Schema`	`OrderSchema`

THING	CONVENTION	EXAMPLE
Type from a schema	PascalCase, inferred	`type Order = z.infer<typeof OrderSchema>`
Auth provider	`<Scheme>Auth`, in `<scheme>.auth.ts`	`OAuth2Auth` in `oauth2.auth.ts`
Utility module	`<domain>.utils.ts`	`pagination.utils.ts`
Fixture	camelCase noun, no `get` prefix	`mockServer`, not `getMockServer`
Factory	`build<Thing>`	`buildOrder`
Assertion method	`expect<Property>`	`expectSecurityHeaders`
Matcher	`to<Property>`	`toSatisfyContract`

Tags

Every test carries at least two: a suite tag and a domain tag. Tags are how a pipeline selects a subset without anybody maintaining a list of file paths.

TAG	MEANS
`@smoke`	Must pass before a deploy. Keep it to a handful — a smoke suite that takes ten minutes is not a smoke suite.
`@regression`	The main body of the suite.
`@contract`	Asserts conformance to a published schema or document.
`@security`	Authorisation, input handling, response hygiene.
`@live`	**Spends real-API quota.** `--grep-invert @live` runs everything else for free.
`@slow`	Legitimately slow — a report, a bulk import. Excluded from the fast loop.
`@read-only`	Safe against a read-only environment; issues no mutating verb.
`@flaky`	Known-unstable and being fixed. Should always have an owner and a ticket.
`@users`, `@orders`, …	The domain area, one per test.

Selecting — SHELL

```shell
npx playwright test --grep @smoke
npx playwright test --grep "@users.*@regression"
npx playwright test --grep-invert "@slow|@flaky"
npx playwright test --grep-invert @live       # spend no API quota
```

Writing a test

The shape TYPESCRIPT

```typescript
import { test, expect } from '../../src/fixtures';

test.describe('user creation', () => {
  test('rejects a duplicate email @regression @users', async ({ api, http, data }) => {
    // Arrange — through the service, which registers its own cleanup.
    const existing = await api.users.create();

    // Act — the raw response, because the assertion is about the status.
    const response = await http
      .post('/users')
      .json({ email: existing.email, firstName: data.person.firstName() })
      .send();

    // Assert — one behaviour, with the reason visible in the message.
    response.expectStatus(409);
    expect(response).toMatchSchema(problemDetails, 'problem-details');
    expect(response.path('detail')).toContain('email');
  });
});
```

- **One behaviour per test.** A test asserting three unrelated things reports the first failure and hides the other two.
- **Arrange through services, assert in the test.** Setup that goes through the service is setup that stays correct when the API changes.
- **Name the test as the behaviour, not the mechanics.** "rejects a duplicate email", not "POST /users returns 409".
- **Prefer `response.parse(Schema)` over `response.json()`.** It makes the shape a checked fact, and everything after it is fully typed.
- **Use `soft()` when asserting many properties of one payload.** One run then reports every violation instead of only the first.

Anti-patterns

INSTEAD OF…	DO THIS	BECAUSE
`await sleep(2000)`	`await waitFor(() => …)`	States the condition, fails with a useful message, and is as fast as the system allows
`http.get('/v1/users/42')` in a test	`api.users.find(42)`	A rename becomes one edit rather than a repository search
`const user = response.json() as User`	`const user = response.parse(UserSchema)`	A cast is a wish; a parse is a check

INSTEAD OF…	DO THIS	BECAUSE
`expect(response.status).toBe(200)`	`response.expectOk()`	The failure message carries the request, timing and body
A hard-coded `test@example.com`	`uniqueId()`	Survives a uniqueness constraint and a second run
Asserting on page one of a list	`followOffset` / `followLinkHeader`	Page two is where pagination bugs live
`test.describe.serial` to fix ordering	Independent tests with their own setup	Serial suites hide the coupling instead of removing it
Sharing state in a worker fixture	Test-scoped fixtures	Shared mutable state makes failures depend on worker count
`expect(errors.length).toBe(0)` on a GraphQL call	`result.expectNoErrors()`	GraphQL answers 200 with the failure in the body; a status assertion passes anyway

TypeScript

- `strict` and `noUncheckedIndexedAccess` are on. An indexed read is `T | undefined` and must be handled — that rule alone prevents a whole class of run-time errors.
- `import type` for type-only imports. Enforced by `consistent-type-imports`.
- Explicit return types on exported functions. The compiler infers them, but a reader should not have to.
- No `any`. Use `unknown` and narrow — that is what `UnknownRecord` and the type guards are for.
- Path aliases (`@core/*`, `@utils/*`) over long relative chains.

Comments

Comment the **why**, never the what. A comment that restates the code is a comment that will drift out of step with it. Every non-obvious decision in this framework carries a comment explaining the alternative that was rejected and the reason — that is the comment that is still useful in two years.

Review checklist

- Does it import from `@fixtures`?
- Is it tagged with a suite tag and a domain tag?
- Does everything it creates get cleaned up?
- Is the data generated rather than hard-coded?
- Is there a sleep? A retried assertion? A cast where a parse would do?
- Does a new endpoint have a registered schema?
- Would the failure message tell somebody at 3am what broke?
- Does a new source file have a documentation entry? (`npm run docs` will tell you.)
- Does `npm run validate` pass?

Troubleshooting

Failure modes, flakiness and the tools for diagnosing them

Failure modes, what actually causes them, and how to find out. Ordered by how often they come up.

Start-up failures

MESSAGE	CAUSE	FIX
`Invalid environment configuration`	A variable is missing or malformed. The message lists each one by name.	Copy `.env.example` to `.env` and fill in the named variables.
`Cannot reach <url>`	`globalSetup` could not open a connection at all.	Check `TEST_ENV`, the URL in `environments.ts`, and whether the target needs a VPN or tunnel.
`No credentials for role "admin"`	`getUser('admin')` was called with no `ADMIN_USERNAME/ADMIN_PASSWORD`.	Set both, or use `hasUser('admin')` to skip rather than fail.
405 with "daily request limit"	The `api.restful-api.dev` anonymous quota (50 requests / 24 h) is spent. Not a defect and not a transport fault.	Nothing — the live suite skips and `tests/contract/objects.stubbed.spec.ts` covers the same lifecycle offline. Use `--grep-invert @live` to skip it deliberately.
`Refusing to send POST …: read-only`	The environment is marked `readOnly` and a test tried to write.	Point `TEST_ENV` at a writable environment, or tag the test `@read-only` and make it read-only in fact.

Authentication

SYMPTOM	LIKELY CAUSE
`invalid_client` from the token endpoint	Client credentials sent in the wrong place. Providers differ; try `clientAuth: 'body'`.
401s that start partway through a run	A token expired mid-run. The store renews 30s early — if the API issues very short tokens, that skew may need raising.
401 on the first request only	The auth provider is producing headers, but the request set its own `authorization`, which always wins.
Session auth logs in on every request	The login response set no cookie matching `cookieNames`. The error lists which cookies actually arrived.
`storage/` session keeps being reused after a password change	Delete `storage/*.json`. The captured session outlives the credential that created it.

Flakiness

An API suite has four common sources of flake, and they have different fixes. Guessing between them is what makes flakiness feel unfixable.

SOURCE	HOW IT LOOKS	FIX
Eventual consistency	A resource is created, then a read of it 404s — intermittently	`waitFor` the read, not a sleep
Shared state between tests	Passes at `--workers=1`, fails at `--workers=8`, and the failure moves	Generate unique data; move mutable state to test scope
Rate limiting	Bursts of 429, often at the start of a run	Lower `workers`, use `mapWithConcurrency` when seeding, and let the client honour `Retry-After`
Genuine instability in the target	Random 502/503 unrelated to the test	Retries already handle it — but raise it with the service owner; a suite is a good early-warning system

Confirming it is ordering, not the test — SHELL

```
npx playwright test --workers=1        # passes → shared state or rate limiting
npx playwright test --repeat-each=5    # fails sometimes → genuinely non-deterministic
```

Contract failures

MESSAGE	MEANING
`violates the registered schema "x"`	The contract guard fired. The API returned something the schema does not allow — either the API changed, or the schema is wrong.
`documents no response for status 4xx`	The OpenAPI document has no entry for that status. Usually the document is incomplete, not the API.
`External $ref … is not supported`	The specification references another file. Bundle it first.
`No operation documented for GET /x`	The suite is calling an endpoint the document does not describe — a real finding, not a tooling problem.

NOTE A contract failure is **never** retried. That is structural: the response listeners run outside the client's retry catch, precisely so a schema violation appears once with a clear message rather than three times.

Streaming

SYMPTOM	CAUSE
A WebSocket test misses the reply it was waiting for	It should not — the client buffers from connect. If it still happens, the predicate does not match the frame; log `client.messages`.
A test hangs at the end and the worker never exits	An SSE stream or socket was left open. Use the `sse` and `socket` fixtures, which close everything on teardown.

SYMPTOM	CAUSE
SSE events never arrive	A proxy is buffering the stream. Confirm with `curl -N`; the framework cannot fix a proxy.
Keep-alive comments arrive as events	They should not — a line starting with `:` is a comment and is dropped by `parseFrame`. If yours differ, the server is not sending valid framing.

Performance results that do not make sense

- **A p95 much worse than the p50** usually means the suite is competing with itself. Performance tests run at `workers: 1` for this reason; check the project actually ran under it.
- **The first sample is always slowest.** That is connection setup and JIT. `sample()` discards warm-up runs by default.
- **Every route has one sample.** `routeOf` collapses identifiers, but only the common shapes. Add yours to it if the routes look like `/users/abc123def`.

Diagnostic tools

TOOL	COMMAND	SHOWS
Debug logs	`LOG_LEVEL=debug LOG_BODIES=true`	Every request, response, retry and backoff decision
Trace viewer	`npx playwright show-trace <trace.zip>`	Every request and response, replayable, with timings
Recording	`reports/recordings/<test>.json`	The full exchange for a failed test, redacted
Latency report	attached to every test	Per-route p50/p90/p95/p99
Run summary	`reports/summary.json`	Totals, pass rate, the ten slowest tests, every failure
Stub-server log	`mockServer.requests`	Exactly what the client sent, including headers and body

Tooling

SYMPTOM	FIX
`program.parse` is undefined, or a dependency exports nothing	A pruned or corrupted install — `npm uninstall` can remove a transitive dependency Playwright needs. `rm -rf node_modules && npm ci`.
Modules load with empty exports after a clean install	A poisoned Node compile cache. Delete `$TMPDIR/node-compile-cache`; it regenerates.
No `summary.json`, no HTML report after a run	`--reporter=` on the command line replaces the config's list. Drop it.
`npm run docs` fails naming a file	That file has no documentation entry. Add one in `scripts/docs/content/`. This is the gate working.
ESLint reports a rule you disagree with	Change it in `eslint.config.mjs`, with a comment explaining why — as the existing exceptions do.

SYMPTOM	FIX
Most tests skip rather than run	Either the API quota is spent, or no credentials are configured for the `setup` project. Both are designed to skip rather than fail; the skip reason says which.

API index

Every exported symbol and where it lives

Every exported symbol in the framework — 148 in total — extracted from the source. Click a file to jump to its reference entry.

SYMBOL	KIND	DEFINED IN	SUMMARY
AccessExpectation	INTERFACE	src/utils/security.utils.ts	One cell of an authorisation matrix: who, doing what, and what should happen.
ApiKeyAuth	CLASS	src/auth/static.auth.ts	API-key authentication.
ApiObjectSchema	CONST	src/services/object.service.ts	A read: `GET /objects/{id}`.
ApiResponse	CLASS	src/core/api.response.ts	A completed HTTP exchange.
apiUrl	FUNCTION	src/config/env.config.ts	Absolute URL for a path, applying the environment's version prefix.
architecture	CONST	scripts/docs/content/guides.mjs	
auditCors	FUNCTION	src/utils/security.utils.ts	Checks a CORS preflight.
auditDisclosure	FUNCTION	src/utils/security.utils.ts	Checks a response for information disclosure.
auditSecurityHeaders	FUNCTION	src/utils/security.utils.ts	Checks the baseline security headers a public API should send.
AuthenticationError	CLASS	src/core/errors.ts	A token could not be obtained or refreshed.
AuthProvider	INTERFACE	src/core/http.client.ts	Supplies credentials for outgoing requests.
BaseService	CLASS	src/core/base.service.ts	
BasicAuth	CLASS	src/auth/static.auth.ts	HTTP Basic — `Authorization: Basic base64(user:pass)`.
BodyKind	TYPE	src/types/index.ts	How the request body is encoded on the wire.
buildAccessMatrix	FUNCTION	src/utils/security.utils.ts	Builds the full matrix from a list of roles and operations.
buildUser	CONST	src/utils/data.utils.ts	A representative user payload.

SYMBOL	KIND	DEFINED IN	SUMMARY
CachedToken	INTERFACE	src/auth/token.store.ts	A token with the moment it stops being usable.
CleanupRegistry	CLASS	src/utils/cleanup.registry.ts	
config	CONST	src/config/env.config.ts	Resolved, validated configuration.
ConfigurationError	CLASS	src/core/errors.ts	The environment or .env file is wrong.
ContractViolationError	CLASS	src/core/errors.ts	The response disagreed with the OpenAPI document for that operation.
conventions	CONST	scripts/docs/content/guides.mjs	
CreatedObjectSchema	CONST	src/services/object.service.ts	A create response: the only one that carries createdAt.
decodeJwt	FUNCTION	src/auth/jwt.ts	Splits and base64url-decodes a token.
DeleteAcknowledgementSchema	CONST	src/services/object.service.ts	The delete acknowledgement.
detectLanguage	FUNCTION	scripts/docs/highlight.mjs	
diff	FUNCTION	src/utils/diff.utils.ts	Every difference between two payloads.
DiffOptions	INTERFACE	src/utils/diff.utils.ts	
EDGE_CASE_STRINGS	CONST	src/utils/data.utils.ts	Strings that break naive input handling.
ENVIRONMENT_NAMES	CONST	src/config/environments.ts	
EnvironmentDefinition	INTERFACE	src/config/environments.ts	A deployment the suite can be pointed at.
EnvironmentName	TYPE	src/config/environments.ts	
ENVIRONMENTS	CONST	src/config/environments.ts	
esc	FUNCTION	scripts/docs/render.mjs	
ExchangeRecord	INTERFACE	src/types/index.ts	One completed exchange, as written to the run log and the HTML report.

SYMBOL	KIND	DEFINED IN	SUMMARY
ExchangeRecorder	CLASS	src/mocks/recorder.ts	
expect	CONST	src/fixtures/custom-matchers.ts	`expect.extend` returns a *new* `expect` carrying the matchers' types, which is why the result is captured rather than discarded.
fileAnchor	FUNCTION	scripts/docs/render.mjs	
findPaginationDefects	FUNCTION	src/utils/pagination.utils.ts	Checks a paginated endpoint for the two defects pagination tests exist to catch: an item that appears on two pages, and one that appears on none.
findSchema	FUNCTION	src/contracts/schema.registry.ts	The schema for a request/status pair, or `undefined` when none is registered.
followLinkHeader	FUNCTION	src/utils/pagination.utils.ts	Follows `Link: rel="next"` until the server stops sending one.
followOffset	FUNCTION	src/utils/pagination.utils.ts	Follows offset/limit pagination until fewer than `limit` items come back.
formatDiff	FUNCTION	src/utils/diff.utils.ts	A readable multi-line rendering, for an assertion message.
formatFindings	FUNCTION	src/utils/security.utils.ts	Formats findings for an assertion message or a report attachment.
fromRoot	FUNCTION	src/utils/file.utils.ts	Resolves a path relative to the repository root.
getHeader	FUNCTION	src/utils/header.utils.ts	Case-insensitive single header lookup.
getUser	FUNCTION	src/config/env.config.ts	Credentials for a role, or a precise error naming the two variables to set.
globalSetup	FUNCTION	src/hooks/global.setup.ts	
globalTeardown	FUNCTION	src/hooks/global.teardown.ts	
glossary	CONST	scripts/docs/content/guides.mjs	
GraphQlClient	CLASS	src/protocols/graphql.client.ts	
hasPath	FUNCTION	src/utils/jsonpath.utils.ts	True when the path resolves to anything other than `undefined`.
hasUser	FUNCTION	src/config/env.config.ts	True when a role has usable credentials — use to skip rather than fail.

SYMBOL	KIND	DEFINED IN	SUMMARY
HeaderMap	TYPE	src/types/index.ts	Header names are compared case-insensitively throughout the framework.
highlight	FUNCTION	scripts/docs/highlight.mjs	Highlights a snippet.
HttpClient	CLASS	src/core/http.client.ts	
HttpError	CLASS	src/core/errors.ts	A response arrived, but not one the caller was willing to accept.
HttpMethod	TYPE	src/types/index.ts	Every verb the client can issue.
identifier	CONST	src/contracts/schemas.ts	An identifier that may be a UUID or a numeric string — very common.
INJECTION_PAYLOADS	CONST	src/utils/data.utils.ts	Payloads a security test sends where a normal value is expected.
inline	FUNCTION	scripts/docs/render.mjs	Inline markup: code, bold, text.
isJsonContentType	FUNCTION	src/utils/header.utils.ts	True when a content type is JSON, including +json suffixes like HAL.
isoDateTime	CONST	src/contracts/schemas.ts	ISO 8601 instant, e.g.
judgeAccess	FUNCTION	src/utils/security.utils.ts	Judges an authorisation response.
LANGUAGE_LABEL	CONST	scripts/docs/highlight.mjs	
LatencyCollector	CLASS	src/utils/performance.utils.ts	Collects timings across a test or a whole run.
LatencySummary	INTERFACE	src/utils/performance.utils.ts	A summary of many samples.
leafPaths	FUNCTION	src/utils/jsonpath.utils.ts	Every leaf path in a payload, as dotted strings.
logger	CONST	src/utils/logger.ts	The shared logger.
Logger	CLASS	src/utils/logger.ts	
looksLikeXml	FUNCTION	src/utils/xml.utils.ts	True when a body looks like XML, so the response wrapper can pick a parser.
matchesPath	FUNCTION	src/contracts/schema.registry.ts	Compares a {param} pattern with a concrete path, ignoring any version prefix so one registration works across /v1/users and /users.
MockServer	CLASS	src/mocks/mock.server.ts	

SYMBOL	KIND	DEFINED IN	SUMMARY
`MultipartPart`	INTERFACE	`src/types/index.ts`	One part of a `multipart/form-data` upload.
`MUTATING_METHODS`	CONST	`src/types/index.ts`	Verbs that must not run against a read-only environment.
`nextLink`	FUNCTION	`src/utils/header.utils.ts`	The `next` URL from a `Link` header, when the server sent one.
`NoAuth`	CLASS	`src/auth/static.auth.ts`	Sends no credentials.
`OAuth2Auth`	CLASS	`src/auth/oauth2.auth.ts`	
`ObjectErrorSchema`	CONST	`src/services/object.service.ts`	The API's error shape.
`ObjectService`	CLASS	`src/services/object.service.ts`	
`offsetPage`	FUNCTION	`src/contracts/schemas.ts`	Offset/limit pagination, the most common REST convention.
`OpenApiContract`	CLASS	`src/contracts/openapi.ts`	
`overview`	CONST	`scripts/docs/content/guides.mjs`	Authored prose for the narrative pages.
`parseContentType`	FUNCTION	`src/utils/header.utils.ts`	
`ParsedCookie`	INTERFACE	`src/utils/header.utils.ts`	One `Set-Cookie` entry, split into its name, value and attributes.
`parseLinkHeader`	FUNCTION	`src/utils/header.utils.ts`	Parses `Link: <…>; rel="next", <…>; rel="last"` — GitHub-style pagination.
`parseRateLimit`	FUNCTION	`src/utils/header.utils.ts`	
`parseRetryAfter`	FUNCTION	`src/utils/header.utils.ts`	`Retry-After` in milliseconds.
`parseSetCookie`	FUNCTION	`src/utils/header.utils.ts`	Parses `Set-Cookie`.
`parseXml`	FUNCTION	`src/utils/xml.utils.ts`	Parses an XML document into a plain object.
`PathParams`	TYPE	`src/types/index.ts`	Values substituted into a templated path such as `/users/{id}`.
`percentile`	FUNCTION	`src/utils/performance.utils.ts`	A percentile using nearest-rank on a sorted sample.

SYMBOL	KIND	DEFINED IN	SUMMARY
playbooks	CONST	scripts/docs/content/guides.mjs	
PollTimeoutError	CLASS	src/core/errors.ts	A waitFor-style helper gave up before its condition became true.
PostListSchema	CONST	src/services/post.service.ts	
PostSchema	CONST	src/services/post.service.ts	
PostService	CLASS	src/services/post.service.ts	
PUBLIC_APIS	CONST	src/config/environments.ts	Additional public APIs the demonstration suite reaches directly.
QueryParams	TYPE	src/types/index.ts	
QueryValue	TYPE	src/types/index.ts	A single query-string value.
RateLimitInfo	INTERFACE	src/utils/header.utils.ts	Rate-limit counters, using the widely adopted X-RateLimit-* convention.
readAll	FUNCTION	src/utils/jsonpath.utils.ts	Reads every value a path matches.
ReadOnlyEnvironmentError	CLASS	src/core/errors.ts	A mutating verb was attempted against an environment marked read-only.
readPath	FUNCTION	src/utils/jsonpath.utils.ts	Reads a single value.
RecordedRequest	INTERFACE	src/mocks/mock.server.ts	A request the server received, kept so tests can assert on it.
redactHeaders	FUNCTION	src/utils/header.utils.ts	
registerSchemas	FUNCTION	src/contracts/schema.registry.ts	Registers many at once — the usual form for a service's contract file.
renderBlocks	FUNCTION	scripts/docs/render.mjs	
renderFileEntry	FUNCTION	scripts/docs/render.mjs	Renders one documented file: purpose, change guidance and API.
RequestBuilder	CLASS	src/core/request.builder.ts	
RequestSender	INTERFACE	src/core/request.builder.ts	Implemented by the HTTP client.
RequestSpec	INTERFACE	src/types/index.ts	A fully resolved request, ready to be sent.

SYMBOL	KIND	DEFINED IN	SUMMARY
RequestTimeoutError	CLASS	src/core/errors.ts	The request never completed within its timeout.
RequestTiming	INTERFACE	src/types/index.ts	Wall-clock timings captured for every request.
ResponseSnapshot	INTERFACE	src/core/api.response.ts	Everything captured from the wire.
safeJson	FUNCTION	src/core/errors.ts	JSON.stringify that never throws on cycles or BigInt.
sample	FUNCTION	src/utils/performance.utils.ts	Runs an operation repeatedly and summarises the result.
SchemaValidationError	CLASS	src/core/errors.ts	A payload did not match the schema it was validated against.
SecurityFinding	INTERFACE	src/utils/security.utils.ts	One thing worth fixing, with enough context to act on it.
SEEDED_COUNT	CONST	src/mocks/objects.stub.ts	How many objects the live list endpoint returns.
seededFaker	FUNCTION	src/utils/data.utils.ts	A Faker instance seeded from a string, so a title maps to stable data.
slug	FUNCTION	scripts/docs/render.mjs	
soapFault	FUNCTION	src/utils/xml.utils.ts	
SoapFault	INTERFACE	src/utils/xml.utils.ts	A SOAP <Fault> when the response carries one.
SseClient	CLASS	src/protocols/sse.client.ts	
SseOptions	INTERFACE	src/protocols/sse.client.ts	
Stub	INTERFACE	src/mocks/mock.server.ts	One registered stub.
stubFromRecording	FUNCTION	src/mocks/mock.server.ts	A stub built from a recorded exchange — see recorder.ts.
stubObjectsApi	FUNCTION	src/mocks/objects.stub.ts	Registers the whole /objects contract on a stub server.
StubResponse	INTERFACE	src/mocks/mock.server.ts	What a stub sends back.
summarize	FUNCTION	src/utils/performance.utils.ts	Summarises a set of durations in milliseconds.

SYMBOL	KIND	DEFINED IN	SUMMARY
SummaryReporter	CLASS	src/reporters/summary.reporter.ts	
test	CONST	src/fixtures/api.fixture.ts	
TIMEOUTS	CONST	src/config/timeouts.ts	Named time budgets.
tokenStore	CONST	src/auth/token.store.ts	Process-wide store.
TokenStore	CLASS	src/auth/token.store.ts	
TransportError	CLASS	src/core/errors.ts	The transport failed: DNS, TLS, connection reset, unreachable host.
troubleshooting	CONST	scripts/docs/content/guides.mjs	
truncate	FUNCTION	src/core/errors.ts	Keeps error messages readable when a server returns a very large body.
uniqueId	FUNCTION	src/utils/data.utils.ts	A run-unique identifier, safe to embed in names that must not collide.
UnknownRecord	TYPE	src/types/index.ts	A plain object with unknown values — safer than any for parsed payloads.
UserRole	TYPE	src/config/env.config.ts	
UserService	CLASS	src/services/template.service.ts	
validateJsonSchema	FUNCTION	src/contracts/json-schema.ts	Validates a value against a JSON Schema document.
ValidationResult	INTERFACE	src/types/index.ts	Outcome of a schema or contract check.
WebSocketClient	CLASS	src/protocols/websocket.client.ts	
WebSocketOptions	INTERFACE	src/protocols/websocket.client.ts	
workedExample	CONST	scripts/docs/content/guides.mjs	

Glossary

Terms used in the code and in this document

Terms used in the code and in this document, in the sense this framework uses them.

TERM	MEANING HERE
Service object	A class that owns one resource's paths and payloads and exposes domain methods. The API equivalent of a page object.
Fixture	A Playwright-managed dependency, set up before a test and torn down after. Constructed only if the test names it.
Worker scope	A fixture built once per worker process and shared by every test in it.
Snapshot	The complete captured response — status, headers, body buffer, timing — from which `ApiResponse` is built.
Verdict	A response the API meant to send: a 422, a 403. Never retried, as distinct from a transport fault.
Transport fault	A failure before any response arrived: DNS, TLS, reset, timeout. Retried with backoff.
Contract guard	The automatic fixture that validates every response against its registered schema when `STRICT_CONTRACTS` is on.
Schema registry	The map from method + path pattern + status to a schema, which is what lets the guard find the right one.
Idempotency key	A header promising the server that a repeated request is the same request — what makes retrying a POST safe.
Eventual consistency	A write that is acknowledged before it is visible everywhere. The main cause of flaky API tests.
Read-only environment	One flagged in `environments.ts` where the client refuses to send any mutating verb.
Problem details	RFC 9457, the standard error body: `type`, `title`, `status`, `detail`, `instance`.
Cursor pagination	Paging by an opaque token rather than an offset, so results stay stable while the underlying data changes.
Link header	RFC 8288 pagination, where the server sends the next page's URL and the client needs no knowledge of its parameters.
SSE	Server-Sent Events: a one-way `text/event-stream` where events are `field: value` lines separated by blank lines.
NDJSON	Newline-delimited JSON — one complete JSON value per line, used for streaming and bulk export.
Shape	Which paths exist in a payload and of what type, ignoring values. What a baseline comparison should actually compare.
Breaking change	In shape terms: a field removed, or a field whose type changed. Adding a field is not one.
Access matrix	The full grid of role × operation with the expected outcome — generated, not hand-written, so omissions are caught.

TERM	MEANING HERE
Redaction	Replacing a credential with a correlatable stub before it reaches a log, a report or a recording.
Blob report	Playwright's intermediate per-shard report format, merged into one HTML report after a sharded run.
Executable model	An in-memory implementation of a real API's contract, driven by the same service object, so exhaustive coverage can run offline and unlimited.
Echo server	A service that returns exactly what it received. Neutral in a way a stub is not, because a stub shares its author with the client it is testing.
Quota refusal	A rate-limit response that is neither a defect nor a transport fault. Detected and skipped, because failing on it teaches people to ignore the suite.
Flaky	Here: passed only after a retry. Counted separately from passed by the summary reporter, so retries cannot hide rot.